

Khả năng tái tạo và phần mềm

ONE LOVE. ONE FUTURE.

Trong bài giảng này



Khủng hoảng khả năng tái

tạo Đó là gì?

Tệ đến mức nào?



Phần mềm có thể giúp ích gì?

Sử dụng khi nào?

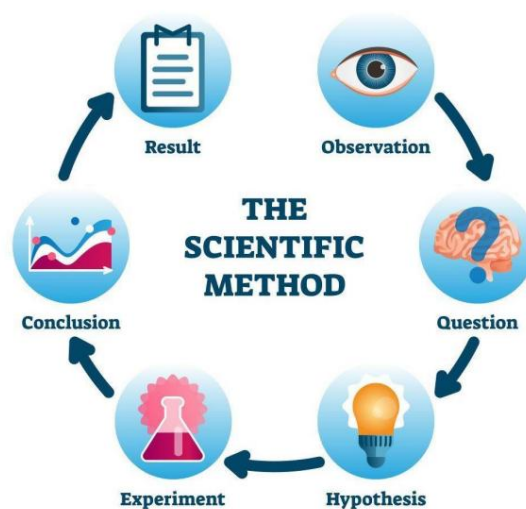
```
8 // Dear programmer:
9 // When I wrote this code, only god and
10 // I knew how it worked.
11 // Now, only god knows it!
12 //
13 // Therefore, if you are trying to optimize
14 // this routine and it fails (most surely),
15 // please increase this counter as a
16 // warning for the next person:
17 //
18 // total_hours_wasted_here = 254
19 //
20
```

Khả năng tái tạo là gì?

- Khả năng tái tạo là khả năng của toàn bộ một thí nghiệm hoặc nghiên cứu có thể được sao chép, bởi cùng một nhà nghiên cứu hoặc bởi một người khác làm việc độc lập.
- Dữ liệu có thể tái tạo - khả năng lặp lại là mức độ thống nhất của các thử nghiệm hoặc phép đo trên các mẫu vật sao chép của cùng một người quan sát trong cùng một phòng thí nghiệm.
- Nghiên cứu có thể tái tạo bằng máy tính - ý tưởng rằng sản phẩm cuối cùng của nghiên cứu học thuật là bài báo cùng với môi trường tính toán đầy đủ được sử dụng để tạo ra kết quả trong bài báo như mã, dữ liệu, v.v. có thể được sử dụng để tái tạo kết quả và tạo ra công trình mới dựa trên nghiên cứu.



Tại sao chúng ta cần nó?



Khả năng tái tạo có liên quan gì đến MLOps?

- Bảo tồn kiến thức • Nếu người

khác không thể sao chép công trình của bạn, kiến thức có thể bị mất

- Minh bạch và Trách nhiệm giải trình • Để đảm

bảo rằng những người khác có thể xác minh các khiếu nại của bạn trước khi đưa vào sản xuất • Tuân thủ quy

định • Để bảo đảm tài liệu chính

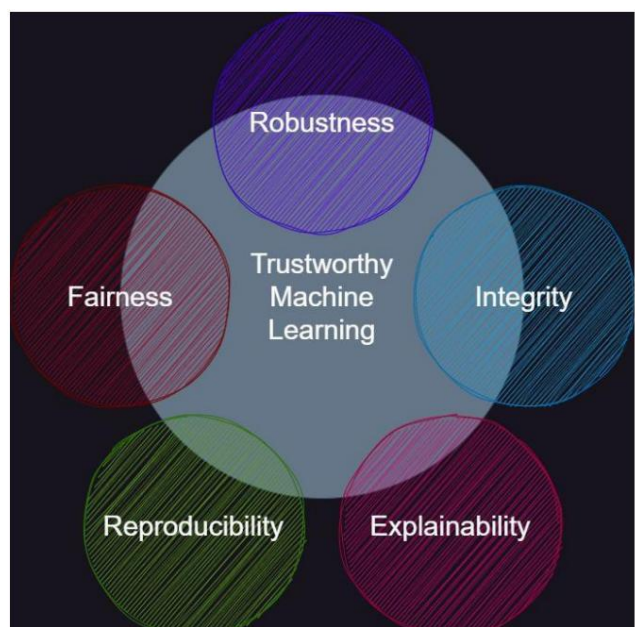
xác nhằm đảm bảo mọi thứ đều tuân thủ • Cải tiến liên tục

- Đảm bảo rằng những cải tiến đối với đường ống là có thật chứ không phải là hiện vật của hiệu ứng ngẫu nhiên



ML đáng tin cậy

- Khả năng tái tạo là một thành phần quan trọng trong ML đáng tin cậy
- Hình ảnh một tác nhân AI được sử dụng để chẩn đoán. Nếu không có khả năng tái tạo, hai người có cùng triệu chứng có thể được chẩn đoán khác nhau

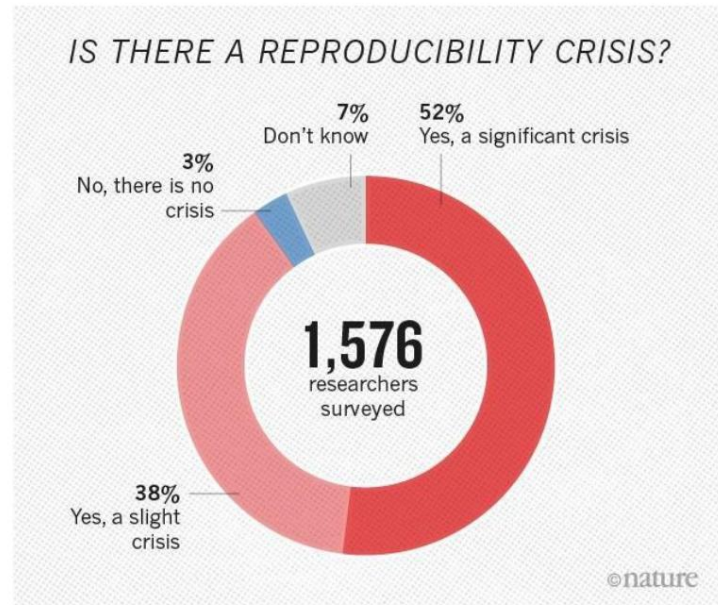


Chúng ta đang trong cơn khủng hoảng!



Có sự báo động ngày càng tăng về những kết quả không thể tái tạo được. Giải thích bao gồm

- Tăng cường mức độ giám sát • Độ phức tạp của các thí nghiệm và thống kê
- Áp lực lên các nhà nghiên cứu



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

7

Một ví dụ: Rắc rối trong phòng thí nghiệm

Công ty công nghệ sinh học Amgen có một nhóm gồm khoảng 100 nhà khoa học đang cố gắng tái tạo các phát hiện của 53 bài báo "mang tính bước ngoặt" về nghiên cứu ung thư được các phòng thí nghiệm có uy tín công bố trên các tạp chí hàng đầu.

[Chỉ có 6 trong số 53 nghiên cứu được sao chép](#)¹ (khoảng 10%).

REPRODUCIBILITY OF RESEARCH FINDINGS

Preclinical research generates many secondary publications, even when results cannot be reproduced.

Journal impact factor	Number of articles	Mean number of citations of non-reproduced articles*	Mean number of citations of reproduced articles
>20	21	248 (range 3–800)	231 (range 82–519)
5–19	32	169 (range 6–1,909)	13 (range 3–24)

Results from ten-year retrospective analysis of experiments performed prospectively. The term 'non-reproduced' was assigned on the basis of findings not being sufficiently robust to drive a drug-development programme.

*Source of citations: Google Scholar, May 2011.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

[1] <https://www.reuters.com/article/2012/03/28/us-science-cancer-idUSBRE82R12P20120328/>

8

Một ví dụ khác: Chúng ta chỉ là con người

Những lựa chọn (chủ quan) mà chúng ta đưa ra trong quá trình nghiên cứu có thể tác động đến kết luận

ONE DATA SET, MANY ANALYSTS

Twenty-nine research teams reached a wide variety of conclusions using different methods on the same data set to answer the same question (about football players' skin colour and red cards).

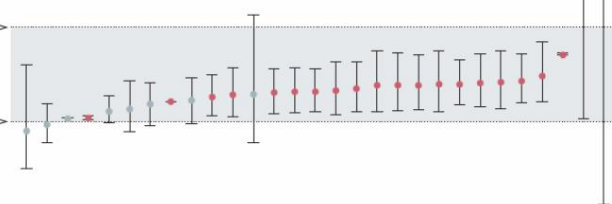


Dark-skinned players four times more likely than light-skinned players to be given a red card.

- Statistically significant effect
- Non-significant effect

Twice as likely

Equally likely



Point estimates and 95% confidence intervals. *Truncated upper bounds.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

9

Chúng ta đang cố gắng làm gì trong nghiên cứu?

<https://paperswithcode.com/>

ML Reproducibility Challenge 2021 Spring

RC2021Spring

Online: Jul 20 2021 <https://paperswithcode.com/rc2020> reproducibility.challenge@gmail.com

Please see the venue website for more information.
Submission Start: Apr 01 2021 12:00AM UTC-6, End: TBD UTC-6

Add: ML Reproducibility Challenge 2021 Spring Submission

All Papers Claimed

Search by paper title and metadata

Conference: All Conferences

ToTTo: A Controlled Table-To-Text Generation Dataset
Ankur P. Parikh, Suzhi Wang, Sebastian Gehrmann, Manal Faruqi, Bhuvan Dhingra, Diyi Yang, Dipanjan Das
24 Nov 2020 EMNLP 2020 Readers: Everyone 0 Replies

Tired of Topic Models? Clusters of Pretrained Word Embeddings Make for Fast and Good Topics too!
Suzanna Sia, Ayush Dubey, Sabrina J. Michel
24 Nov 2020 EMNLP 2020 Readers: Everyone 0 Replies

Towards Interpreting BERT for Reading Comprehension Based QA
Sahana Ramnath, Preksha Nema, Deep Sahni, Mitesh M. Khapra
24 Nov 2020 EMNLP 2020 Readers: Everyone 1 Reply

Dialogue Response Ranking Training with Large-Scale Human Feedback Data
Xiang Gao, Yizhe Zhang, Michel Galley, Chris Brockett, Bill Dolan
24 Nov 2020 EMNLP 2020 Readers: Everyone 0 Replies

Data Rejuvenation: Exploiting Inactive Training Examples for Neural Machine Translation
Wenxiang Jiao, Xing Wang, Shilin He, Irwin King, Michael R. Lyu, Zhaopeng Tu
24 Nov 2020 EMNLP 2020 Readers: Everyone 0 Replies

Danh sách kiểm tra cho hội nghị

3. If you ran experiments...

- (a) Did you include the code, data, and instructions needed to **reproduce** the main experimental results (either in the supplemental material or as a URL)?
 - The instructions should contain the exact command and environment needed to run to reproduce the results.
 - Please see the [NeurIPS code and data submission guidelines](#) for more details.
 - Main experimental results include your new method and baselines. You should try to capture as many of the minor experiments in the paper as possible. If a subset of experiments are reproducible, you should state which ones are.
 - While we encourage release of code and data, we understand that this might not be possible, so "no because the code is proprietary" is an acceptable answer.
 - At submission time, to preserve anonymity, remember to release anonymized versions.
- (b) Did you specify all the **training details** (e.g., data splits, hyperparameters, how they were chosen)?
 - The full details can be provided with the code, but the important details should be in the main paper.
- (c) Did you report **error bars** (e.g., with respect to the random seed after running experiments multiple times)?
 - Answer "yes" if you report error bars, confidence intervals, or statistical significance tests for your main experiments.
- (d) Did you include the amount of **compute** and the type of **resources** used (e.g., type of GPUs, internal cluster, or cloud provider)?
 - Ideally, you would provide the compute required for each of the individual experimental runs as well as the total compute.
 - Note that your full research project might have required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper). The total compute used may be harder to characterize, but if you can do that, that would be even better.
 - You are also encouraged to use a CO2 emissions tracker and provide that information. See, for example, the [experiment impact tracker](#) (Henderson et al.), the [ML CO2 impact calculator](#) (Lacoste et al.), and [CodeCarbon](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

10

Nhìn kỹ hơn vào máy học

⚡ Thực hiện lại bài báo 255. Kiểm định giả thuyết về
 “Đặc điểm của giấy” nào có ảnh hưởng đến khả năng tái tạo?

A Step Toward Quantifying Independently Reproducible Machine Learning Research

Edward Raff
 Booz Allen Hamilton
 raff.edward@bah.com
 University of Maryland, Baltimore County
 raff.edward@umbc.edu

Abstract

What makes a paper independently reproducible? Debates on reproducibility center around intuition or assumptions but lack empirical results. Our field focuses on releasing code, which is important, but is not sufficient for determining reproducibility. We take the first step toward a quantifiable answer by manually attempting to implement 255 papers published from 1984 until 2017, recording features of each paper, and performing statistical analysis of the results. For each paper, we did not look at the authors code, if released, in order to prevent bias toward discrepancies between code and paper.

Table 1: Significance test of which paper properties impact reproducibility. Results significant at $\alpha \leq 0.05$ marked with “*”.

Feature	p-value
Year Published	0.964
Year First Attempted	0.674
Venue Type	0.631
Rigor vs Empirical*	1.55×10^{-9}
Has Appendix	0.330
Looks Intimidating	0.829
Readability*	9.68×10^{-25}
Algorithm Difficulty*	2.94×10^{-5}
Pseudo Code*	2.31×10^{-4}
Primary Topic*	7.039×10^{-4}
Exemplar Problem	0.720
Compute Specified	0.257
Hyperparameters Specified*	8.45×10^{-6}
Compute Needed*	8.75×10^{-5}
Authors Reply*	6.01×10^{-8}
Code Available	0.213
Pages	0.364
Publication Venue	0.342
Number of References	0.740
Number Equations*	0.004
Number Proofs	0.130
Number Tables*	0.010
Number Graphs/Plots	0.139
Number Other Figures	0.217
Conceptualization Figures	0.365
Number of Authors	0.497



ĐẠI HỌC BÁCH KHOA HÀ NỘI
 HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

11

Mức độ tái tạo

Khả năng tái tạo không phải là nhị phân, mà là một quang phổ

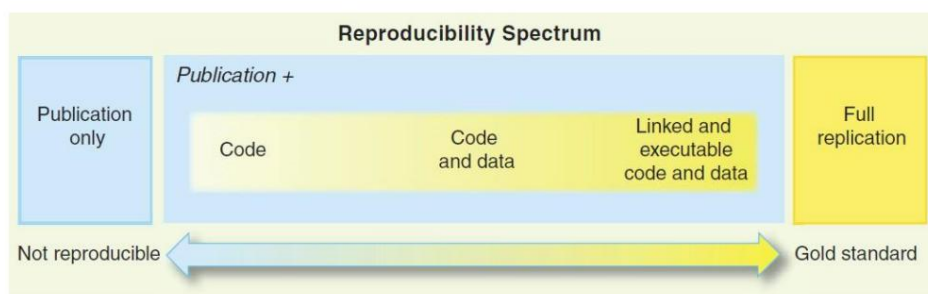


Fig. 1. The spectrum of reproducibility.

Ví dụ từ ML

$$\begin{Bmatrix} 1 \\ \end{Bmatrix} == \begin{Bmatrix} 1 \\ \end{Bmatrix}$$

m1 ckpt m2 ckpt

VS

Dataset	Model Architecture	Random Init	Transfer	Parameters	ImageNet Top5
RETINA	Resnet-50	96.4% ± 0.05	96.7% ± 0.04	23570408	92.2% ± 0.06
RETINA	Inception-v3	96.6% ± 0.13	96.7% ± 0.05	22881424	93.9%
RETINA	CBR-LargeT	96.2% ± 0.04	96.2% ± 0.04	8532480	77.5% ± 0.03
RETINA	CBR-LargeW	95.8% ± 0.04	95.8% ± 0.05	8432128	75.1% ± 0.3
RETINA	CBR-Small	95.7% ± 0.04	95.8% ± 0.01	2108672	67.6% ± 0.3
RETINA	CBR-Tiny	95.8% ± 0.03	95.8% ± 0.01	1076480	73.5% ± 0.05



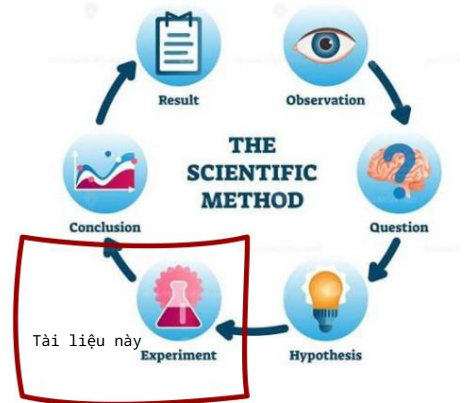
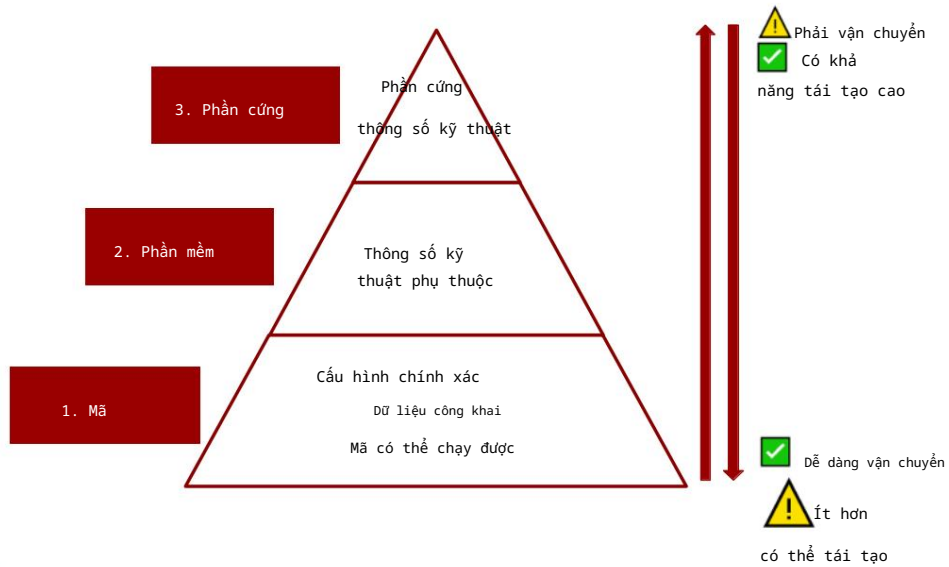
ĐẠI HỌC BÁCH KHOA HÀ NỘI
 HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

12

Chúng ta có thể làm gì về vấn đề này?

Hãy đảm bảo rằng bạn ghi lại mọi thứ về các thí nghiệm của mình

🔥 Hệ thống phân cấp khả năng tái tạo ML của Nicki 🔥



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

13

Mức độ tái tạo 1

⚡ Có rất nhiều lựa chọn chủ quan mà chúng ta thực hiện khi chạy thử nghiệm trong máy học, đáng chú ý nhất là các siêu tham số.

Các tham số trong tập lệnh Trình phân tích đối số	Các tập tin cấu hình
<pre>class hparams: lr = 0.1 batch_size = 16 num_layers = 5</pre>	<pre>python my_script.py \ --lr 0.1 \ --batch_size 16 \ --num_layers 5</pre>
<pre>experiment1.yaml lr: 0.001 batch_size: 16 num_layers: 5 python my_script.py \ config=experiment1.yaml</pre>	
✅ Dễ dàng để mã hóa ⚠️ Không dễ để cấu hình khi đang chạy ⚠️ Thông tin thử nghiệm có thể bị mất nếu không cẩn thận	✅ Dễ dàng cấu hình ⚠️ Rơi vào người dùng để lưu cấu hình
	✅ Có thể cấu hình cao ✅ Cấu hình được lưu một cách có hệ thống (và được kiểm soát phiên bản)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

14

Mức độ tái tạo 1



Hydra là một khuôn khổ để cấu hình các ứng dụng phức tạp (ML) một cách tinh tế <https://github.com/facebookresearch/hydra>

Ví dụ:

```
├── conf
│   ├── config.yaml
│   └── dataset
│       ├── cifar10.yaml
│       └── imagenet.yaml
└── my_app.py
```

```
import hydra
from omegaconf import DictConfig

@hydra.main(config_path="config.yaml")
def my_app(cfg: DictConfig) -> None:
    print(cfg.pretty())

if __name__ == "__main__":
    my_app()
```

Các tùy chọn

💡 khác <https://github.com/IDSIA/>

💡 sacred <https://mlflow.org/>



Mức độ tái tạo 2

Đối với python: Chỉ cần sử dụng hệ thống quản lý gói

Ví dụ:

💡 [Conda](#)

💡 [Ông dẫn](#)

💡 [venv](#)

💡 [pyenv](#)

```
(lightning) C:\Users\nsde\Documents\metrics>conda env list
# conda environments:
#
base                  C:\Users\nsde\Anaconda3
ensemble              C:\Users\nsde\Anaconda3\envs\ensemble
laplace               C:\Users\nsde\Anaconda3\envs\laplace
lightning             * C:\Users\nsde\Anaconda3\envs\lightning
mixerensemble         C:\Users\nsde\Anaconda3\envs\mixerensemble
mlops                 C:\Users\nsde\Anaconda3\envs\mlops
protein               C:\Users\nsde\Anaconda3\envs\protein
pvae                  C:\Users\nsde\Anaconda3\envs\pvae
stochman              C:\Users\nsde\Anaconda3\envs\stochman

(lightning) C:\Users\nsde\Documents\metrics>
```

```
(lightning) C:\Users\nsde\Documents\metrics>conda list
# packages in environment at C:\Users\nsde\Anaconda3\envs\lightning:
#
# Name                    Version                    Build                Channel
abs1-py                   1.2.0                     pypi_0               pypi
aiosignal                 1.2.0                     pypi_0               pypi
aiosignal                 1.2.0                     pypi_0               pypi
alabaster                 0.7.12                    pypi_0               pypi
asttokens                 2.0.5                     pypi_0               pypi
async-timeout             4.0.2                     pypi_0               pypi
atomicwrites              1.4.1                     pypi_0               pypi
attrs                     22.1.0                    pypi_0               pypi
babel                     2.10.3                    pypi_0               pypi
backcall                  0.2.0                     pyhd3eb1b0_0         pypi
beautifulsoup4            4.11.1                    pypi_0               pypi
black                     22.8.0                    pypi_0               pypi
blas                      2.116                     mkl                  conda-forge
blas-devel                3.9.0                     16_win64_mkl         conda-forge
bleach                    5.0.1                     pypi_0               pypi
brotlipy                  0.7.0                     py38h294d835_1004    conda-forge
build                     0.8.0                     pypi_0               pypi
ca-certificates           2022.07.19                haa95532_0           pypi
cachetools                5.2.0                     py38haa95532_0       pypi
certifi                   2022.9.14                 py38haa95532_0       pypi
cffi                      1.15.1                    py38h294d835_1004    conda-forge
cfv                       3.3.1                     pypi_0               pypi
charset-normalizer        2.1.1                     pyhd8ed1ab_0         conda-forge
check-manifest            0.48                       pypi_0               pypi
click                     8.1.3                     pypi_0               pypi
cloudpickle               2.2.0                     pypi_0               pypi
colorama                  0.4.5                     py38haa95532_0       pypi
commonmark                0.9.1                     pypi_0               pypi
contourpy                 1.0.5                     pypi_0               pypi
coverage                  6.4.4                     pypi_0               pypi
cryptography              37.0.4                    py38h7941b4_0        conda-forge
cudatoolkit              11.6.0                    hcbea762_10          conda-forge
cycler                    0.11.0                    pypi_0               pypi
decorator                 5.1.1                     pyhd3eb1b0_0         pypi
defusedxml                0.7.1                     pypi_0               pypi
distlib                   0.3.6                     pypi_0               pypi
docutils                  0.17.1                    pypi_0               pypi
dython                    0.7.2                     pypi_0               pypi
```



Mức độ tái tạo 3

💡 Cách dễ nhất để ai đó sao chép tác phẩm của bạn là đưa cho họ máy tính của bạn.

💡 Thay vì làm điều này, chúng ta hãy trao cho nhau một bản sao ảo của máy chúng ta



Máy ảo hoạt động bằng cách sử dụng phần cứng từ máy chủ và tạo CPU, RAM ảo, lưu trữ cho mỗi máy ảo. Máy ảo máy móc hoàn toàn độc lập với chủ nhà



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

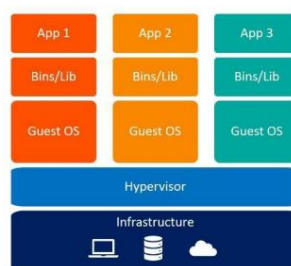
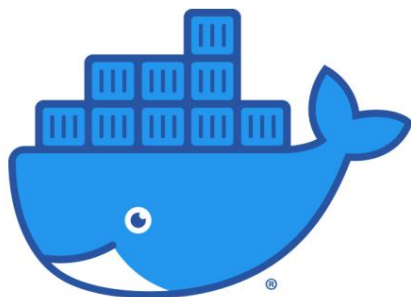
17

Mức độ tái tạo 3

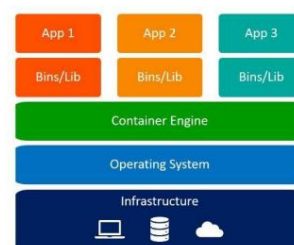
💡 Ưu điểm cốt lõi của VM là về nguyên tắc nó có thể chạy trên bất kỳ máy chủ nào mà không cần thay đổi vì nó độc lập.

💡 Docker có thể được coi là phiên bản nhẹ hơn của VM đầy đủ.

💡 VM là sự cô lập của máy móc, trong khi Container là sự cô lập của các quy trình



Virtual Machines



Containers

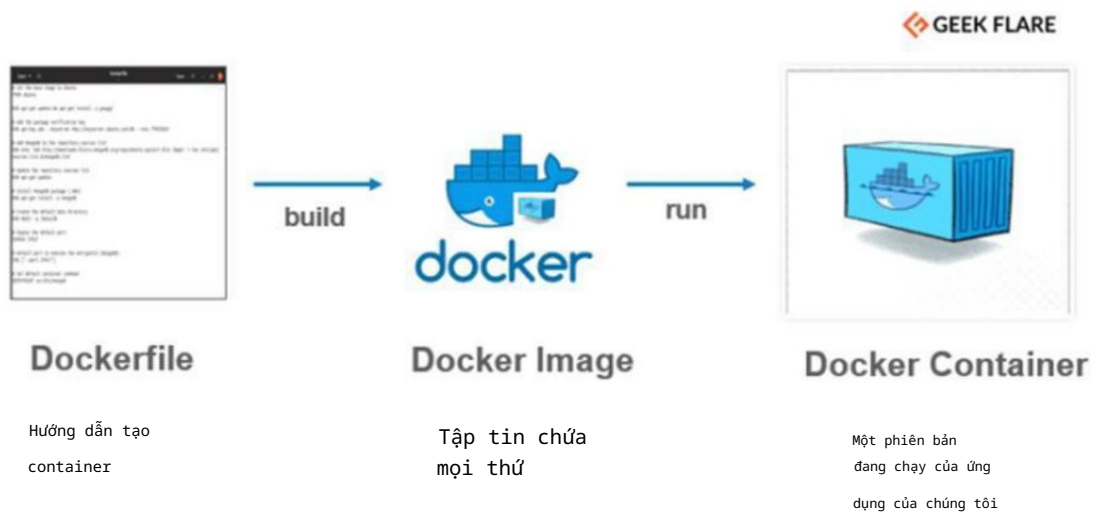


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

18

Mức độ tái tạo 3

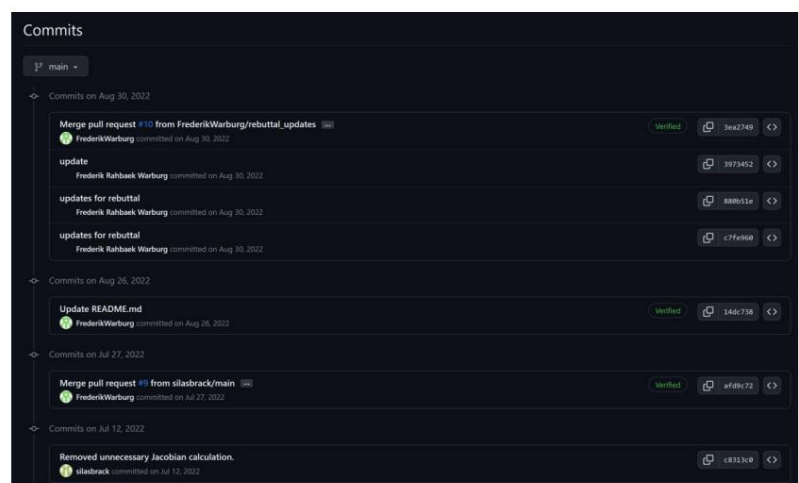
Một cách để tạo các ứng dụng được chứa trong container = máy ảo (VM) có chi phí thấp



Quy trình 6 bước để tạo ra phần mềm có thể tái tạo



Sử dụng kiểm soát phiên bản



Quy trình 6 bước để tạo ra phần mềm có thể tái tạo



Sử dụng kiểm soát phiên bản



Sử dụng mẫu

```

├── LICENSE
├── Makefile
├── README.md
├── data
│   ├── external
│   ├── interim
│   ├── processed
│   └── raw
├── docs
├── models
├── notebooks
├── references
├── reports
│   └── figures
├── requirements.txt
├── setup.py
├── src
│   ├── __init__.py
│   ├── data
│   │   └── make_dataset.py
│   ├── features
│   │   └── build_features.py
│   ├── models
│   │   ├── predict_model.py
│   │   └── train_model.py
│   └── visualization
│       └── visualize.py
└── tox.ini

```

<- Makefile with commands like 'make data' or 'make train'
 <- The top-level README for developers using this project.
 <- Data from third party sources.
 <- Intermediate data that has been transformed.
 <- The final, canonical data sets for modeling.
 <- The original, immutable data dump.
 <- A default Sphinx project; see sphinx-doc.org for details
 <- Trained and serialized models, model predictions, or model summaries
 <- Jupyter notebooks. Naming convention is a number (for ordering),
 the creator's initials, and a short '-' delimited description, e.g.
 '1.0-jpp-initial-data-exploration'.
 <- Data dictionaries, manuals, and all other explanatory materials.
 <- Generated analysis as HTML, PDF, LaTeX, etc.
 <- Generated graphics and figures to be used in reporting
 <- The requirements file for reproducing the analysis environment, e.g.
 generated with 'pip freeze > requirements.txt'
 <- makes project pip installable (pip install -e .) so src can be imported
 <- Source code for use in this project.
 <- Makes src a Python module
 <- Scripts to download or generate data
 <- Scripts to turn raw data into features for modeling
 <- Scripts to train models and then use trained models to make
 predictions
 <- Scripts to create exploratory and results oriented visualizations
 <- tox file with settings for running tox; see tox.readthedocs.io



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

21

Quy trình 6 bước để tạo ra phần mềm có thể tái tạo



Sử dụng kiểm soát phiên bản



Sử dụng mẫu



Viết ra các phụ thuộc của bạn (và
sử dụng môi trường ảo)

```

requirements.txt
1 Click==7.0
2 Flask==1.1.1
3 gunicorn==19.9.0
4 itsdangerous==1.1.0
5 Jinja2==2.10.1
6 MarkupSafe==1.1.1
7 Werkzeug==0.15.6
8
9

```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

22

Quy trình 6 bước để tạo ra phần mềm có thể tái tạo



Sử dụng kiểm soát phiên bản



Sử dụng mẫu



Viết ra các phụ thuộc của bạn
(và sử dụng môi trường ảo)



Ghi lại mã của bạn!



Kiểm tra mã của bạn



Đóng gói mã của bạn

```
1 # Import a base image so we don't have to start from scratch
2 #FROM python:3.10-slim
3 FROM huggingface/transformers-pytorch-cpu
4
5 # Use EXPOSE so we can give docker run the appropriate commandline argument (PORT) as:
6 # docker run predict:latest -e PORT=8000
7 EXPOSE $PORT
8 ENV LC_ALL=C.UTF-8
9 ENV LANG=C.UTF-8
10
11 # Run a bunch of linux commands
12 RUN apt update && \
13     apt install --no-install-recommends -y build-essential gcc & \
14     apt clean & rm -rf /var/lib/apt/lists/*
15
16 # Copy the essential files from our folder to docker container.
17 COPY src/ src/
18 COPY requirements_predict.txt requirements_predict.txt
19 COPY setup.py setup.py
20
21 RUN pip install -r requirements_predict.txt --no-cache-dir
22
23 RUN dvc init --no-scm
24 RUN dvc remote add -d gcloud_storage gs://mlops-dataset-small
25 RUN dvc pull
26
27 # Set working directory as / and install dependencies
28 WORKDIR /
29
30 RUN mkdir app
31
32 # Set entry point, i.e. which file we run with which argument when running the docker container.
33 # The -u flag makes it print to console rather than the docker log file.
34 ENTRYPOINT ["python", "-u", "src/models/predict_model.py"]
35 CMD exec uvicorn src.models.predict_model:app --host 0.0.0.0 --workers 1 --port $PORT
36 #ENTRYPOINT ["uvicorn", "src.models.predict_model:app", "--host", "0.0.0.0", "--workers", "1", "--port", "$PORT"]
```

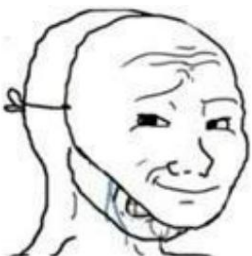


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

25

Meme

Programmers



This code is unreadable and your dataset is flawed! No one will be able to reproduce your results!



It's not my fault the legacy environment is messed up! We still have 97.3% unit test coverage!

Scientists



This code is unreadable and your dataset is flawed. No one will be able to reproduce your results.



I know



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

26

Môi trường đào tạo của chúng tôi

1



Thiết bị đầu cuối SSH

Môi trường đào tạo của chúng tôi

- Nếu bạn đang tham dự một buổi hướng dẫn hoặc hội thảo:
 - một VM đã được cung cấp cho mỗi học viên
- Nếu bạn đang tự học hoặc học lại khóa học này, bạn có thể:
 - cài đặt Docker cục bộ (như đã giải thích trong chương "Cài đặt Docker")
 - cài đặt Docker trên máy ảo đám mây
 - sử dụng <https://www.play-with-docker.com/> để nhận ngay khóa đào tạo môi trường



Máy ảo Docker của chúng tôi

Phần này giả định rằng bạn đang theo học khóa học này như một phần của hướng dẫn, đào tạo hoặc hội thảo, trong đó mỗi học viên được cấp một Docker VM riêng.

- VM được tạo ngay trước khi đào tạo.
- VM sẽ hoạt động trong suốt quá trình đào tạo.
- VM sẽ bị hủy ngay sau khi đào tạo.
- VM được tải sẵn Docker và một số công cụ hữu ích khác.



Docker là gì ?

- “Cài đặt Docker” thực sự có nghĩa là “Cài đặt Docker Engine và CLI”.
- Docker Engine là một daemon (một dịch vụ chạy ở chế độ nền).
- Daemon này quản lý các container, theo cùng cách mà một trình quản lý ảo hóa quản lý VM.
- Chúng ta tương tác với Docker Engine bằng cách sử dụng Docker CLI.
- Docker CLI và Docker Engine giao tiếp thông qua một API.
- Có nhiều chương trình và thư viện máy khách khác sử dụng API đó.



Tại sao chúng ta không chạy Docker cục bộ?

- Chúng tôi sẽ tải xuống hình ảnh container và các gói phân phối.
- Điều này có thể gây áp lực lên WiFi cục bộ và làm chậm chúng tôi.
- Thay vào đó, chúng tôi sử dụng VM từ xa có kết nối tốt.
- Trong một số trường hợp hiếm hoi, việc cài đặt Docker cục bộ sẽ gặp khó khăn:
 - không có quyền truy cập của quản trị viên/root (máy tính được quản lý bởi IT của công ty nghiêm ngặt)
 - CPU hoặc hệ điều hành 32 bit
 - phiên bản hệ điều hành cũ (ví dụ: CentOS 6, OSX trước Yosemite, Windows 7)
- Tốt hơn là dành thời gian tìm hiểu về container hơn là loay hoay với trình cài đặt!



Kết nối với Máy ảo của bạn

Bạn cần một máy khách SSH.

- Trên OS X, Linux và các hệ thống UNIX khác, chỉ cần sử dụng
ssh: `$ ssh <login>@<ip-address>`
- Trên Windows, nếu bạn không có máy khách SSH, bạn có thể tải xuống:
 - Putty (www.putty.org)
 - Git BASH (<https://git-for-windows.github.io/>)
 - MobaXterm (<https://mobaxterm.mobatek.net/>)



Lịch sử đuôi

Lịch sử vó của người hướng dẫn có sẵn trực tuyến theo thời gian thực.

Lưu ý địa chỉ IP của máy ảo của người hướng dẫn (ABCD).

Mở `http://ABCD:1088` trong trình duyệt của bạn và bạn sẽ thấy lịch sử.

Lịch sử được cập nhật theo thời gian thực (sử dụng kết nối WebSocket).

Biểu tượng sẽ có màu xanh khi WebSocket được kết nối.

Nếu nó chuyển sang màu đỏ, tải lại trang sẽ khắc phục được lỗi.



Kiểm tra máy ảo của bạn

Sau khi đăng nhập, hãy đảm bảo rằng bạn có thể chạy lệnh Docker cơ bản:

```
$ phiên bản docker
```

Khách hàng:

Phiên bản: 18.03.0-ce

Phiên bản API: 1.37

Phiên bản Go: go1.9.4

Cam kết Git: 0520e24

Được xây dựng: Thứ tư 21 tháng 3 năm 2018 23:10:06

Hệ điều hành/Arch: linux/amd64

Thực nghiệm: sai

Người điều phối: bầy đàn

Máy chủ:

Động cơ:

Phiên bản: 18.03.0-ce

Phiên bản API: 1.37 (phiên bản tối thiểu 1.12)

Phiên bản Go: go1.9.4

Cam kết Git: 0520e24

Được xây dựng: Thứ tư 21 tháng 3 năm 2018 23:08:35

Hệ điều hành/Arch: linux/amd64

Thực nghiệm: sai



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Cài đặt

Người lái tàu



cài đặt



Mục tiêu

Vào cuối bài học này, bạn sẽ biết: • Cách cài đặt Docker.

- Khi nào sử dụng sudo khi chạy lệnh Docker.

Lưu ý: nếu bạn được cung cấp máy ảo đào tạo để hướng dẫn thực hành, bạn có thể bỏ qua chương này vì máy ảo đó đã cài Docker và Docker đã được thiết lập để chạy mà không cần sudo.



Cài đặt Docker

Có nhiều cách để cài đặt Docker.

Chúng ta có thể phân biệt tùy

ý: • Cài đặt Docker trên máy Linux hiện có (vật lý hoặc VM) • Cài đặt Docker trên macOS hoặc Windows • Cài đặt Docker trên một nhóm VM đám mây



Cài đặt Docker trên Linux

- Phương pháp được khuyến nghị là cài đặt các gói được cung cấp bởi Công ty Docker:
 - thêm kho lưu trữ gói của Docker Inc. vào cấu hình hệ thống của bạn
 - cài đặt Docker Engine
- Hướng dẫn cài đặt chi tiết (từng bản phân phối) có sẵn trên: <https://docs.docker.com/engine/installation/>
- Bạn cũng có thể cài đặt từ tệp nhị phân (nếu bản phân phối của bạn không được hỗ trợ): <https://docs.docker.com/engine/installation/linux/docker-ce/binaries/> • Để thiết lập nhanh môi trường phát triển, Docker cung cấp một tập lệnh cài đặt tiện lợi: `curl -fsSL get.docker.com | sh`



Các gói Docker Inc. so với các gói phân phối

- Docker Inc. phát hành phiên bản mới hàng tháng (edge) và hàng quý (ổn định)
- Các bản phát hành có sẵn ngay lập tức trên kho lưu trữ gói của Docker Inc.
- Các bản phân phối Linux không phải lúc nào cũng cập nhật lên phiên bản Docker mới nhất (Đôi khi, việc cập nhật sẽ phá vỡ nguyên tắc của họ đối với các nâng cấp lớn/nhỏ)
- Đôi khi, một số bản phân phối đã mang theo các gói có bản vá tùy chỉnh
- Đôi khi, các bản vá này đã thêm các lỗi bảo mật nghiêm trọng 😞
- Việc cài đặt thông qua kho lưu trữ của Docker Inc. là một công việc bổ sung. nhưng nói chung là đáng giá!



Cài đặt Docker trên macOS và Windows

- Trên macOS, phương pháp được khuyến nghị là sử dụng Docker Desktop cho Máy Mac:
<https://docs.docker.com/docker-for-mac/install/>
- Trên Windows 10 Pro, Enterprise và Education, bạn có thể sử dụng Docker Desktop cho Windows:
<https://docs.docker.com/docker-for-windows/install/>
- Trên các phiên bản Windows cũ hơn, bạn có thể sử dụng Docker Toolbox:
https://docs.docker.com/toolbox/toolbox_install_windows/
- Trên Windows Server 2016, bạn cũng có thể cài đặt công cụ gốc:
<https://docs.docker.com/install/windows/docker-ee/>



Máy tính để bàn Docker

- Phiên bản Docker đặc biệt có sẵn cho Mac và Windows • Tích hợp tốt với hệ điều hành máy chủ: • được cài đặt như các ứng dụng người dùng thông thường trên máy chủ • cung cấp GUI thân thiện với người dùng để chỉnh sửa cấu hình và cài đặt Docker
- Chỉ hỗ trợ chạy một Docker VM tại một thời điểm .
. nhưng chúng ta có thể sử dụng docker-machine, Docker Toolbox, VirtualBox, v.v. để tạo một cụm.



Nội dung bên trong Docker Desktop

- Tận dụng hệ thống ảo hóa hệ điều hành máy chủ (ví dụ: [API Hypervisor](#) trên macOS) • Chạy một VM nhỏ (trong suốt với việc sử dụng hàng ngày của chúng tôi) • Truy cập tài nguyên mạng như các ứng dụng thông thường
(và do đó, hoạt động tốt hơn với VPN và tường lửa doanh nghiệp)
- Hỗ trợ chia sẻ hệ thống tập tin thông qua các ổ đĩa
(chúng ta sẽ nói về điều này sau)



Chạy Docker trên macOS và Windows

Khi bạn thực thi phiên bản docker từ thiết bị đầu cuối: • CLI kết nối với Docker Engine qua một socket chuẩn, • Docker Engine thực tế đang chạy trong VM, . nhưng CLI không biết hoặc không quan tâm đến điều đó, • CLI gửi yêu cầu bằng REST API, • Docker Engine trong VM xử lý yêu cầu, • CLI nhận phản hồi và hiển thị cho bạn.

Mọi giao tiếp với Docker Engine đều diễn ra thông qua API.

Điều này cũng cho phép sử dụng Engine từ xa giống hệt như Engine cục bộ.



PSA quan trọng về bảo mật

• Nếu bạn có quyền truy cập vào ổ cắm điều khiển Docker, bạn có thể tiếp quản máy móc

(Bởi vì bạn có thể chạy các container sẽ truy cập vào tài nguyên của máy)

• Do đó, trên máy Linux, người dùng docker tương đương với root • Bạn nên hạn chế quyền truy cập vào nó giống như bạn bảo vệ root • Theo mặc định, ổ cắm điều khiển Docker thuộc về nhóm docker • Bạn có thể thêm người dùng đáng tin cậy vào nhóm docker • Nếu không, bạn sẽ phải thêm tiền tố sudo vào mọi lệnh docker,

ví

dụ: `sudo docker phiên bản`



Những container
đầu tiên của chúng tôi

21



Bồn nhựa nhiều màu sắc

Mục tiêu

Vào cuối bài học này, bạn sẽ:

- Thấy Docker hoạt động.

- Bắt đầu container đầu tiên của bạn.



Xin chào thế giới

Trong môi trường Docker của bạn, chỉ cần chạy lệnh sau:

```
$ docker run busybox echo xin chào thế giới xin chào thế giới
```

(Nếu Docker của bạn mới cài đặt, bạn cũng sẽ thấy một vài dòng bổ sung, tương ứng với quá trình tải xuống hình ảnh busybox.)



Đó là thùng đựng đầu tiên của chúng tôi!

- Chúng tôi đã sử dụng một trong những hình ảnh nhỏ nhất và đơn giản nhất hiện có: busybox.
- busybox thường được sử dụng trong các hệ thống nhúng (điện thoại, bộ định tuyến.)
- Chúng tôi đã chạy một quy trình duy nhất và hiển thị hello world.



Một container hữu ích hơn

Hãy chạy một container thú vị hơn:

```
$ docker run -it ubuntu  
root@04c0bb0a6c07: /#
```

- Đây là một container hoàn toàn mới.
- Chạy hệ thống Ubuntu cơ bản, không có tiện ích gì.
- -it là viết tắt của -i -t.
- -i yêu cầu Docker kết nối chúng ta với stdin của container.
- -t cho Docker biết rằng chúng ta muốn một thiết bị đầu cuối giả.



Làm gì đó trong container của chúng tôi

Hãy thử chạy figlet trong container

```
của chúng tôi. root@04c0bb0a6c07:/# figlet  
hello bash: figlet: command not found
```

Được rồi, chúng ta cần phải cài đặt nó.



Cài đặt một gói trong container của chúng tôi

Chúng tôi muốn figlet, vì vậy hãy

```
cài đặt nó: root@04c0bb0a6c07:/# apt-get update  
...
```

Đã tải 1514 kB trong 14 giây (103 kB/giây)

Đang đọc danh sách gói... Đã xong

```
root@04c0bb0a6c07:/# apt-get install figlet Đang đọc danh sách  
gói... Đã xong
```

...

Một phút sau, figlet đã được cài đặt!



Hãy thử chạy chương trình mới cài đặt của chúng tôi

Chương trình figlet lấy một thông điệp làm tham số.

```
root@04c0bb0a6c07:/# figlet xin chào
```

```

  _
 | L _ | | | ' _ \ / \ _
 | / | _ / | | | | ( ) | _ \

```

```
L | L \ _ L L \ _ / Đẹp quá!
```



Đếm các gói hàng trong container

Hãy kiểm tra xem có bao nhiêu gói được cài đặt ở đó.

```
root@04c0bb0a6c07:/# dpkg -l | wc -l
97
```

- dpkg -l liệt kê các gói được cài đặt trong container của chúng tôi
- wc -l đếm chúng

Chúng tôi có bao nhiêu gói trên máy chủ?



Đếm các gói trên máy chủ

Thoát khỏi container bằng cách đăng xuất khỏi shell, như bạn vẫn thường làm.

(Ví dụ với ^D hoặc thoát)

```
root@04c0bb0a6c07:/# thoát
```

Bây giờ, hãy

thử: • chạy `dpkg -l | wc -l`. Có bao nhiêu gói được cài đặt? • chạy `figlet`. Điều đó có hiệu quả không?



So sánh container và host

Thoát khỏi container bằng cách đăng xuất khỏi shell bằng phím ^D hoặc `exit`.

Bây giờ hãy thử chạy `figlet`. Nó có hoạt động không?

(Không nên; trừ khi tình cờ bạn đang chạy trên máy đã cài đặt `figlet` trước đó.)



Máy chủ và container là những thứ độc lập

- Chúng tôi chạy một container Ubuntu trên máy chủ Linux/Windows/macOS.
- Chúng có các gói khác nhau, độc lập.
- Cài đặt thứ gì đó trên máy chủ không làm lộ nó ra vùng chứa.
- Và ngược lại.
- Ngay cả khi cả máy chủ và vùng chứa đều có cùng bản phân phối Linux!
- Chúng ta có thể chạy bất kỳ container nào trên bất kỳ máy chủ nào.
(Một ngoại lệ: Các container Windows chỉ có thể chạy trên máy chủ Windows; ít nhất là ở thời điểm hiện tại.)



Container của chúng ta đâu?

- Container của chúng ta hiện đang ở trạng thái dừng.
- Container vẫn tồn tại trên đĩa, nhưng tất cả tài nguyên tính toán đã được giải phóng.
- Chúng ta sẽ xem sau cách quay lại container đó.



Bắt đầu một container khác

Nếu chúng ta khởi động một container mới và thử chạy figlet một lần nữa thì sao?

```
$ docker run -it ubuntu
```

```
root@b13c164401fb:/# figlet bash: figlet:
```

```
lệnh không tìm thấy
```

- Chúng tôi bắt đầu một container hoàn toàn mới.
- Hình ảnh Ubuntu cơ bản đã được sử dụng và figlet không có ở đây.



Container của tôi đâu?

- Chúng ta có thể tái sử dụng thùng chứa mà chúng ta đã dành thời gian để tùy chỉnh không?

Chúng ta có thể, nhưng đó không phải là quy trình làm việc mặc định của Docker.

- Vậy quy trình làm việc mặc định là gì?

Luôn bắt đầu bằng một container mới. Nếu chúng ta cần cài đặt thứ gì đó trong container, hãy tạo một hình ảnh tùy chỉnh.

- Có vẻ phức tạp quá!

Chúng ta sẽ thấy rằng thực ra nó khá dễ!

- Và mục đích là gì?

Điều này nhấn mạnh vào tính tự động hóa và khả năng lặp lại. Hãy cùng xem lý do tại sao .



Thú cưng vs. Gia súc

- Trong phép ẩn dụ “thú cưng đấu với gia súc”, có hai loại máy chủ. • Thú cưng: • có tên riêng biệt và cấu hình độc đáo • khi chúng bị mất điện, chúng tôi sẽ làm mọi cách có thể để khắc phục chúng
- Gia súc:
 - có tên chung (ví dụ với số) và cấu hình chung • cấu hình được thực thi bởi quản lý cấu hình, hình ảnh vàng . • khi chúng bị ngừng hoạt động, chúng tôi có thể thay thế chúng ngay lập tức bằng một cái mới
- Mối liên hệ giữa Docker và container là gì?



Môi trường phát triển cục bộ

- Khi chúng tôi sử dụng VM cục bộ (ví dụ như VirtualBox hoặc VMware), quy trình làm việc trông như thế này: • tạo VM từ mẫu cơ sở (Ubuntu, CentOS.) • cài đặt các gói, thiết lập môi trường • làm việc trên dự án • khi hoàn tất, tắt VM • lần sau chúng ta cần làm việc trên dự án, khởi động lại VM khi chúng ta rời khỏi • nếu chúng ta cần điều chỉnh môi trường, chúng ta sẽ thực hiện trực tiếp
- Theo thời gian, cấu hình VM phát triển, phân kỳ. • Chúng tôi không có cách rõ ràng, đáng tin cậy và xác định để cung cấp môi trường.



- Với Docker, quy trình làm việc trông như thế này:
 - tạo hình ảnh container với môi trường dev của chúng tôi
 - chạy container với hình ảnh đó
 - làm việc trên dự án
 - khi hoàn tất, hãy tắt container
 - lần tới khi chúng ta cần làm việc trên dự án, hãy bắt đầu một container mới
 - nếu chúng ta cần điều chỉnh môi trường, chúng ta tạo ra một hình ảnh mới
- Chúng tôi có định nghĩa rõ ràng về môi trường của mình và có thể chia sẻ nó một cách đáng tin cậy với người khác.
- Chúng ta hãy cùng xem cách tạo hình ảnh tùy chỉnh bằng figlet trong các chương tiếp theo nhé!

???



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

EN:- Chạy container đầu tiên của chúng tôi :FR:- Lancer nos primes conteneurs

39

Container
nền



Container nền



Mục tiêu

Những container đầu tiên của chúng tôi có tính tương tác.

Bây giờ chúng ta sẽ xem cách:

- Chạy một container không tương tác.
- Chạy một container ở chế độ nền.
- Liệt kê các container đang chạy.

Kiểm tra nhật ký của một container.

- Dừng một container.
- Liệt kê các container đã dừng.



Một container không tương tác

Chúng tôi sẽ chạy một container tùy chỉnh nhỏ.

Container này chỉ hiển thị thời gian mỗi giây. `$ docker run jpetazzo/clock`

```
Thứ sáu, 20 tháng 2, 00:28:53 UTC 2015 Thứ sáu, 20
tháng 2, 00:28:54 UTC 2015 Thứ sáu, 20 tháng 2, 00:28:55
UTC 2015
```

...

- Container này sẽ chạy mãi mãi.

- Để dừng, hãy nhấn

^C. • Docker đã tự động tải xuống hình ảnh jpetazzo/clock. • Hình ảnh này là hình ảnh người dùng, được tạo bởi jpetazzo. • Chúng



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ta sẽ nghe thêm về hình ảnh người dùng (và các loại hình ảnh khác) sau.43

Khi ^C không hoạt động.

Đôi khi, ^C sẽ không đủ.

Tại sao? Và làm sao chúng ta có thể dừng container trong trường hợp đó?



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Điều gì xảy ra khi chúng ta nhấn ^C

SIGINT được gửi đến vùng chứa, điều này có nghĩa là:

- SIGINT được gửi đến PID 1 (trường hợp mặc định)
- SIGINT được gửi đến các tiến trình nền trước khi chạy với -ti Nhưng có một trường hợp đặc biệt đối với PID 1: nó bỏ qua tất cả các tín hiệu!
- ngoại trừ SIGKILL và SIGSTOP
- ngoại trừ các tín hiệu được xử lý rõ ràng TL, DR: có nhiều trường hợp mà ^C sẽ không dừng vùng chứa.



Tại sao PID 1 lại đặc biệt?

- PID 1 có một số trách nhiệm bổ sung:
 - nó khởi động (trực tiếp hoặc gián tiếp) mọi tiến trình khác
 - khi một tiến trình thoát, các tiến trình của nó được “chuyển đổi cha mẹ” theo PID 1
- Khi PID 1 thoát, mọi thứ dừng lại:
 - trên một máy “thông thường”, nó gây ra lỗi kernel panic
 - trong một container, nó sẽ giết chết tất cả các tiến trình
 - Chúng tôi không muốn PID 1 dừng lại một cách vô tình
 - Đó là lý do tại sao nó có các biện pháp bảo vệ bổ sung này



Vậy thì làm sao để ngăn chặn những thùng chứa này?

- Khởi động một terminal khác và quên chúng đi (bây giờ!) •

Chúng ta sẽ sớm tìm hiểu về docker kill



Chạy một container trong nền

Container có thể được khởi động ở chế độ nền, với cờ -d (chế độ daemon): `$ docker run -d jpetazzo/clock`

`47d677dcfba4277c6cc68fcaa51f932b544cab1a187c853b7d0 caf4e8debe5ad`

- Chúng ta không thấy đầu ra của container. • Nhưng đừng lo: Docker sẽ thu thập đầu ra đó và ghi lại! • Docker cung cấp cho chúng ta ID của container.



Liệt kê các container đang chạy

Làm sao chúng ta có thể kiểm tra xem container của chúng ta vẫn đang chạy?

Với docker ps, giống như lệnh ps của UNIX, sẽ liệt kê các tiến trình đang chạy.

```
$ docker ps
HÌNH ẢNH ID CONTAINER          ... TẠO          TRẠNG THÁI
...
47d677dcfba4 jpetazzo/đồng hồ ... phút ...          2 phút trước Lên 2
```

Docker cho chúng ta biết:

- ID (đã cắt bớt) của container của chúng tôi.
- Hình ảnh được sử dụng để khởi động container.
- Container của chúng tôi đã chạy (Up) trong vài phút.
- Thông tin khác (COMMAND, PORTS, NAMES) mà chúng tôi sẽ giải thích sau.



Bắt đầu nhiều container hơn

Chúng ta hãy bắt đầu thêm hai thùng chứa nữa.

```
$ docker run -d jpetazzo/clock
57ad9bdfc06bb4407c47220cf59ce21585dce9a1298d7a67488 359aeaea8ae2a
```

```
$ docker run -d jpetazzo/clock
068cc994ffd0190bbe025ba74e4c0771a5d8f14734af772ddee 8dc1aaf20567d
```

Kiểm tra xem docker ps có báo cáo đúng cả 3 container không.



Chỉ xem container cuối cùng đã bắt đầu

Khi nhiều container đang chạy, sẽ hữu ích khi chỉ xem container cuối cùng được khởi động.

Điều này có thể thực hiện được bằng cờ -l ("Cuối cùng"):

```
$ docker ps -l
HÌNH ẢNH ID CONTAINER          ... TẠO
TRẠNG THÁI                    ...
068cc994ffd0 jpetazzo/đồng hồ ...      2 phút trước
Lên 2 phút ...
```



Chỉ xem ID của các container

Nhiều lệnh Docker sẽ hoạt động trên ID container: docker stop, docker rm.

Nếu chúng ta chỉ muốn liệt kê ID của các container (không có các cột khác hoặc dòng tiêu đề), chúng ta có thể sử dụng cờ -q ("Quiet", "Quick"):

```
$ docker ps -q
068cc994ffd0
57ad9bdfc06b
47d677dcfba4
```



Kết hợp cờ

Chúng ta có thể kết hợp `-l` và `-q` để chỉ xem ID của container được khởi động cuối cùng:

```
$ docker ps -lq
068cc994ffd0
```

Thoạt nhìn, có vẻ như điều này đặc biệt hữu ích trong các tập lệnh.

Tuy nhiên, nếu chúng ta muốn khởi động một container và lấy ID của nó theo cách đáng tin cậy, tốt hơn là sử dụng `docker run -d`, chúng ta sẽ đề cập đến điều này sau.

(Sử dụng `docker ps -lq` dễ xảy ra tình trạng chạy đua: điều gì sẽ xảy ra nếu người khác, hoặc một chương trình hoặc tập lệnh khác, khởi động một container khác ngay trước khi chúng ta chạy `docker ps -lq`?)



Xem nhật ký của một container

Chúng tôi đã nói với bạn rằng Docker sẽ ghi lại dữ liệu đầu ra của container.

Chúng ta hãy cùng xem nhé.

```
$ docker logs 068
Thứ sáu 20 tháng 2 00:39:52 UTC 2015
Thứ sáu 20 tháng 2 00:39:53 UTC 2015
...
```

- Chúng tôi đã chỉ định tiền tố của ID vùng chứa đầy đủ.
- Tất nhiên, bạn có thể chỉ định ID đầy đủ.
- Lệnh `logs` sẽ xuất ra toàn bộ nhật ký của container.
(Đôi khi, điều đó là quá đáng. Chúng ta hãy xem cách giải quyết vấn đề đó.)



Chỉ xem phần đuôi của các bản ghi

Để tránh bị làm phiền bởi mười một trang kết quả, chúng ta có thể sử dụng tùy chọn `--tail`:

```
$ docker logs --tail 3 068
Thứ sáu 20 tháng 2 00:55:35 UTC 2015
Thứ sáu 20 tháng 2 00:55:36 UTC 2015
Thứ sáu 20 tháng 2 00:55:37 UTC 2015
```

- Tham số là số dòng mà chúng ta muốn xem.



Theo dõi nhật ký theo thời gian thực

Giống như lệnh UNIX chuẩn `tail -f`, chúng ta có thể theo dõi nhật ký của vùng chứa:

```
$ docker logs --tail 1 --follow 068
Thứ sáu 20 tháng 2 00:57:12 UTC 2015
Thứ sáu 20 tháng 2 00:57:13 UTC 2015
^C
```

- Điều này sẽ hiển thị dòng cuối cùng trong tệp nhật ký.
- Sau đó, nó sẽ tiếp tục hiển thị nhật ký theo thời gian thực.
- Sử dụng `^C` để thoát.



Dừng container của chúng tôi

Có hai cách để chúng ta có thể chấm dứt container bị tách rời. • Giết nó bằng lệnh `docker kill`. • Dừng nó bằng lệnh `docker stop`.

Đầu tiên, container sẽ dừng ngay lập tức bằng cách sử dụng tín hiệu `KILL`.

Thứ hai thì nhẹ nhàng hơn. Nó gửi tín hiệu `TERM` và sau 10 giây, nếu container chưa dừng lại, nó sẽ gửi `KILL`.

Lưu ý: tín hiệu `KILL` không thể bị chặn và sẽ buộc phải chấm dứt container.



Dừng container của chúng tôi

Hãy dừng một trong những container đó: \$

```
docker stop 47d6 47d6
```

Việc này sẽ mất 10 giây:

- Docker gửi tín hiệu `TERM`; • container

không phản ứng với tín hiệu này (đó là một tập lệnh Shell đơn giản không có xử lý tín hiệu đặc biệt);

- 10 giây sau, vì container vẫn đang chạy, Docker gửi tín hiệu `KILL`; • tín hiệu này sẽ chấm dứt

container.



Giết các container còn lại

Chúng ta hãy bắt đầu nhấn hơn với hai thùng chứa còn lại:

```
$ docker giết 068 57ad 068 57ad
```

Các lệnh dừng và hủy có thể sử dụng nhiều ID vùng chứa.

Những container đó sẽ bị chấm dứt ngay lập tức (không có sự chậm trễ 10 giây).

Hãy kiểm tra xem các container của chúng ta có hiển thị nữa không:

```
$ docker ps
```



Liệt kê các container đã dừng

Chúng ta cũng có thể thấy các container đã dừng bằng tùy chọn -a (--all).

```
$ docker ps -a CONTAINER
ID IMAGE ... CREATED 068cc994ffd0 jpetazzo/clock ... 3 phút trước TRẠNG THÁI
57ad9bdfc06b jpetazzo/clock ... 3 phút trước 21 phút trước Đã thoát (137)
47d677dcfba4
jpetazzo/clock ... 23 phút trước Đã thoát (137) 3 phút 21 phút trước Đã thoát (137)
trước 5c1dfd4d81f1
jpetazzo/clock ... 40 phút trước Đã thoát (0) 40 phút trước b13c164401fb ubuntu 53 phút trước
... 55 phút trước Đã thoát (130)
```



Mục tiêu

Trong bài học này, chúng ta sẽ tìm hiểu về một khái niệm quan trọng của Docker: đặt tên container.

Việc đặt tên cho phép chúng

ta:

- Tham chiếu dễ dàng đến một vùng

- Đảm bảo tính duy nhất của một vùng chứa cụ thể.

Chúng ta cũng sẽ thấy lệnh `inspect` cung cấp nhiều thông tin chi tiết về một container.



Đặt tên cho container của chúng tôi

Cho đến nay, chúng ta đã tham chiếu các container bằng ID của chúng.

Chúng tôi đã sao chép và dán ID hoặc sử dụng tiền tố rút gọn.

Nhưng mỗi container cũng có thể được tham chiếu theo tên của nó.

Nếu một container được đặt tên là `thumbnail-worker`, tôi có thể thực hiện:

```
$ docker logs thumbnail-worker $ docker stop  
thumbnail-worker v.v.
```



Tên mặc định

Khi chúng ta tạo một container, nếu chúng ta không cung cấp tên cụ thể, Docker sẽ tự chọn tên cho chúng ta.

Nó sẽ là sự kết hợp của:

- Một tâm trạng (tức giận, ngớ ngẩn, nghi ngờ, buồn chán.)
- Tên của một nhà phát minh nổi tiếng (tesla, darwin, wozniak.)

Ví dụ: happy_curie, clever_hopper, jovial_lovelace .



Chỉ định tên

Bạn có thể đặt tên cho container khi tạo nó.

```
$ docker run --name ticktock jpetazzo/đồng hồ
```

Nếu bạn chỉ định một tên đã tồn tại, Docker sẽ từ chối tạo vùng chứa.

Điều này cho phép chúng ta thực thi tính duy nhất của một nguồn tài nguyên nhất định.



Đổi tên container

- Bạn có thể đổi tên container bằng lệnh `docker rename`.
- Điều này cho phép bạn “giải phóng” một tên mà không phá hủy tên liên quan thủng chứa.



Kiểm tra một container

Lệnh `docker inspect` sẽ xuất ra bản đồ JSON rất chi tiết.

```
$ docker kiểm tra <containerID> [{
```

```
...
```

```
(nhiều trang JSON ở đây)
```

```
...
```

Có nhiều cách để tiếp nhận thông tin đó.



Phân tích cú pháp JSON bằng Shell

- Bạn có thể grep và cắt hoặc awk đầu ra của docker inspect.
 - Xin đừng làm vậy.
 - Thật đau đớn.
 - Nếu bạn thực sự phải phân tích cú pháp JSON từ Shell, hãy sử dụng JQ! (Thật tuyệt.)
- ```
$ docker kiểm tra <containerID> | jq .
```
- Chúng ta sẽ thấy một giải pháp tốt hơn mà không cần thêm công cụ nào.



## Sử dụng --format

Bạn có thể chỉ định một chuỗi định dạng, chuỗi này sẽ được phân tích cú pháp bởi gói text/template của Go. \$ docker inspect

```
--format '{{ json .Created }}' <containerID> "2015-02-24T07:21:11.712240394Z"
```

- Cú pháp chung là bao quanh biểu thức bằng dấu ngoặc nhọn kép.
- Biểu thức bắt đầu bằng dấu chấm biểu diễn đối tượng JSON.
- Sau đó, mỗi trường hoặc thành viên có thể được truy cập theo cú pháp ký hiệu chấm.
- Từ khóa json tùy chọn yêu cầu đầu ra JSON hợp lệ. (ví dụ ở đây nó thêm dấu ngoặc kép bao quanh.)



# Vào bên trong một container

71



Người đứng bên trong một container

## Mục tiêu

Trên máy chủ hoặc máy ảo truyền thống, đôi khi chúng ta cần:

- đăng nhập vào máy (bằng SSH hoặc trên bảng điều khiển),
- phân tích đĩa (bằng cách tháo đĩa hoặc khởi động lại bằng hệ thống cứu hộ).

Trong chương này, chúng ta sẽ xem cách thực hiện điều đó bằng container.



## Nhận một vỏ

Thỉnh thoảng, chúng ta muốn đăng nhập vào một máy nào đó.

Trong một thế giới hoàn hảo, điều này không cần thiết.

- Bạn cần cài đặt hoặc cập nhật các gói (và cấu hình của chúng)?  
Sử dụng quản lý cấu hình. (ví dụ: Ansible, Chef, Puppet, Salt.)

- Bạn cần xem nhật ký và số liệu?

Thu thập và truy cập chúng thông qua một nền tảng tập trung.

Tuy nhiên, trong thế giới thực. chúng ta thường cần quyền truy cập shell!



## Không nhận được vỏ

Ngay cả khi không có hệ thống triển khai hoàn hảo, chúng ta vẫn có thể thực hiện nhiều thao tác mà không cần shell. • Cài đặt

các gói có thể (và nên) được thực hiện trong ảnh container. • Cấu hình có thể được thực hiện

ở cấp độ ảnh hoặc khi container khởi động. • Cấu hình động có thể được lưu trữ trong một ổ đĩa (chia sẻ với một thùng chứa).

• Nhật ký được ghi vào stdout sẽ được Docker Engine tự động thu thập. • Các nhật ký khác có thể được ghi vào ổ đĩa được chia sẻ. • Thông tin quy trình và số liệu có thể nhìn thấy từ máy chủ.

Chúng ta hãy để dành việc ghi nhật ký, khối lượng. cho sau này, nhưng hãy cùng xem xét thông tin quy trình!



## Xem các tiến trình chứa từ máy chủ

Nếu bạn chạy Docker trên Linux, các tiến trình chứa sẽ hiển thị trên máy chủ.

```
$ ps faux | less
```

• Cuộn quanh đầu ra của lệnh này. • Bạn sẽ thấy vùng chứa jpetazzo/clock. • Một tiến trình được chứa trong vùng chứa cũng giống như bất kỳ tiến trình nào khác trên máy chủ. • Chúng ta có thể sử dụng các công cụ như lsof, strace để phân tích chúng.





## Sự khác biệt giữa tiến trình chứa và tiến trình lưu trữ là gì?

- Mỗi quy trình (có chứa hoặc không) đều thuộc về không gian tên và nhóm c.
- Không gian tên và nhóm c xác định những gì một quy trình có thể "nhìn thấy" và "LÀM".
- Tương tự: mỗi tiến trình (có chứa hoặc không) chạy với một UID (ID người dùng) cụ thể.
- UID=0 là root và có quyền nâng cao. Các UID khác là bình thường người dùng.

Chúng tôi sẽ cung cấp thêm thông tin chi tiết về không gian tên và cgroup sau.



## Đưa một vỏ vào một container đang chạy

- Đôi khi, dù sao chúng ta cũng cần phải có một cái vỏ.
- Chúng ta có thể chạy một số máy chủ SSH trong vùng chứa .
- Nhưng sử dụng docker exec sẽ dễ hơn.  

```
$ docker exec -ti ticktock sh
```
- Thao tác này tạo ra một tiến trình mới (chạy sh) bên trong vùng chứa.
- Điều này cũng có thể được thực hiện "thủ công" bằng công cụ nsenter.



## Những lưu ý

- Công cụ mà bạn muốn chạy cần phải tồn tại trong vùng chứa.
- Một số công cụ (như `ip netns exec`) cho phép bạn đính kèm vào một không gian tên tại một thời điểm.

(Điều này cho phép bạn thiết lập giao diện mạng, ngay cả khi bạn không có `ifconfig` hoặc `ip` trong vùng chứa.)

- Quan trọng nhất: container cần phải chạy.
- Điều gì xảy ra nếu container dừng lại hoặc bị sập?



## Nhận được một vỏ trong một container dừng lại

- Một container đã dừng chỉ là nơi lưu trữ (như ổ đĩa).
- Chúng ta không thể SSH vào ổ đĩa hoặc ổ USB!
- Chúng ta cần kết nối đĩa với một máy đang chạy.
- Điều đó có ý nghĩa như thế nào trong thế giới container?



## Phân tích một container đã dừng

Như một bài tập, chúng ta sẽ cố gắng tìm ra lỗi của jpetazzo/crashtest.

```
docker chạy jpetazzo/crashtest
```

Container khởi động nhưng sau đó dừng lại ngay lập tức mà không có bất kỳ kết quả nào.

MacGyver sẽ làm gì? TM

Đầu tiên, hãy kiểm tra trạng thái của container đó.

```
docker ps -l
```



## Xem các thay đổi hệ thống tập tin

- Chúng ta có thể sử dụng docker diff để xem các tập tin đã được thêm vào/thay đổi/LOẠI BỎ.

```
docker diff <container_id> • ID container
```

được hiển thị bằng docker ps -l. • Chúng ta cũng có thể

thấy nó bằng docker ps -lq. • Đầu ra của docker

diff cho thấy một số tệp nhật ký thú vị!



## Truy cập các tập tin

- Chúng ta có thể trích xuất các tập tin bằng docker cp.

```
docker cp <container_id>:/var/log/nginx/error.log .
```

- Sau đó chúng ta có thể xem tệp nhật ký đó.

```
cat error.log (Thư mục /
```

```
run/nginx không tồn tại.)
```



## Khám phá một container bị rơi

- Chúng ta có thể khởi động lại một container bằng lệnh docker start .

- . Nhưng có lẽ nó sẽ lại sụp đổ ngay lập tức!

- Chúng ta không thể chỉ định một chương trình khác để chạy với docker start • Nhưng chúng ta có thể tạo một hình ảnh mới từ container

```
bị sập docker commit <container_id> debugimage
```

- Sau đó, chúng ta có thể chạy một container mới từ hình ảnh đó, với điểm vào tùy chỉnh

```
docker run -ti --entrypoint sh debugimage
```

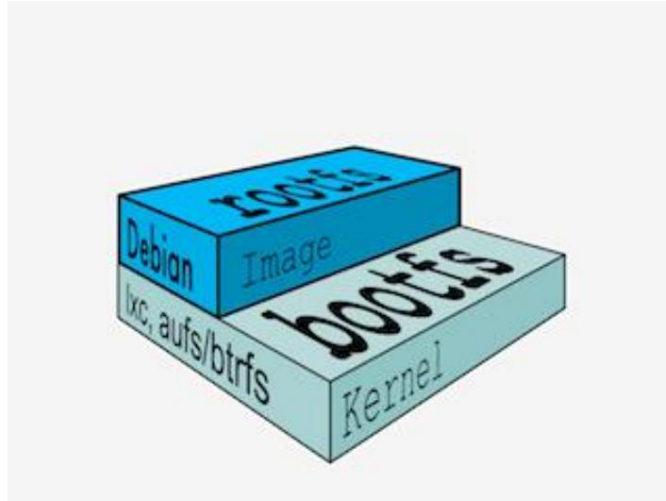


## Nhận được một dump hoàn chỉnh

- Chúng ta cũng có thể dump toàn bộ hệ thống tập tin của một container.
  - Điều này được thực hiện với `docker export`.
  - Nó tạo ra một kho lưu trữ tar. `docker export <container_id> | tar`
- tv Điều này sẽ cung cấp danh sách chi tiết về nội dung của container.



Hiểu biết  
Hình ảnh Docker



hình ảnh



## Mục tiêu

Trong phần này, chúng tôi sẽ giải

thích: • Hình ảnh là

gì. • Lớp là gì. •

Các không gian tên hình ảnh khác

nhau. • Cách tìm kiếm và tải xuống hình

ảnh. • Thẻ hình ảnh và thời điểm sử dụng chúng.



## Hình ảnh là gì?

- Hình ảnh = tệp + siêu dữ liệu • Các

tệp này tạo thành hệ thống tệp gốc của vùng chứa của chúng tôi. • Siêu dữ liệu

có thể chỉ ra một số thứ, ví dụ: • tác giả của hình ảnh • lệnh thực thi trong  
vùng chứa khi khởi động nó • các  
biến môi trường cần thiết lập

- vân vân.

- Hình ảnh được tạo thành từ các lớp, về mặt khái niệm được xếp chồng lên nhau. • Mỗi lớp có thể thêm, thay đổi và xóa tệp và/hoặc siêu dữ liệu. • Hình ảnh có thể chia sẻ các lớp để tối ưu hóa việc sử dụng đĩa, thời gian truyền và sử dụng bộ nhớ.



## Ví dụ cho một ứng dụng web Java

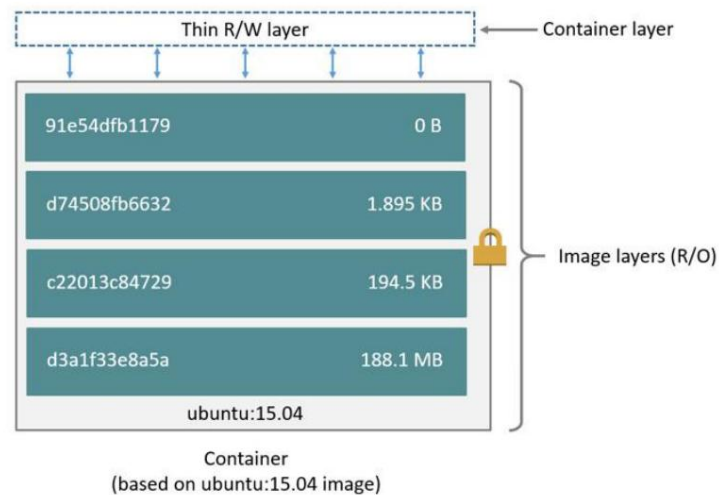
Mỗi mục sau đây sẽ tương ứng với một lớp: • Lớp cơ sở CentOS • Các gói  
và tệp cấu hình do CNTT  
tại địa phương của chúng tôi thêm vào • JRE

- Mèo Tom

- Các phụ thuộc của ứng dụng • Mã và tài sản ứng dụng • Cấu hình ứng dụng  
(Lưu ý: cấu hình ứng dụng thường được thêm vào bởi các tiện ích phối hợp.)



## Lớp đọc-ghi



lớp



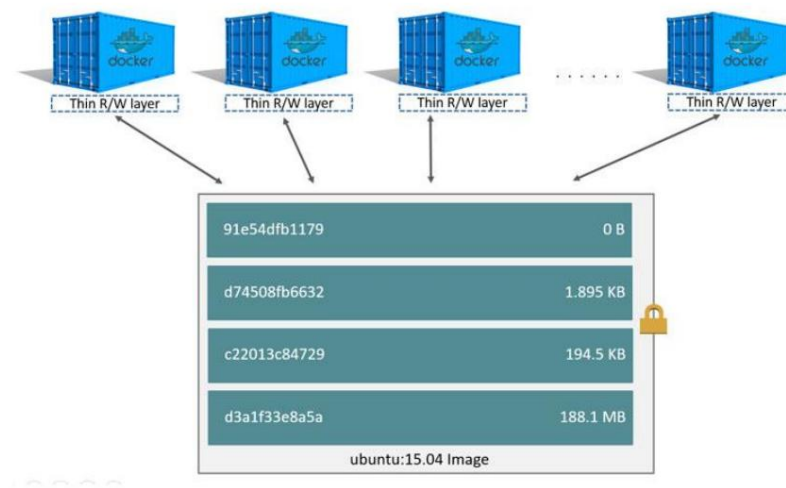
## Sự khác biệt giữa container và hình ảnh

- Một hình ảnh là một hệ thống tập tin chỉ đọc.
- Một container là một tập hợp các quy trình được đóng gói, chạy trong một bản sao đọc-ghi của hệ thống tập tin đó.
- Để tối ưu hóa thời gian khởi động container, copy-on-write được sử dụng thay vì copy thông thường.
- docker run khởi động một container từ một hình ảnh nhất định.





## Nhiều container chia sẻ cùng một hình ảnh



## So sánh với lập trình hướng đối tượng

- Về mặt khái niệm, hình ảnh tương tự như lớp.
- Về mặt khái niệm, lớp tương tự như kế thừa.
- Về mặt khái niệm, container tương tự như thẻ hiện.



## Đợi đã.

Nếu một hình ảnh chỉ đọc, chúng ta có thể thay đổi nó như thế nào? • Chúng ta không thể làm vậy.

- Chúng ta tạo một container mới từ hình ảnh đó. • Sau đó, chúng ta thực hiện các thay đổi đối với container đó. • Khi chúng ta hài lòng với những thay đổi đó, chúng ta chuyển đổi chúng thành một lớp mới.
- Một hình ảnh mới được tạo ra bằng cách xếp chồng lớp mới lên trên lớp hình ảnh cũ.



## Một vấn đề con gà và quả trứng

- Cách duy nhất để tạo một hình ảnh là bằng cách “đóng băng” một container.
- Cách duy nhất để tạo một container là bằng cách khởi tạo một hình ảnh. •

Trợ giúp!



## Tạo ra những hình ảnh đầu tiên

Có một hình ảnh trống đặc biệt được gọi là scratch.

- Nó cho phép xây dựng từ đầu.

Lệnh `docker import` sẽ tải một tarball vào Docker. • Tarball được nhập sẽ trở thành một hình ảnh độc lập. • Hình ảnh mới đó có một lớp duy nhất.

Lưu ý: bạn có thể sẽ không bao giờ phải tự mình làm điều này.



## Tạo hình ảnh khác

`docker cam kết`

- Lưu tất cả các thay đổi được thực hiện đối với một container vào một lớp mới. • Tạo một hình ảnh mới (thực tế là một bản sao của container).
- `docker build` (được sử dụng 99% thời gian) •

Thực hiện một chuỗi xây dựng có thể lặp lại. •

Đây là phương pháp được ưu tiên!

Chúng tôi sẽ giải thích cả hai phương pháp ngay sau đây.



## Không gian tên hình ảnh

Có ba không gian tên: • Hình ảnh chính

thức ví dụ như ubuntu,

busybox . • Hình ảnh người

dùng (và tổ chức) ví dụ như jpetazzo/clock

• Hình ảnh tự lưu trữ

ví dụ registry.example.com:5000/my-private/image

Chúng ta hãy cùng giải thích từng cái một.



## Không gian tên gốc

Không gian tên gốc dành cho hình ảnh chính thức.

Chúng được quản lý bởi Docker Inc.

Chúng thường được biên soạn và duy trì bởi bên thứ ba.

Những hình ảnh đó bao

gồm: • Hình ảnh nhỏ, "swiss-army-knife" như busybox. •

Hình ảnh distro được sử dụng làm cơ sở cho bản dựng của bạn, như ubuntu, fedora. •

Các thành phần và dịch vụ sẵn sàng sử dụng, như redis, postgresql. • Hơn

150 tại thời điểm này!



## Không gian tên người dùng

Không gian tên người dùng lưu trữ hình ảnh cho người dùng và tổ chức Docker Hub.

Ví dụ:

jpetazzo/clock

Người dùng Docker Hub là:

jpetazzo

Tên hình ảnh là:

cái đồng hồ



## Không gian tên tự lưu trữ

Không gian tên này chứa những hình ảnh không được lưu trữ trên Docker Hub mà trên các cơ quan đăng ký của bên thứ ba.

Chúng chứa tên máy chủ (hoặc địa chỉ IP) và tùy chọn là cổng của máy chủ đăng ký.

Ví dụ:

localhost:5000/wordpress

• localhost:5000 là máy chủ và cổng của sổ đăng ký • wordpress

là tên của hình ảnh Các ví dụ khác: quay.io/

coreos/etcd gcr.io/

google-containers/hugo



## Bạn lưu trữ và quản lý hình ảnh như thế nào?

Hình ảnh có thể được lưu trữ:

- Trên máy chủ Docker của bạn.
- Trong sổ đăng ký Docker.

Bạn có thể sử dụng ứng dụng khách Docker để tải xuống (kéo) hoặc tải lên (đẩy) hình ảnh.

Nói chính xác hơn: bạn có thể sử dụng Docker Client để yêu cầu Docker Engine đẩy và kéo hình ảnh vào và ra khỏi sổ đăng ký.



## Hiển thị hình ảnh hiện tại

Hãy cùng xem hiện có những hình ảnh nào trên máy chủ của chúng ta.

| \$ hình ảnh docker      |          |                                      |
|-------------------------|----------|--------------------------------------|
| KHO Fedora              | TAG      | ID HÌNH ẢNH ĐÃ TẠO KÍCH THƯỚC        |
|                         | mới nhất | ddd5c9c1d0f2 3 ngày trước 204,7 MB   |
| centos                  | mới nhất | d0e7f81ca65c 3 ngày trước 196,6 MB   |
| ubuntu redis            | mới nhất | 07c86167cdc4 4 ngày trước 188 MB     |
| postgres                | mới nhất | 4f5f397d4b7c 5 ngày trước 177,6 MB   |
| alpine debian           | mới nhất | afe2b5e1859b 5 ngày trước 264,5 MB   |
| busybox                 | mới nhất | 70c557e50ed6 5 ngày trước 4.798 MB   |
| training/               | mới nhất | f50f9524513f 6 ngày trước 125,1 MB   |
| namer mới nhất          | mới nhất | 3240943c9ea3 2 tuần trước 1.114 MB   |
| jpetazzo/clock mới nhất |          | 902673acc741 9 tháng trước 289,3 MB  |
|                         |          | 12068b93616f 12 tháng trước 2.433 MB |



## Tìm kiếm hình ảnh

Chúng tôi không thể liệt kê tất cả hình ảnh trên số đăng ký từ xa, nhưng chúng tôi có thể tìm kiếm theo một từ khóa cụ thể:

\$ docker tìm kiếm marathon

| TÊN                                                                      | SỰ MIÊU TẢ                  | STARS CHÍNH THỨC |
|--------------------------------------------------------------------------|-----------------------------|------------------|
| TỰ ĐỘNG                                                                  |                             |                  |
| tàng trung lưu/marathon [OK]                                             | Một khởi tạo toàn cụm và... | 105              |
| mesoscloud/marathon [Được]                                               | Cuộc chạy marathon          | 31               |
| mesosphere/marathon-lb Script để cập nhật haproxy b... 22 [OK]           |                             |                  |
| tobilg/mongodb-marathon Một hình ảnh Docker để bắt đầu ...<br>[ĐƯỢC RỒI] |                             | 4                |

- “Ngôi sao” biểu thị mức độ phổ biến của hình ảnh.
- Hình ảnh “chính thức” là những hình ảnh nằm trong không gian tên gốc.
- Hình ảnh “tự động” được Docker Hub tự động xây dựng. (Điều này có nghĩa là công thức xây dựng của chúng luôn có sẵn.)



## Tải xuống hình ảnh

Có hai cách để tải hình ảnh xuống.

- Rõ ràng, với lệnh docker pull.
- Ngầm định, khi thực hiện lệnh docker run và không tìm thấy hình ảnh cục bộ.



## Kéo một hình ảnh

```
$ docker pull debian:jessie
```

Đang kéo kho lưu trữ debian

b164861940b8: Tải xuống hoàn tất b164861940b8:

Đang kéo hình ảnh (jessie) từ debian d1881793a057: Tải xuống hoàn tất

- Như đã thấy trước đó, hình ảnh được tạo thành từ các lớp.
- Docker đã tải xuống tất cả các lớp cần thiết.
- Trong ví dụ này, :jessie chỉ ra phiên bản Debian chính xác nào chúng ta muốn.

Đây là thẻ phiên bản.



## Hình ảnh và thẻ

- Hình ảnh có thể có thẻ.
- Thẻ xác định phiên bản hoặc biến thể của hình ảnh.
- `docker pull ubuntu` sẽ tham chiếu đến `ubuntu:latest`.
- Thẻ `:latest` thường được cập nhật thường xuyên.





## Khi nào thì (không) sử dụng thẻ

Không chỉ định thẻ:

- Khi thực hiện thử nghiệm và tạo mẫu nhanh. •

Khi thử nghiệm. • Khi

bạn muốn có phiên bản mới nhất.

Chỉ định các

thẻ: • Khi ghi lại một quy trình vào một tập

lệnh. • Khi đưa vào sản xuất.

- Để đảm bảo rằng cùng một phiên bản sẽ được sử dụng ở mọi nơi.

- Để đảm bảo khả năng lặp lại sau này.

Điều này tương tự như những gì chúng ta làm với `pip install`, `npm install`, v.v.



## Hình ảnh đa vòm

- Một hình ảnh có thể hỗ trợ nhiều kiến trúc • Chính

xác hơn, một thẻ cụ thể trong một kho lưu trữ nhất định có thể có: • một manifest

duy nhất tham chiếu đến một hình ảnh cho một kiến trúc duy nhất •

một danh sách manifest (hoặc manifest fat) tham chiếu đến

nhiều hình ảnh • Trong danh sách manifest, mỗi hình ảnh được xác định bằng

- sự kết hợp của: `os`

(`linux`, `windows`) • `architecture` (`amd64`,

`arm`, `arm64`.) • các trường tùy chọn như `variant` (cho `arm` và `arm64`), `os.version` (cho `windows`)



## Làm việc với hình ảnh đa vòm

- Docker Engine sẽ kéo các hình ảnh “gốc” khi có sẵn (các hình ảnh khớp với hệ điều hành/kiến trúc/biến thể của riêng nó)
- Chúng ta có thể yêu cầu một nền tảng hình ảnh cụ thể với --platform
- Docker Engine có thể chạy các hình ảnh không phải gốc nhờ QEMU+binfmt (tự động trên Docker Desktop; với một chút thiết lập trên Linux)



## Tóm tắt phần

Chúng tôi đã học được cách:

- Hiểu về hình ảnh và lớp.
- Hiểu về không gian tên hình ảnh Docker.
- Tìm kiếm và tải xuống hình ảnh.



Trong phần này, chúng ta sẽ tạo hình ảnh container đầu tiên.

Đây sẽ là một hình ảnh phân phối cơ bản, nhưng chúng tôi sẽ cài đặt trước gói figlet.

Chúng tôi sẽ:

- Tạo một container từ hình ảnh cơ sở.
- Cài đặt phần mềm thủ công vào vùng chứa và biến nó thành một hình ảnh mới.
- Tìm hiểu về các lệnh mới: `docker commit`, `docker tag` và `docker diff`.



## Kế hoạch

1. Tạo một container (với `docker run`) bằng cách sử dụng bản phân phối cơ sở của chúng tôi sự lựa chọn.
2. Chạy một loạt lệnh để cài đặt và thiết lập phần mềm của chúng tôi trong thùng chứa.
3. (Tùy chọn) xem xét các thay đổi trong container bằng `docker diff`.
4. Biến container thành một image mới bằng lệnh `docker commit`.
5. (Tùy chọn) thêm thẻ vào hình ảnh bằng thẻ `docker`.



## Thiết lập container của chúng tôi

Khởi động một container Ubuntu:

```
$ docker run -it ubuntu
root@<yourContainerId>:#!/
```

Chạy lệnh apt-get update để làm mới danh sách các gói có thể cài đặt.

Sau đó chạy lệnh apt-get install figlet để cài đặt chương trình mà chúng ta quan tâm.

```
root@<yourContainerId>:#!/ apt-get update && apt-get install figlet ĐÀU RA CỦA LỆNH APT-GET
```



## Kiểm tra các thay đổi

Gõ exit tại dấu nhắc của vùng chứa để thoát khỏi phiên tương tác.

Bây giờ hãy chạy docker diff để xem sự khác biệt giữa ảnh cơ sở và container của chúng ta. \$ docker diff

```
<yourContainerId> C /root A /root/.bash_history C /
tmp C /usr
C /usr/bin A /usr/bin/figlet
```

...



## Docker theo dõi những thay đổi của hệ thống tập tin

Như đã giải thích trước đó: •

Một hình ảnh chỉ đọc. • Khi chúng

ta thực hiện thay đổi, chúng sẽ diễn ra trong một bản sao của hình ảnh. • Docker có thể hiển

thị sự khác biệt giữa hình ảnh và bản sao của nó. • Về hiệu suất, Docker sử dụng hệ thống sao chép

khi ghi. (tức là việc khởi động một container dựa trên một hình ảnh lớn sẽ không tạo ra một bản sao lớn.)



## Lợi ích bảo mật sao chép khi ghi

- docker diff cung cấp cho chúng ta một cách dễ dàng để kiểm tra các thay đổi

(theo kiểu Tripwire)

- Container cũng có thể được khởi động ở chế độ chỉ đọc

(hệ thống tập tin gốc của chúng sẽ chỉ đọc, nhưng chúng vẫn có thể có khối lượng dữ liệu đọc-ghi)



## Cam kết thay đổi của chúng tôi vào một hình ảnh mới

Lệnh docker commit sẽ tạo một lớp mới với những thay đổi đó và một hình ảnh mới sử dụng lớp mới này.

```
$ docker commit <yourContainerId>
<ID hình ảnh mới>
```

Đầu ra của lệnh docker commit sẽ là ID cho hình ảnh mới tạo của bạn.

Chúng ta có thể sử dụng nó như một đối số cho lệnh docker run.



## Kiểm tra hình ảnh mới của chúng tôi

Hãy chạy hình ảnh này:

```
$ docker run -it <newImageId>
root@fcfb62f0bfde:/# figlet xin chào
```

```

 _
 | | _ _ | | | ' _ | _
 \ / \ | | / | _ / | | | - \
 | | () |
 | | _ _ | | _ / Nó có tác
dụng! 
```



## Đánh dấu hình ảnh

Việc tham chiếu đến hình ảnh bằng ID của nó không tiện lợi. Thay vào đó, hãy gắn thẻ cho nó.

Chúng ta có thể sử dụng lệnh tag:

```
$ docker tag <newImageId> figlet
```

Nhưng chúng ta cũng có thể chỉ định thẻ như một đối số bổ sung để cam kết:

```
$ docker commit <containerId> figlet
```

Và sau đó chạy nó bằng thẻ của nó:

```
$ docker run -it figlet
```



## Tiếp theo là gì?

Quy trình thủ công = tệ.

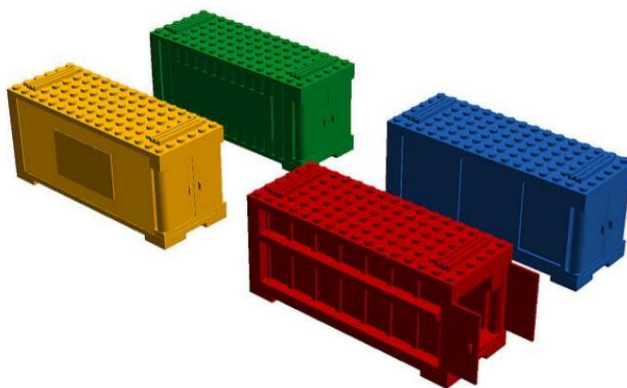
Quy trình tự động = tốt.

Ở chương tiếp theo, chúng ta sẽ tìm hiểu cách tự động hóa quy trình xây dựng bằng cách viết Dockerfile.



# Xây dựng hình ảnh Docker với Tập Docker

123



Công trường xây dựng có container



## Mục tiêu

Chúng tôi sẽ tự động xây dựng một hình ảnh container bằng Dockerfile.

Vào cuối bài học này, bạn sẽ có thể:

- Viết Dockerfile.

- Xây dựng hình ảnh từ Dockerfile.



## Tổng quan về Dockerfile

- Dockerfile là công thức xây dựng cho hình ảnh Docker.
- Nó

chứa một loạt các hướng dẫn cho Docker biết hình ảnh được tạo ra như thế nào được xây dựng.

- Lệnh docker build sẽ xây dựng một hình ảnh từ Dockerfile.



## Viết Dockerfile đầu tiên của chúng tôi

Dockerfile của chúng ta phải nằm trong một thư mục mới và trống.

1. Tạo một thư mục để lưu Dockerfile của chúng ta. \$

```
mkdir myimage
```

2. Tạo

một Dockerfile bên trong thư mục này.

```
$ cd myimage $ vim
Dockerfile
```

Tất nhiên, bạn có thể sử dụng bất kỳ trình soạn thảo nào khác mà bạn chọn.



Nhập nội dung này vào Dockerfile của chúng ta.

TỪ ubuntu

CHẠY apt-get update

CHẠY apt-get cài đặt figlet

- FROM chỉ ra hình ảnh cơ sở cho bản dựng của chúng

- ta. • Mỗi dòng RUN sẽ được Docker thực thi trong quá trình dựng. •

Các lệnh RUN của chúng ta phải không tương tác. (Không thể cung cấp bất kỳ đầu vào nào cho Docker trong quá trình

dựng.) • Trong nhiều trường hợp, chúng ta sẽ thêm cờ -y vào apt-get.



Hãy xây dựng nó!

Lưu tệp của chúng ta, sau đó thực hiện:

```
$ docker build -t figlet . • -t chỉ ra thẻ
```

áp dụng cho hình ảnh.

```
• . chỉ ra vị trí của bối cảnh xây dựng.
```

Chúng ta sẽ nói thêm về bối cảnh xây dựng sau.

Để đơn giản hơn: đây là thư mục chứa Dockerfile của chúng ta.



## Điều gì xảy ra khi chúng ta xây dựng hình ảnh?

Tùy thuộc vào việc chúng ta có sử dụng BuildKit hay không!

Nếu có nhiều đường màu xanh và dòng đầu tiên trông như thế này:

```
[+] Xây dựng 1.8s (4/6)
```

. sau đó chúng ta sử dụng BuildKit.

Nếu đầu ra chủ yếu là đen trắng và dòng đầu tiên trông như thế này: Đang gửi ngữ cảnh xây dựng tới daemon Docker 2.048kB

. sau đó chúng ta sử dụng trình xây dựng "cổ điển" hoặc "kiểu cũ".



## Có nên sử dụng BuildKit hay không

Trình xây dựng cổ điển:

- sao chép toàn bộ “bối cảnh xây dựng” vào Docker Engine
- tuyến tính (xử lý các dòng lần lượt)
- yêu cầu Docker Engine đầy đủ

Bộ công cụ xây dựng:

- chỉ chuyển các phần của “bối cảnh xây dựng” khi cần thiết
- sẽ song song hóa các hoạt động (khi có thể)
- có thể chạy trong các vùng chứa không có đặc quyền (ví dụ trên Kubernetes)



## Với trình xây dựng cổ điển

Đầu ra của lệnh docker build trông như thế này:

```
docker build -t figlet .
```

Gửi ngữ cảnh xây dựng đến Docker daemon 2.048kB Bước 1/3: Tải ubuntu ---> f975c5035748

Bước 2/3: CHẠY apt-get update ---> Chạy trong e01b294dbffd (...đầu ra của lệnh RUN...)

Xóa container trung gian e01b294dbffd ---> eb8d9b561b37 Bước 3/3: CHẠY

apt-get install figlet ---> Chạy trong c29230d70f9b (...đầu ra của lệnh RUN...)

Tháo bỏ container trung gian c29230d70f9b ---> 0dfd7a253f21

Đã xây dựng thành công 0dfd7a253f21  
Đã gắn thẻ figlet:latest thành công

- Đầu ra của lệnh RUN đã bị bỏ qua.
- Chúng ta hãy giải thích ý nghĩa của đầu ra này.



## Gửi cảnh xây dựng đến Docker

Gửi cảnh xây dựng đến daemon Docker 2.048 kB

- Bối cảnh xây dựng là thư mục . được cung cấp cho docker build. • Nó được gửi (dưới dạng lưu trữ) bởi máy khách Docker đến daemon Docker. • Điều này cho phép sử dụng máy từ xa để xây dựng bằng các tệp cục bộ. • Hãy cẩn thận (hoặc kiên nhẫn) nếu thư mục đó lớn và liên kết của bạn chậm. • Bạn có thể tăng tốc quá trình bằng tệp .dockerignore • Nó yêu cầu docker bỏ qua các tệp cụ thể trong thư mục • Chỉ bỏ qua các tệp mà bạn không cần trong bối cảnh xây dựng!



## Thực hiện từng bước

Bước 2/3: CHẠY apt-get update ---> Chạy  
trong e01b294dbffd (...đầu ra của lệnh  
RUN...)

Xóa container trung gian e01b294dbffd  
---> eb8d9b561b37

- Một container (e01b294dbffd) được tạo từ hình ảnh cơ sở. • Lệnh RUN được thực thi trong container này.
- Container được cam kết vào một hình ảnh (eb8d9b561b37). • Container xây dựng (e01b294dbffd) được xóa. • Đầu ra của bước này sẽ là hình ảnh cơ sở cho bước tiếp theo.



## Với BuildKit

```
[+] Tòa nhà 7.9s (7/7) HOÀN THÀNH
=> [nội bộ] tải định nghĩa bản dựng từ Dockerfile => đang chuyển dockerfile: 98B => [nội bộ] 0.0 giây
tải .dockerignore 0.0 giây 0.0 giây

=> => chuyển ngữ cảnh: 2B 0.0 giây
=> [nội bộ] tải siêu dữ liệu cho docker.io/library/ubuntu:latest 1.2s
=> [1/3] Từ docker.io/library/ubuntu@sha256:cf31af331f38d1d7158470e095b132acd126a7180a54f263d386 3.2 giây
=> => giải quyết docker.io/library/ubuntu@sha256:cf31af331f38d1d7158470e095b132acd126a7180a54f263d386 0.0 giây
=> => sha256:cf31af331f38d1d7158470e095b132acd126a7180a54f263d386da88eb681d93 1,20kB / 1,20kB 0,0s
=> => sha256:1de4c5e2d8954bf5fa9855f8b4c9d3c3b97d1d380efe19f60f3e4107a66f5cae 943B / 943B 0,0s
=> => sha256:6a98cbe39225dadedba04e21dbe5900ad604739b07a9fa351dd10a6ebad4c1b 3,31kB / 3,31kB 0,0s
=> => sha256:80bc30679ac1fd798f3241208c14accd6a364cb8a6224d1127dfb1577d10554f 27,14MB / 27,14MB 2,3 giây
=> => sha256:9bf18fab4cfbf479fa9f8409ad47e2702c63241304c2cdd4c33f2a1633c5f85e 850B / 850B 0,5 giây
=> => sha256:5979309c983a2adef352538937475cf961d49c34194fa2aab142effe19ed9c1 189B / 189B 0,4s
=> => trích xuất sha256:80bc30679ac1fd798f3241208c14accd6a364cb8a6224d1127dfb1577d10554f 0,7 giây
=> => đang trích xuất sha256:9bf18fab4cfbf479fa9f8409ad47e2702c63241304c2cdd4c33f2a1633c5f85e 0.0 giây
=> => đang trích xuất sha256:5979309c983a2adef352538937475cf961d49c34194fa2aab142effe19ed9c1 0.0 giây
=> [2/3] CHẠY apt-get update => [3/3] CHẠY apt-get 2,5 giây
install figlet => xuất ra ảnh => xuất các lớp => > ghi ảnh 0,9 giây
0,1 giây
0,1 giây
0.0 giây
0.0 giây
```

sha256:3b8aee7b444ab775975dfba691a72d8ac24af2756e0a024e056e3858d5a23f7c => => đặt tên thành docker.io/library/figlet



## Hiểu đầu ra của BuildKit

- BuildKit chuyển Dockerfile và bối cảnh xây dựng (đây là hai giai đoạn [nội bộ] đầu tiên)
- Sau đó thực hiện các bước được xác định trong Dockerfile ([1/3], [2/3], [3/3])
- Cuối cùng, nó xuất kết quả của bản dựng (định nghĩa hình ảnh + tập hợp các lớp)



## Đầu ra đơn giản của BuildKit

- Khi chạy BuildKit trong ví dụ như đường ống CI, đầu ra của nó sẽ khác nhau
- Chúng ta có thể thấy cùng một định dạng đầu ra bằng cách sử dụng `--progress=plain`



## Hệ thống lưu trữ đệm

Nếu bạn chạy lại cùng một bản dựng, nó sẽ diễn ra ngay lập tức. Tại sao?

- Sau mỗi bước dựng, Docker sẽ chụp ảnh nhanh hình ảnh kết quả.
- Trước khi thực hiện một bước, Docker sẽ kiểm tra xem nó đã dựng cùng một bản dựng hay chưa sự liên tiếp.

- Docker sử dụng các chuỗi chính xác được xác định trong Dockerfile của bạn, vì vậy:
    - `RUN apt-get install figlet cowsay` khác với `RUN apt-get cài đặt cowsay figlet`
    - `CHẠY apt-get update` không được thực hiện lại khi các bản sao được cập nhật
- Bạn có thể buộc xây dựng lại bằng lệnh `docker build --no-cache ....`



## Chạy hình ảnh

Hình ảnh thu được không khác gì hình ảnh được tạo ra thủ công.

```
$ docker run -ti figlet
root@91f3c974c9a1:/# figlet xin chào
```

Tuyệt vời!



## Sử dụng hình ảnh và lịch sử xem

Lệnh `history` liệt kê tất cả các lớp tạo nên một hình ảnh.

Đối với mỗi lớp, nó sẽ hiển thị thời gian tạo, kích thước và lệnh tạo.

Khi một hình ảnh được xây dựng bằng Dockerfile, mỗi lớp tương ứng với một dòng của Dockerfile.

```
$ docker lịch sử figlet
```

HÌNH ẢNH ĐÃ TẠO ĐƯỢC TẠO BỞI f9e8f1642759 Khoảng một giờ trước /bin/sh -c apt-get install

KÍCH CỠ

fi 1.627 MB

7257c37726a1 Khoảng một giờ trước /bin/sh -c apt-get update 21,58 MB

```
07c86167cdc4 4 ngày trước /bin/sh -c #(nop) CMD ["/bin 0 B
```

```
<missing> 4 ngày trước /bin/sh -c echo /bin/sh -c sed -i 's/^#\s*\(\
```

```
'#!/bin/sh' <missing> 4 ngày trước 194,5 kB
```

<mất tích> 4 ngày trước

```
/bin/sh -c #(nop) THÊM tệp:b 187,8 MB
```





## Tại sao lại là sh -c?

- Trên UNIX, để bắt đầu một chương trình mới, chúng ta cần hai lệnh gọi hệ thống:
  - `fork()`, để tạo một tiến trình con mới;
  - `execve()`, để thay thế tiến trình con mới bằng chương trình để chạy.
- Về mặt khái niệm, `execve()` hoạt động như sau:
 

```
execve(chương trình, [danh sách, của, đối số])
```
- Khi chúng ta chạy một lệnh, ví dụ `ls -l /tmp`, cần phải có thứ gì đó để phân tích cú pháp lệnh.
 

(tức là chia chương trình và các đối số của nó thành một danh sách.)
- Vỏ thường làm như vậy.
 

(Nó cũng xử lý việc mở rộng các biến môi trường và những thứ đặc biệt như ~.)



## Tại sao lại là sh -c?

- Khi chúng ta thực hiện lệnh `RUN ls -l /tmp`, trình xây dựng Docker cần phân tích cú pháp lệnh.
- Thay vì triển khai trình phân tích cú pháp của riêng mình, nó chuyển giao công việc cho vỏ bọc.
- Đó là lý do tại sao chúng ta thấy `sh -c ls -l /tmp` trong trường hợp đó.

Nhưng chúng ta cũng có thể tự mình thực hiện các công việc phân

tích cú pháp. • Điều này có nghĩa là truyền cho RUN một danh

sách các đối số. • Đây được gọi là cú pháp `exec`.



## Cú pháp Shell so với cú pháp exec

Các lệnh Dockerfile thực thi một cái gì đó có thể có hai dạng: • chuỗi thuần túy hoặc cú pháp shell: `RUN apt-get install figlet` • danh sách JSON hoặc cú pháp exec: `RUN ["apt-get", "install", "quả sung con"]`

Chúng ta sẽ thay đổi Dockerfile để xem nó ảnh hưởng thế nào đến hình ảnh kết quả.



## Sử dụng cú pháp exec trong Dockerfile của chúng tôi

Hãy thay đổi Dockerfile của chúng ta như sau!

`TỪ ubuntu`

`CHẠY apt-get update`

`CHẠY ["apt-get", "install", "figlet"]`

Sau đó xây dựng Dockerfile mới.

`$ docker build -t figlet .`



## Lịch sử với cú pháp exec

So sánh lịch sử mới:

```
$ docker lịch sử figlet
```

```
HÌNH ẢNH ĐÃ TẠO ĐƯỢC TẠO BỞI 27954bb5faaf 10 giây trước apt-get install figlet 7257c37726a1
```

```
Khoảng một giờ trước /bin/sh -c apt-get update 07c86167cdc4 4 ngày trước /bin/sh -c #(nop) CMD ["/bin 0 B
```

KÍCH CỠ

1.627 MB

21,58MB

```
<missing> 4 ngày trước /bin/sh -c sed -i 's/^#\s*\(\ 1.895 kB
```

```
ngày trước 194,5 kB /bin/sh -c echo '#!/bin/sh' <missing> 4
```

```
<mất tích> 4 ngày trước /bin/sh -c #(nop) THÊM tệp:b 187,8 MB
```

- Cú pháp thực thi chỉ định lệnh chính xác để thực thi.
- Cú pháp shell chỉ định lệnh sẽ được gói trong `/bin/sh -c "..."`.



## Khi nào sử dụng cú pháp exec và cú pháp shell

- cú pháp shell:
  - dễ viết hơn
  - nội suy các biến môi trường và các biểu thức shell khác
  - tạo một tiến trình bổ sung (`/bin/sh -c ...`) để phân tích chuỗi
  - yêu cầu `/bin/sh` phải tồn tại trong container
- cú pháp exec:
  - khó viết (và đọc!) hơn
  - truyền tất cả các đối số mà không cần xử lý thêm
  - không tạo ra một quá trình bổ sung
  - không yêu cầu `/bin/sh` phải tồn tại trong container



Mẹo chuyên nghiệp: shell exec được tích hợp sẵn

Shell POSIX có một lệnh tích hợp có tên là exec.

exec phải theo sau một chương trình và các đối số của nó.

Theo quan điểm của người

dùng: • có vẻ như shell thoát ngay sau khi lệnh được thực thi, • trên thực tế, shell thoát ngay trước khi lệnh được thực thi; • hay đúng hơn, shell được thay thế bằng lệnh.



## Ví dụ sử dụng exec

**CMD** `exec figlet -f script xin chào`

Trong ví dụ này, `sh -c` vẫn sẽ được sử dụng, nhưng `figlet` sẽ là PID 1 trong vùng chứa.

Shell sẽ được thay thế bằng `figlet` khi `figlet` bắt đầu thực thi.

Điều này cho phép chạy các quy trình như PID 1 mà không cần sử dụng JSON.



# CMD và

## ĐIỂM VÀO

149



Cửa ra vào container

## Mục tiêu

Trong bài học này, chúng ta sẽ tìm hiểu về hai lệnh Dockerfile quan trọng: CMD và ENTRYPOINT.

Các lệnh này cho phép chúng ta thiết lập lệnh mặc định để chạy trong một vùng chứa.



## Xác định lệnh mặc định

Khi mọi người chạy container, chúng tôi muốn chào đón họ bằng một thông điệp chào hay và sử dụng phông chữ tùy chỉnh.

Để làm được điều đó, chúng ta sẽ thực hiện:

```
figlet -f script xin chào
```

- Tập lệnh -f yêu cầu figlet sử dụng phông chữ lạ mắt.
- hello là thông báo mà chúng ta muốn nó hiển thị.



## Thêm CMD vào Dockerfile của chúng tôi

Dockerfile mới của chúng ta sẽ trông như thế này:

```
TỪ ubuntu
CHẠY apt-get update
CHẠY ["apt-get", "install", "figlet"]
CMD figlet -f script xin chào
```

- CMD định nghĩa lệnh mặc định để chạy khi không có lệnh nào được đưa ra.
- Nó có thể xuất hiện ở bất kỳ điểm nào trong tập tin.
- Mỗi CMD sẽ thay thế và ghi đè lên CMD trước đó.
- Do đó, mặc dù bạn có thể có nhiều dòng lệnh CMD, nhưng điều đó đều vô dụng.



## Xây dựng và kiểm tra hình ảnh của chúng tôi

Hãy cùng xây dựng nó:

```
$ docker build -t figlet .
```

```
...
```

Đã xây dựng thành công 042dff3b4a8d

Đã gắn thẻ figlet:latest thành công

Và chạy nó:

```
$ docker chạy figlet
```

```
 _
| | | | | / \ | |
| / | / / \ _ _ _ _ | _
```

```
| | / | / | / | \ _ /
```



## Ghi dè CMD

Nếu chúng ta muốn đưa shell vào vùng chứa của mình (thay vì chạy figlet), chúng ta chỉ cần chỉ định một chương trình khác để chạy:

```
$ docker run -it figlet bash
root@7ac86a641116:/#
```

- Chúng tôi đã chỉ định bash.
- Nó thay thế giá trị của CMD.



## Sử dụng ENTRYPOINT

Chúng tôi muốn có thể chỉ định một thông báo khác trên dòng lệnh, trong khi vẫn giữ nguyên figlet và một số tham số mặc định.

Nói cách khác, chúng tôi muốn có thể thực hiện được điều này:

```
$ docker run figlet chào
```

The diagram shows a 3D coordinate system with several axes. A vertical axis is labeled 'z' at the top. A horizontal axis pointing to the right is labeled 'x'. A diagonal axis pointing towards the bottom-left is labeled 'y'. There are also several other axes and lines, some of which are labeled with letters like 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z'. Some of these labels are in different colors (red, green, blue). The diagram is a technical drawing of a 3D space, possibly representing a crystal structure or a mathematical model.

Chúng ta sẽ sử dụng động từ ENTRYPOINT trong Dockerfile.





## Thêm ENTRYPOINT vào Dockerfile của chúng tôi

Dockerfile mới của chúng ta sẽ trông như thế này:

```
TỪ ubuntu
CHẠY apt-get update
CHẠY ["apt-get", "install", "figlet"]
ENTRYPOINT ["figlet", "-f", "script"] • ENTRYPOINT định
```

nghĩa một lệnh cơ sở (và các tham số của nó) cho  
thùng chứa.

- Các đối số dòng lệnh được thêm vào các tham số đó.
- Giống như CMD, ENTRYPOINT có thể xuất hiện ở bất kỳ đâu và thay thế giá trị trước đó.

Tại sao chúng tôi sử dụng cú pháp JSON cho ENTRYPOINT?



## Ý nghĩa của cú pháp JSON so với chuỗi

- Khi CMD hoặc ENTRYPOINT sử dụng cú pháp chuỗi, chúng được gói trong sh-c.

- Để tránh việc bao bọc này, chúng ta có thể sử dụng cú pháp JSON.

Nếu chúng ta sử dụng ENTRYPOINT với cú pháp chuỗi thì sao?

```
$ docker run figlet chào
```

Lệnh sau sẽ chạy trong hình ảnh figlet:

```
sh -c "figlet -f script" chào
```



Hãy cùng xây dựng nó:

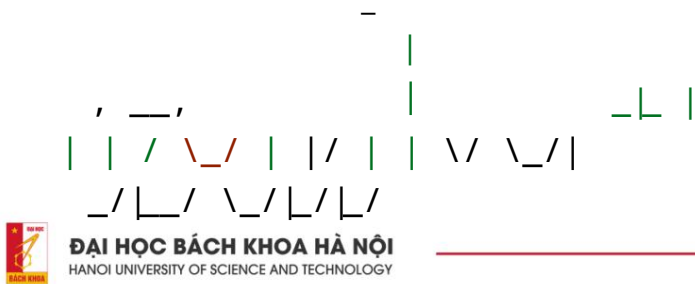
```
$ docker build -t figlet .
...
```

Đã xây dựng thành công 36f588918d73

Đã gắn thẻ figlet:latest thành công

Và chạy nó:

```
$ docker run figlet chào
```



159

## Sử dụng CMD và ENTRYPOINT cùng nhau

Nếu chúng ta muốn xác định một thông báo mặc định cho vùng chứa của mình thì sao?

Sau đó chúng ta sẽ sử dụng ENTRYPOINT và CMD cùng nhau.

- ENTRYPOINT sẽ xác định lệnh cơ sở cho vùng chứa của chúng ta.
- CMD sẽ xác định tham số mặc định cho lệnh này.
- Cả hai đều phải sử dụng cú pháp JSON.

160



## Ghi đề các tham số mặc định của hình ảnh

Bây giờ chúng ta hãy truyền thêm đôi số cho hình ảnh.

```
$ docker chạy myfiglet hola mundo
```

- | - |  
 ..  
 | - | -  
 | \_ | \_ | /  
 \ / \ \_ | / / | | / \ \_  
 \_ / \ \_ / \ \_ / | | / \ \_ /  
 \ \_ / Chúng tôi đã

ghi đè CMD nhưng vẫn sử dụng ENTRYPOINT.

## Ghi dè ENTRYPOINT

Nếu chúng ta muốn chạy shell trong container của mình thì sao?

Chúng ta không thể chỉ chạy `docker run myfiglet bash` vì điều đó sẽ chỉ yêu cầu figlet hiển thị từ `"bash"`.

Chúng tôi sử dụng tham số --entrypoint:

```
$ docker run -it --entrypoint bash myfiglet
root@6027e44e2955: /#
```

## Tóm tắt CMD và ENTRYPOINT

- `docker run myimage` thực thi `ENTRYPOINT` + `CMD`
  - `docker run myimage args` thực thi `ENTRYPOINT` + `args` (ghi đè `CMD`)
  - `docker run --entrypoint prog myimage` thực thi `prog` (ghi đè cả hai)
- .small[ | Lệnh | ĐIỂM VÀO | CMD | Kết quả |—————|—————|—  
 —|— | `docker run figlet` | `none` | `none` | Sử dụng các giá trị từ ảnh cơ sở  
 (bash) | `docker run figlet hola` | `none` | `none` | Lỗi (không tìm thấy tệp thực  
 thi hola) | `docker run figlet` | `figlet -f script` | `none` | `figlet -f script`  
 | `docker run figlet hola` | `figlet -f script` | `none` | `figlet -f script hola` |  
`docker run figlet` | `none` | `figlet -f script` | `figlet -f script`  
 | `docker run figlet hola` | không có | `figlet -f script` | Lỗi ( không tìm  
 thấy tệp thực thi hola) | `docker run figlet` | `figlet -f script` | `hello` | `figlet`  
`-f script hello` | `docker run figlet hola` | `figlet -f script` | `hello`  
 | `figlet -f` tập lệnh xin chào]



## Khi nào sử dụng ENTRYPOINT so với CMD

`ENTRYPOINT` rất phù hợp cho "các tệp nhị phân được chứa trong container".

Ví dụ: `docker run consul --help`

(Giả vờ như phần `docker run` không có ở đó!)

`CMD` rất phù hợp với những hình ảnh có nhiều tệp nhị phân.

Ví dụ: `docker run busybox ifconfig`

(Có nghĩa là phải chỉ ra chương trình nào chúng ta muốn chạy!)

???

:EN:- `CMD` và `ENTRYPOINT` :FR:- `CMD` và `ENTRYPOINT`



Sao chép các tập tin  
trong quá trình xây dựng

167



Các nhà sư chép sách

## Mục tiêu

Cho đến nay, chúng tôi đã cài đặt các thứ vào hình ảnh chứa của mình bằng cách tải xuống các gói.

Chúng ta cũng có thể sao chép các tệp từ ngữ cảnh xây dựng vào vùng chứa mà chúng ta đang xây dựng.

Hãy nhớ: bối cảnh xây dựng là thư mục chứa Dockerfile.

Trong chương này, chúng ta sẽ tìm hiểu về từ khóa Dockerfile mới: COPY.



## Xây dựng một số mã C

Chúng tôi muốn xây dựng một container biên dịch chương trình "Hello world" cơ bản bằng C.

Sau đây là chương trình hello.c:

```
int main () { puts("Xin
 chào thế giới!"); return 0;

}
```

Hãy tạo một thư mục mới và đặt tập tin này vào đó.

Sau đó chúng ta sẽ viết Dockerfile.



## Tập Docker

Trên Debian và Ubuntu, gói build-essential sẽ cung cấp cho chúng ta trình biên dịch.

Khi cài đặt, đừng quên chỉ định cờ -y, nếu không quá trình xây dựng sẽ thất bại (vì quá trình xây dựng không thể tương tác).

Sau đó chúng ta sẽ sử dụng lệnh COPY để đưa tệp nguồn vào trong vùng chứa.

TỪ ubuntu CHẠY apt-

get update CHẠY apt-get install -y

build-essential SAO CHÉP hello.c / CHẠY make hello CMD /hello

Tạo Dockerfile này.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

171

Kiểm tra chương trình C của chúng tôi

- Tạo hello.c và Dockerfile trong cùng một thư mục.
- Chạy docker build -t hello . trong thư mục này.

- Chạy lệnh docker run hello, bạn sẽ thấy Hello, world!.

Thành công!



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

172



## COPY và bộ nhớ đệm xây dựng

- Chạy lại bản dựng. •

Bây giờ, hãy sửa đổi hello.c và chạy lại bản dựng. •

Docker có thể lưu trữ đệm các bước liên quan

đến COPY. • Các bước đó sẽ không được thực hiện lại nếu các tệp chưa được thay đổi.



## Chi tiết

- Chúng ta có thể COPY toàn bộ thư mục theo cách

đệ quy • Có thể thực hiện ví dụ COPY . .

(nhưng có thể cần một số biện pháp phòng ngừa bổ sung để tránh sao chép quá nhiều) • Trong Dockerfiles cũ hơn, bạn có thể thấy lệnh ADD; coi như lệnh này đã lỗi thời

(tương tự như COPY nhưng có thể tự động trích xuất các tệp lưu trữ)

- Nếu chúng ta thực sự muốn biên dịch mã C trong một container, chúng ta sẽ:
  - đặt nó vào một thư mục khác, với lệnh WORKDIR
  - thậm chí tốt hơn, sử dụng hình ảnh chính thức của gcc



## .dockerignore

- Chúng ta có thể tạo một tệp có tên `.dockerignore` (ở cấp cao nhất của ngữ cảnh xây dựng)
- Tệp này có thể chứa tên tệp và các khối để bỏ qua
- Chúng sẽ không được gửi đến trình xây dựng (và sẽ không xuất hiện trong hình ảnh kết quả)
- Xem [tài liệu](#) để biết thêm chi tiết (có thể có ngoại lệ với `!`, nhiều cấp thư mục với `**.`)



Hãy cùng viết Dockerfiles cho một ứng dụng hiện có!

1. Kiểm tra kho lưu trữ mã
2. Đọc tất cả các hướng dẫn
3. Viết Dockerfile
4. Xây dựng và thử nghiệm chúng riêng lẻ



## Kho lưu trữ mã

Sao chép kho lưu trữ có sẵn tại:

<https://github.com/jpetazzo/wordsmith>

Nó sẽ trông như thế này:

```
├── GIẤY PHÉP
├── ĐỌC TÔI

├── cơ sở dữ liệu/
│ └── từ.sql
├── trang web/
│ ├── dispatcher.go
│ └── tính/
└── từ/

 ├── pom.xml
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

177

## Hướng dẫn

Kho lưu trữ chứa hướng dẫn bằng tiếng Anh và tiếng Pháp. Hiện tại, chúng ta chỉ quan tâm đến phần đầu tiên (về việc viết Dockerfile). Đặt mỗi Dockerfile vào thư mục riêng của nó, như thế này:

```
├── GIẤY PHÉP
├── ĐỌC TÔI

├── cơ sở dữ liệu/
│ ├── `Dockerfile`
│ └── từ.sql
├── trang web/
│ ├── `Dockerfile`
│ ├── dispatcher.go
│ └── tính/
└── từ/

 ├── `Dockerfile`
 └── pom.xml
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

178

Xây dựng và chạy từng Dockerfile riêng lẻ.

Đối với db, chúng ta có thể thấy một số thông báo xác nhận rằng tập dữ liệu đã được tải thành công (một số dòng INSERT trong đầu ra của vùng chứa).

Đối với web và word, chúng ta có thể thấy một số thông báo trông giống như "máy chủ đã khởi động thành công".

Đó là tất cả những gì chúng ta quan tâm lúc này!

Câu hỏi thường: hãy đảm bảo rằng mỗi container dừng lại đúng cách khi nhấn Ctrl-C. ???



## Kiểm tra với tập Compose

Đặt tập Compose sau vào thư mục gốc của kho lưu trữ:

phiên bản: "3" dịch

vụ: db: xây

dựng: db từ:

xây dựng: từ web: xây

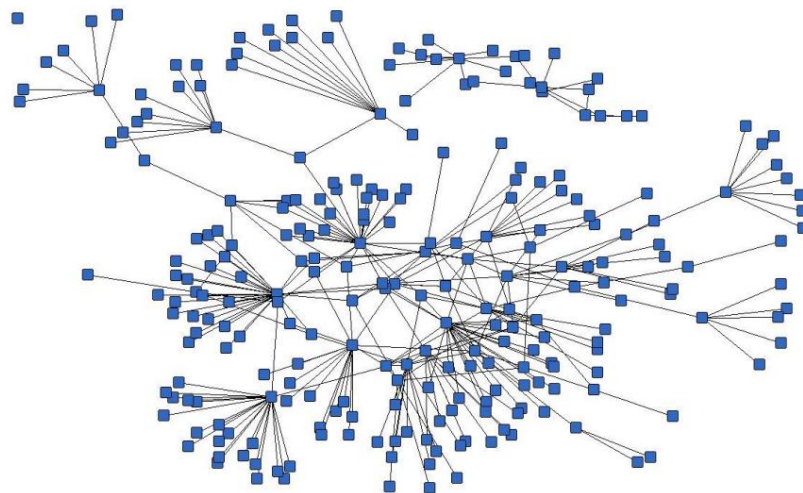
dựng:

web cổng: - 8888:80

Kiểm tra toàn bộ ứng dụng bằng cách đưa ngăn xếp lên và kết nối với cổng 8888.



# Cơ bản về mạng lưới container



Một mạng lưới đồ thị dày đặc

## Mục tiêu

Bây giờ chúng ta sẽ chạy các dịch vụ mạng (chấp nhận yêu cầu) trong các vùng chứa.

Vào cuối phần này, bạn sẽ có thể:

- Chạy dịch vụ mạng trong một vùng chứa.

- Kết nối với dịch vụ mạng đó.
- Tìm địa chỉ IP của container.



## Chạy một dịch vụ rất đơn giản

- Chúng ta cần thứ gì đó nhỏ, đơn giản, dễ cấu hình (hoặc thậm chí tốt hơn, điều đó không yêu cầu bất kỳ cấu hình nào cả)
- Chúng ta hãy sử dụng hình ảnh NGINX chính thức (có tên là nginx) • Nó chạy một máy chủ web tĩnh lắng nghe trên cổng 80 • Nó phục vụ trang mặc định "Chào mừng đến với nginx!"



## Chạy máy chủ NGINX

```
$ docker run -d -P nginx
66b1ce719198711292c8f34f84a7b68c3876cf9f67015e752b9 4e189d35a204e
```

- Docker sẽ tự động kéo hình ảnh nginx từ Docker Hub
- -d / --detach yêu cầu Docker chạy nó ở chế độ nền
- P / --publish-all yêu cầu Docker công bố tất cả các cổng (công bố = làm cho chúng có thể truy cập được từ các máy tính khác)
- OK, bây giờ chúng ta kết nối với máy chủ web như thế nào?



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Tìm cổng máy chủ web của chúng tôi

- Đầu tiên, chúng ta cần tìm số cổng được Docker sử dụng (bộ chứa NGINX lắng nghe trên cổng 80, nhưng cổng này sẽ được ánh xạ)

- Chúng ta có thể sử dụng docker ps: `$ docker ps`

```
CONTAINER ID IMAGE ...
PORTS e40ffb406c9e nginx ... 0.0.0.0:12345->80/tcp ...
```

- Điều này có nghĩa là:

cổng 12345 trên máy chủ Docker được ánh xạ tới cổng 80 trong vùng chứa

- Bây giờ chúng ta cần kết nối với máy chủ Docker!



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Tìm địa chỉ của máy chủ Docker

- Khi chạy Docker trên máy trạm Linux của bạn: sử dụng localhost hoặc bất kỳ địa chỉ IP nào của máy bạn
- Khi chạy Docker trên máy chủ Linux từ xa: sử dụng bất kỳ địa chỉ IP nào của máy từ xa
- Khi chạy Docker Desktop trên Mac hoặc Windows: sử dụng localhost
- Trong các trường hợp khác (docker-machine, VM cục bộ.): sử dụng địa chỉ IP của Docker VM



## Kết nối với máy chủ web của chúng tôi (GUI)

Trở trình duyệt của bạn đến địa chỉ IP của máy chủ Docker, trên cổng được hiển thị bởi docker ps cho cổng container 80.





## Kết nối với máy chủ web của chúng tôi (CLI)

Bạn cũng có thể sử dụng curl trực tiếp từ máy chủ Docker.

Hãy đảm bảo sử dụng đúng số cổng nếu nó khác với ví dụ bên dưới:

```
$ curl localhost:12345
<!DOCTYPEhtml>
<html>
<đầu>
<title>Chào mừng đến với nginx!</title>
...
```



## Docker biết cổng nào cần ánh xạ bằng cách nào?

- Có siêu dữ liệu trong hình ảnh cho biết "hình ảnh này có thứ gì đó trên cổng 80".
- Chúng ta có thể thấy siêu dữ liệu bằng docker inspect:

```
$ docker kiểm tra --format '{{.Config.ExposedPorts}}' nginx
bản đồ[80/tcp:{}]
```

- Siêu dữ liệu này được thiết lập trong Dockerfile, với từ khóa EXPOSE.
- Chúng ta có thể thấy điều đó với docker history:

```
$ docker lịch sử nginx
```

```
HÌNH ẢNH ĐÃ TẠO 7f70b30f2cc6 11 ngày trước ["nginx" "-g" ""]
```

ĐƯỢC TẠO BỞI

```
/bin/sh -c #(nop) CMD
```

```
<thiếu> 11 ngày trước STOPSIGNAL [SIGTERM]
```

```
/bin/sh -c #(không)
```

```
<missing> 11 ngày trước EXPOSE 80/tcp
```

```
/bin/sh -c #(không)
```



## Tại sao chúng ta không thể kết nối tới cổng 80?

- Máy chủ Docker của chúng tôi chỉ có một cổng 80
- Do đó, chúng ta chỉ có thể có một container tại một thời điểm trên cổng 80
- Do đó, nếu nhiều container muốn cổng 80, chỉ có một container có thể nhận được nó
- Theo mặc định, các container không nhận được số cổng “của chúng” mà là một số ngẫu nhiên một  
(không phải “ngẫu nhiên” như “tiền điện tử ngẫu nhiên”, mà là “phụ thuộc vào nhiều yếu tố khác nhau”)
- Chúng ta sẽ xem sau cách áp dụng số cổng (bao gồm cả cổng 80!)



## Sử dụng nhiều địa chỉ IP

Này, mạng lưới quan hệ của tôi rất mạnh và tôi có một số câu hỏi.

- Tôi có thể xuất bản một container trên 127.0.0.2:80 và một container khác trên 127.0.0.3:80 không?
- Máy của tôi có nhiều địa chỉ IP (công cộng), giả sử là AAAA và BBBB. Tôi có thể có một vùng chứa trên AAAA:80 và một vùng chứa khác trên BBBB:80 không?
- Tôi có toàn bộ mạng con IPV4, tôi có thể phân bổ nó cho các vùng chứa của mình không?
- Còn IPV6 thì sao?

Bạn có thể thực hiện tất cả những điều này khi chạy Docker trực tiếp trên Linux.

(Trên các nền tảng khác thì thường là không, nhưng cũng có một số trường hợp ngoại lệ.)



## Tìm cổng máy chủ web trong một tập lệnh

Việc phân tích cú pháp đầu ra của lệnh docker ps sẽ rất khó khăn.

Có một lệnh giúp chúng ta:

```
$ cổng docker <containerID> 80 0.0.0.0:12345
```



## Phân bổ thủ công số cổng

Nếu bạn muốn tự thiết lập số cổng thì không vấn đề gì:

```
$ docker run -d -p 80:80 nginx $ docker run
-d -p 8000:80 nginx $ docker run -d -p 8080:80
-p 8888:80 nginx
```

- Chúng tôi đang chạy ba máy chủ web NGINX. •

Máy chủ đầu tiên được hiển thị trên cổng

80. • Máy chủ thứ hai được hiển thị trên cổng

8000. • Máy chủ thứ ba được hiển thị trên cổng 8080 và 8888.

Lưu ý: quy ước là port-on-host:port-on-container.



## Đưa container vào cơ sở hạ tầng của bạn

Có nhiều cách để tích hợp container vào mạng của bạn.

- Khởi động container, để Docker phân bổ một cổng công khai cho nó. Sau đó lấy số cổng đó và đưa vào cấu hình của bạn.
- Chọn trước một số cổng cố định khi bạn tạo cấu hình. Sau đó, khởi động container của bạn bằng cách thiết lập số cổng theo cách thủ công.
- Sử dụng một bộ điều phối như Kubernetes hoặc Swarm. Bộ điều phối sẽ cung cấp cơ sở hạ tầng mạng riêng.

Bộ điều phối thường cung cấp các cơ chế cho phép giao tiếp trực tiếp giữa các container trên các máy chủ và xuất bản/cân bằng tải cho lưu lượng truy cập đến.



## Tìm địa chỉ IP của container

Chúng ta có thể sử dụng lệnh `docker inspect` để tìm địa chỉ IP của container.

```
$ docker inspect --format
'{{ .NetworkSettings.IPAddress }}' <yourContainerID>
172.17.0.3
```

- `docker inspect` là một lệnh nâng cao có thể thu thập rất nhiều thông tin về container của chúng ta.
- Ở đây, chúng tôi cung cấp cho nó một chuỗi định dạng để trích xuất chính xác thông tin riêng tư Địa chỉ IP của container.



## Ping container của chúng tôi

Hãy thử ping container của chúng ta từ một container khác.

```
docker run alpine ping <địa chỉ IP>`
PING 172.17.0.X (172.17.0.X): 56 byte dữ liệu
64 byte từ 172.17.0.X: seq=0 ttl=64 time=0.106

64 byte từ 172.17.0.X: seq=1 ttl=64 time=0.250

64 byte từ 172.17.0.X: seq=2 ttl=64 time=0.188
```

Khi chạy trên Linux, chúng ta thậm chí có thể ping trực tiếp địa chỉ IP đó!  
(Và kết nối với các cổng của container ngay cả khi chúng chưa được công bố.)



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Chúng ta sử dụng -p và -P thường xuyên như thế nào?

- Khi chạy một ngăn xếp các container, chúng ta thường sử dụng Compose
  - Compose sẽ xử lý việc hiển thị các container  
(thông qua phần ports: trong tệp docker-compose.yml)
  - Tuy nhiên, việc sử dụng docker run -P để kiểm tra nhanh là khá phổ biến
- docker run -p ... đúng cách                      khi một hình ảnh không PHỐI LẠI một cổng • Hoặc



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Tóm tắt phần

Chúng tôi đã học được cách:

- Mở cổng mạng.
- Kết nối với ứng dụng đang chạy trong vùng chứa.
- Tìm địa chỉ IP của container.

???

:EN:- Phơi bày các thùng chứa đơn :FR:- Phơi sáng một lớp cách ly



Quy  
trình phát triển  
cục bộ với

Người lái tàu



Công trường xây dựng



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Mục tiêu

Vào cuối phần này, bạn sẽ có thể:

- Chia sẻ mã giữa vùng chứa và máy chủ.

- Sử dụng quy trình phát triển cục bộ đơn giản.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Phát triển cục bộ trong container

Chúng tôi muốn giải quyết các vấn đề sau: •

“Hoạt động trên máy của tôi” •

“Không phải phiên bản giống nhau”

• “Thiếu sự phụ thuộc”

Bằng cách sử dụng container Docker, chúng ta sẽ có được môi trường phát triển nhất quán.



## Làm việc trên ứng dụng “namer”

• Chúng ta phải làm việc trên một số ứng dụng có mã ở:  
<https://github.com/jpetazzo/namer>.

• Nó là gì? Chúng tôi vẫn chưa biết!

• Hãy tải mã xuống.

```
$ git clone https://github.com/jpetazzo/namer
```





## Nhìn vào mã

```
$ cd namer $ ls
```

```
-1
```

```
company_name_generator.rb config.ru
```

```
docker-
```

```
compose.yml Dockerfile
```

```
Gemfile
```

—

Aha, Gemfile! Đây là Ruby. Có lẽ vậy. Chúng ta biết điều này. Có thể?



ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Nhìn vào Dockerfile

TỪ hồng ngọc

SAO CHÉP . /src

WORKDIR /src CHẠY

cài đặt bundler

CMD ["rackup", "--host", "0.0.0.0"]

TIẾT LỘ 9292

- Ứng dụng này đang sử dụng một hình ảnh ruby cơ sở. •

Mã được sao chép trong /src. • Các

phụ thuộc được cài đặt bằng bundler. • Ứng dụng được

khởi động bằng rackup. • Nó đang lắng nghe trên

cổng 9292.



ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Xây dựng và chạy ứng dụng "namer"

- Hãy cùng xây dựng ứng dụng với Dockerfile!

```
$ docker build -t namer .
```

 • Sau đó chạy

nó. Chúng ta cần phải phơi bày các cổng của nó.

```
$ docker run -dP người đặt tên
```

- Kiểm tra xem container đang lắng nghe cổng nào.

```
$ docker ps -l
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Kết nối với ứng dụng của chúng tôi

- Trở trình duyệt của chúng ta đến nút Docker của chúng ta, trên cổng được phân bổ cho thùng chứa.
- Nhấn "tải lại" một vài lần.
- Đây là công cụ tạo tên công ty đạt chuẩn ISO, cấp nhà mạng và cấp doanh nghiệp !

(Với 50% sự nhầm nhí hơn so với đối thủ cạnh tranh trung bình!)

(Khoan đã, đó là nhiều hơn 50% hay ít hơn 50%? Dù sao thì!) ứng dụng  
web 1 ứng dụng web 1



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Thực hiện thay đổi cho mã

Tùy chọn

1: • Chỉnh sửa mã cục bộ

- Xây dựng lại hình ảnh
- Chạy lại container

Lựa chọn 2:

- Vào container (với docker exec)
- Cài đặt trình soạn thảo
- Thực hiện thay đổi từ bên trong container

Lựa chọn 3:

- Sử dụng bind mount để chia sẻ các tệp cục bộ với container
- Thực hiện thay đổi cục

bộ • Thay đổi được phản ánh trong vùng chứa



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Tập đầu tiên của chúng tôi

Chúng tôi sẽ yêu cầu Docker ánh xạ thư mục hiện tại tới /src trong container.

```
$ docker run -d -v $(pwd):/src -P namer
```

- -d: container phải chạy ở chế độ tách biệt (ở chế độ nền).
- -v: thư mục máy chủ sau phải được gắn bên trong container.
- -P: xuất bản tất cả các cổng được hình ảnh này hiển thị.
- namer là tên của hình ảnh chúng ta sẽ chạy.
- Chúng tôi

- không chỉ định lệnh để chạy vì lệnh này đã được thiết

lập trong Dockerfile thông qua CMD.

Lưu ý: trên Windows, hãy thay \$(pwd) bằng %cd% (hoặc \${pwd} nếu bạn sử dụng PowerShell).



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Lắp khối lượng bên trong container

Cờ -v gắn một thư mục từ máy chủ của bạn vào vùng chứa Docker.

Cấu trúc cờ là:

`[host-path]:[container-path]:[rw|ro] • [host-path]` và

`[container-path]` được tạo nếu chúng không hiện hữu.

- Bạn có thể kiểm soát trạng thái ghi của ổ đĩa bằng ro và rw tùy chọn.
- Nếu bạn không chỉ định rw hoặc ro, thì mặc định sẽ là rw.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Giữ chặt ngựa của bạn. và cả thú cưỡi của bạn

- Cú pháp -v /path/on/host:/path/in/container là cú pháp “cũ”

- Cú pháp hiện đại trông như thế này:

`--mount`

`type=bind,source=/path/on/host,target=/path/in/container • --mount rõ ràng hơn, nhưng -v gọn gàng hơn • --mount hỗ trợ`

tất cả các loại gắn kết; -v không hỗ trợ gắn kết tmpfs • --mount không thành công nếu đường dẫn

trên máy chủ không tồn tại; -v tạo đường dẫn đó

Với cú pháp mới, lệnh của chúng ta trở thành:

`docker run --`

`mount=type=bind,source=$(pwd),target=/src -dP namer`



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Kiểm tra container phát triển

- Kiểm tra cổng mà container mới của chúng tôi sử dụng.

```
$ docker ps -l
```

CONTAINER ID	IMAGE	LỆNH ĐÃ TẠO
--------------	-------	-------------

TRẠNG THÁI
------------

CỔNG
------

TÊN
-----

045885b68bc5	tên người rackup 3 giây trước	Lên ... 0.0.0.0:32770->9292/tcp ...
--------------	-------------------------------	-------------------------------------

- Mở ứng dụng trong trình duyệt web của bạn.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Thực hiện thay đổi cho ứng dụng của chúng tôi

Khách hàng của chúng tôi thực sự không thích màu chữ của chúng tôi. Hãy thay đổi nó.

```
$ vi company_name_generator.rb
```

Và thay đổi

```
màu sắc: xanh hoàng gia;
```

ĐẾN:

```
màu sắc: đỏ;
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Xem những thay đổi của chúng tôi

- Tải lại ứng dụng trong trình duyệt của chúng tôi.

–

- Màu sắc phải thay đổi.

ứng dụng web 2 ứng

dụng web 2



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Hiểu về khối lượng

- Các ổ đĩa không sao chép hoặc đồng bộ hóa các tập tin giữa máy chủ và thùng chứa
- Những thay đổi được thực hiện trên máy chủ sẽ hiển thị ngay lập tức trong vùng chứa (và

ngược lại) • Khi chạy trên Linux:

- các ổ đĩa và các mount liên kết tương ứng với các thư mục trên máy chủ
- nếu Docker chạy trong máy ảo Linux, các thư mục này sẽ nằm trong máy ảo Linux

- Khi chạy trên Docker Desktop: • các ổ đĩa tương

ứng với các thư mục trong một máy ảo Linux nhỏ đang chạy Docker • quyền truy cập vào các liên kết gắn kết được dịch sang quyền truy cập hệ thống tệp lưu trữ (giống như một hệ thống tập tin mạng)



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Những lưu ý khi sử dụng Docker Desktop

- Khi chạy Docker gốc trên Linux, truy cập mount = gốc

Đầu vào/Đầu ra

- Khi chạy Docker Desktop, truy cập bind mount = truy cập tệp dịch thuật
- Việc dịch truy cập tệp đó có hiệu suất tương đối tốt nói chung (tuy nhiên, hãy cẩn thận vì lệnh npm install lớn này hoạt động trên bind mount!)
- Có một số trường hợp ngoại lệ khi xem tệp (với các cơ chế như inotify)
- Các tính năng như “tải lại trực tiếp” hoặc các chương trình như entr không phải lúc nào cũng hoạt động bình thường (ví dụ như do lưu trữ đệm thuộc tính tệp và các chi tiết thú vị khác!)



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Xóa máy chủ của bạn và đốt mã của bạn

(Đây là tiêu đề [bài đăng trên blog năm 2013](#) của Chad Fowler, trong đó ông giải thích khái niệm về cơ sở hạ tầng bất biến.)

- Hãy làm hỏng thùng chứa của chúng ta.  
(Xóa tệp hoặc bất cứ thứ gì.)
- Bây giờ, chúng ta có thể khắc phục điều này như thế nào?
- Container cũ của chúng tôi (có phiên bản mã màu xanh) vẫn đang chạy.
- Xem nó được hiển thị trên cổng nào:  
docker ps
- Trỏ trình duyệt của chúng tôi tới đó để xác nhận rằng nó vẫn hoạt động tốt.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Tóm lại là cơ sở hạ tầng bất biến

- Thay vì cập nhật máy chủ, chúng tôi triển khai máy chủ mới.
- Điều này có thể là thách thức với các máy chủ cổ điển, nhưng nó lại dễ dàng với thùng chứa.
- Trên thực tế, với Docker, quy trình làm việc hợp lý nhất là xây dựng một hình ảnh mới và chạy nó.
- Nếu có vấn đề gì xảy ra với hình ảnh mới, chúng ta luôn có thể khởi động lại cái cũ.
- Chúng ta thậm chí có thể chạy cả hai phiên bản song song.

Nếu mô hình này nghe có vẻ thú vị, bạn có thể muốn đọc về triển khai xanh lam/xanh lục và triển khai canary.



## Tóm tắt quy trình phát triển

1. Viết Dockerfile để xây dựng một hình ảnh chứa quá trình phát triển của chúng tôi môi trường. (Rails, Django, . và tất cả các phụ thuộc cho ứng dụng của chúng tôi)
2. Bắt đầu một container từ hình ảnh đó. Sử dụng cờ -v để gắn kết mã nguồn bên trong container.
3. Chỉnh sửa mã nguồn bên ngoài vùng chứa bằng các công cụ quen thuộc (vim, emacs, textmate.)
4. Kiểm tra ứng dụng. (Một số khuôn khổ chọn thay đổi tự động. Những cách khác yêu cầu bạn phải Ctrl-C + khởi động lại sau mỗi lần sửa đổi.)
5. Lặp lại các bước 3 và 4 cho đến khi hài lòng.
6. Khi hoàn tất, hãy cam kết + đẩy những thay đổi về mã nguồn.





## Gỡ lỗi bên trong container

Docker có một lệnh gọi là `docker exec`.

Nó cho phép người dùng chạy một tiến trình mới trong một vùng chứa đang chạy.

Nếu đôi khi bạn muốn SSH vào một container: bạn có thể sử dụng `docker exec` thay thế.

Bạn có thể nhận được dấu nhắc shell bên trong một vùng chứa hiện có theo cách này hoặc chạy một quy trình tùy ý để tự động hóa.



## ví dụ docker exec

\$ # Bạn có thể chạy lệnh ruby trong khu vực ứng dụng đang chạy và hơn thế nữa!

```
$ docker exec -it <yourContainerId> bash
```

```
root@5ca27cf74c2e:/opt/namer# irb
```

```
irb(chính):001:0> [0, 1, 2, 3, 4].bản đồ {|x| x * 2}.compact
```

\*\*

```
=> [0, 1, 4, 9, 16]
```

```
irb(chính):002:0> thoát
```



## Dừng container

Bây giờ chúng ta đã hoàn tất, hãy dừng container của mình lại. `$ docker stop <yourContainerID>`

Và xóa nó đi.

```
$ docker rm <yourContainerID>
```



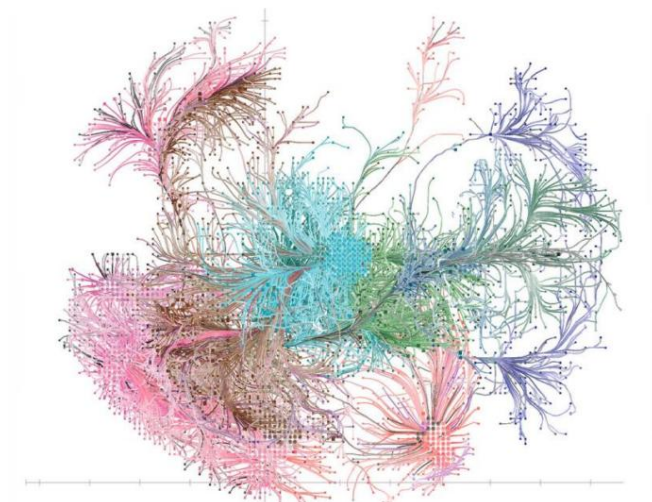
## Tóm tắt phần

Chúng tôi đã học được cách:

- Chia sẻ mã giữa vùng chứa và máy chủ.
- Thiết lập thư mục làm việc của chúng tôi.
- Sử dụng quy trình phát triển cục bộ đơn giản.



# Thùng chứa Mô hình mạng



Một mạng lưới đồ thị dày đặc hơn

## Mục tiêu

Chúng ta sẽ tìm hiểu về CNM (Mô hình mạng lưới container).

Vào cuối bài học này, bạn sẽ có thể:

- Tạo mạng riêng cho một nhóm container.
- Sử dụng tên vùng chứa để kết nối các dịch vụ với nhau.
- Kết nối và ngắt kết nối container với mạng một cách động.
- Đặt địa chỉ IP của vùng chứa.

Chúng tôi cũng sẽ giải thích nguyên lý của mạng phủ và plugin mạng.



## Mô hình mạng lưới container

Docker có "mạng lưới".

Chúng ta có thể quản lý chúng bằng lệnh `docker network`; ví dụ:

```
$ docker network ls
```

ID MẠNG	CẦU TÊN	TÀI XẾ
6bde79dfcf70		cầu
8d9c78725538	không có	vô giá trị
eb0eeab782f4	blog-	chủ nhà
4c1ff84d6d3f	dev blog-prod	lớp phủ
228a4355d548	máy chủ	lớp phủ

Có thể tạo mạng mới (bằng lệnh `docker network create`).

(Lưu ý: mạng không có và máy chủ là đặc biệt; chúng ta hãy tạm gác chúng sang một bên.)



## Mạng là gì?

- Về mặt khái niệm, “mạng” Docker là một công tắc ảo  
(chúng ta cũng có thể nghĩ về nó như một VLAN hoặc một SSID WiFi chẳng hạn)
- Theo mặc định, các container được kết nối với một mạng duy nhất  
(nhưng chúng có thể được kết nối với không hoặc nhiều mạng, thậm chí là động)
- Mỗi mạng có mạng con riêng (phạm vi địa chỉ IP)
- Mạng có thể là cục bộ (đến một Docker Engine duy nhất) hoặc toàn cầu (mở rộng trên nhiều máy chủ)
- Các container có thể có các bí danh mạng cung cấp dịch vụ khám phá dựa trên DNS  
  
(và mỗi mạng có “miền”, “khu vực” hoặc “phạm vi” riêng)



## Khám phá dịch vụ

- Một container có thể được cấp một bí danh mạng  
(ví dụ với lệnh `docker run --net some-network --net-alias db ...`)
- Các container chạy trong cùng một mạng có thể giải quyết mạng đó  
bí danh  
(tức là nếu họ thực hiện tra cứu DNS trên db, nó sẽ cung cấp địa chỉ của container)
- Chúng ta có thể có một vùng chứa db khác nhau trong mỗi mạng  
(điều này tránh xung đột tên giữa các ngăn xếp khác nhau)
- Khi chúng ta đặt tên cho một container, nó sẽ tự động thêm tên dưới dạng  
bí danh mạng  
(tức là `docker run --name xyz ...` giống như `docker run --net-alias xyz ...`)



## Cô lập mạng

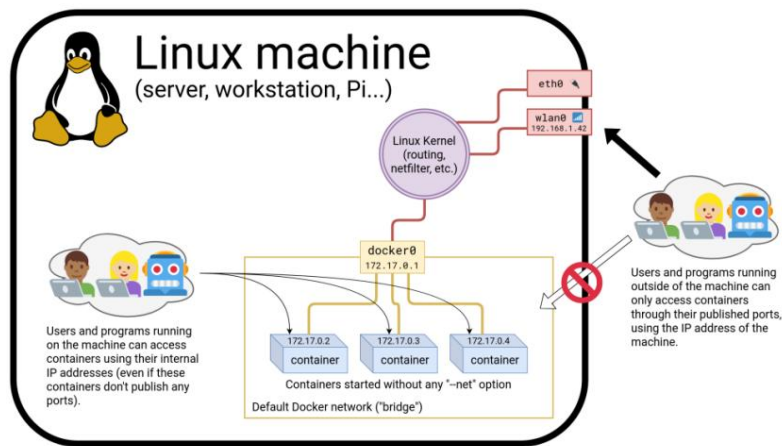
- Mạng bị cô lập
- Theo mặc định, các container trong mạng A không thể tiếp cận được các container trong mạng B
- Một container được kết nối với cả mạng A và B có thể hoạt động như một bộ định tuyến hoặc người đại diện
- Các cổng đã xuất bản luôn có thể truy cập được thông qua địa chỉ máy chủ Docker (docker run -P giúp mọi người có thể truy cập vào cổng container)



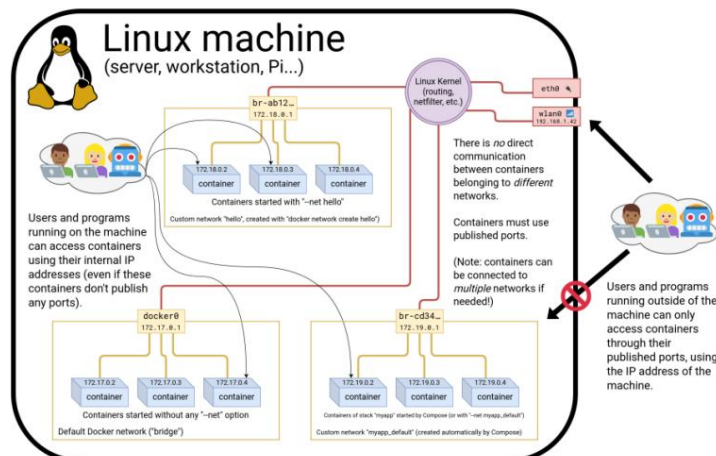
## Cách sử dụng mạng

- Chúng tôi thường tạo một mạng cho mỗi "ngăn xếp" hoặc ứng dụng mà chúng tôi triển khai
- Các ứng dụng hoặc ngăn xếp phức tạp hơn có thể yêu cầu nhiều mạng (ví dụ: giao diện người dùng, giao diện quản trị,..)
- Mạng cho phép chúng tôi triển khai nhiều bản sao của cùng một ngăn xếp (ví dụ: prod, dev, pr-442,..)
- Nếu chúng ta sử dụng Docker Compose, điều này được quản lý tự động cho chúng ta



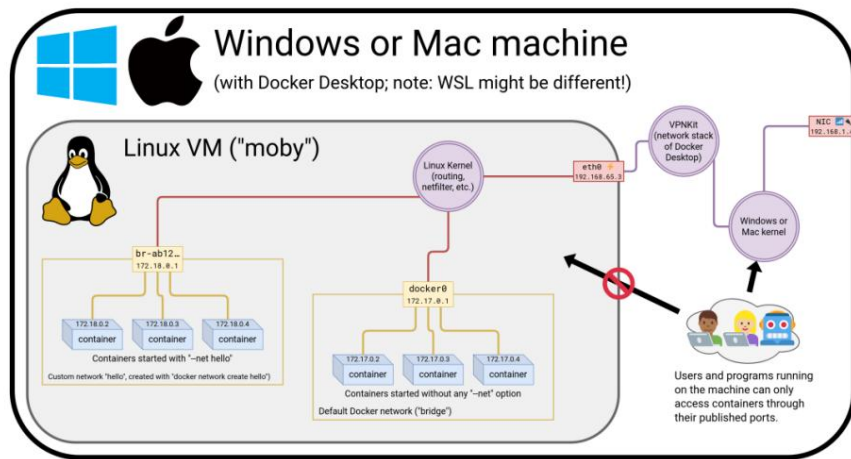


Nhiều container trên mạng cầu nối mặc định, trên máy Linux



Nhiều container trong nhiều mạng cầu nối, trên một máy Linux





Nhiều container trong nhiều mạng cầu nối, trên máy Mac/Windows



## CNM so với CNI

- CNM là mô hình được Docker sử dụng •

Kubernetes sử dụng một mô hình khác, được kiến trúc xung quanh CNI

(CNI là một loại API nằm giữa công cụ chứa và plugin CNI)

- Mô hình Docker:

- nhiều mạng bị cô lập • khám phá dịch vụ trên từng mạng • kết nối mạng đòi hỏi các bước bổ sung

- Mô hình Kubernetes:

- mạng phẳng đơn • khám phá dịch vụ theo không gian tên • cô lập mạng yêu cầu các bước bổ sung (Chính sách mạng)





## Tạo mạng lưới

Hãy tạo một mạng có tên là dev.

```
$ docker network tạo dev
4c1ff84d6d3f1733d3e233ee039cac276f425a9d5228a4355d5
4878293a889ba
```

Bây giờ bạn có thể nhìn thấy mạng bằng lệnh network ls:

```
$ docker mạng ls
```

ID MẠNG	TÊN	TÀI XẾ
6bde79dfcf70	cầu	cầu
8d9c78725538	không có	vô giá trị
eb0eeab782f4	máy chủ	chủ nhà
4c1ff84d6d3f	phát triển	cầu



## Đặt container trên mạng

Chúng ta sẽ tạo một vùng chứa có tên trên mạng này.

Nó sẽ có thể truy cập được bằng tên của nó, es.

```
$ docker run -d --name es --net dev elasticsearch:2
8abb80e229ce8926c7223beb69699f5f34d6f1d438bfc5682db
893e798046863
```



## Giao tiếp giữa các container

Bây giờ, hãy tạo một container khác trên mạng này.

```
$ docker run -ti --net dev alpine sh
root@0ecccdfa45ef:/#
```

Từ container mới này, chúng ta có thể giải quyết và ping container kia bằng cách sử dụng tên được chỉ định của nó:

```
/ # ping es PING es
(172.18.0.2) 56(84) byte dữ liệu. 64 byte từ es.dev (172.18.0.2): icmp_seq=1
ttl=64 thời gian=0,221 ms 64 byte từ es.dev (172.18.0.2): icmp_seq=2 ttl=64 thời gian=0,114 ms 64 byte từ es.dev (172.18.0.2):
icmp_seq=3 ttl=64 thời gian=0,114 ms ^C --- es ping statistics --- 3 gói tin được truyền, 3 gói tin được nhận, 0% mất gói tin,
thời gian 200ms rtt min/avg/max/mdev = 0,114/0,149/0,221/0,052 ms root@0ecccdfa45ef:/#
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Giải quyết địa chỉ container

Kể từ Docker Engine 1.10, việc phân giải tên được thực hiện bằng trình phân giải động.

Ghi chú khảo cổ học: khi CNM được giới thiệu (trong Docker Engine 1.9, tháng 11 năm 2015), giải quyết tên được triển khai với /etc/hosts và được cập nhật mỗi lần CONTAINER được thêm vào/xóa. Điều này có thể gây ra tình trạng chạy đua thúi vị vì /etc/hosts là bind-mount (và không thể cập nhật nguyên tử).

```
[root@0ecccdfa45ef /]# cat /etc/hosts 172.18.0.3
0ecccdfa45ef 127.0.0.1 localhost ::1
fe00::0 ip6-localnet ff00::0 ip6-
localhost ip6-localhost ip6-loopback
mcastprefix ff02::1 ip6-allnodes
ff02::2 ip6-allrouters 172.18.0.2
172.18.0.2
```

là  
es.dev



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

# Khám phá dịch vụ với container

- Hãy thử chạy một ứng dụng yêu cầu hai vùng chứa.
- Container đầu tiên là một máy chủ web.
- Cái còn lại là kho dữ liệu redis.
- Chúng tôi sẽ đặt cả hai vào mạng dev đã tạo trước đó.

## Chạy máy chủ web

- Ứng dụng được cung cấp bởi hình ảnh chứa jpetazzo/trainingwheels.
- Chúng tôi không

biết nhiều về nó nên chúng tôi sẽ thử chạy nó và xem điều gì xảy ra!

Khởi động container, hiển thị tất cả các cổng của nó:

```
$ docker run --net dev -d -P jpetazzo/trainingwheels
```

Kiểm tra cổng đã được phân bổ cho nó: \$

```
docker ps -l
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Kiểm tra máy chủ web

- Nếu chúng ta kết nối với ứng dụng ngay bây giờ, chúng ta sẽ thấy một trang lỗi:

- Nguyên nhân là do dịch vụ Redis không chạy.
- Container này cố gắng giải quyết tên redis.

Lưu ý: chúng tôi không sử dụng FQDN hoặc địa chỉ IP ở đây; chỉ sử dụng redis.



Lỗi bánh xe tập



ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Bắt đầu lưu trữ dữ liệu

- Chúng ta cần khởi động một container Redis.
- Container đó phải nằm trên cùng một mạng với máy chủ web.
- Nó phải có đúng bí danh mạng (redis) để ứng dụng có thể tìm thấy Nó.

Bắt đầu container:

```
$ docker run --net dev --net-alias redis -d redis
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Kiểm tra lại máy chủ web

- Nếu chúng ta kết nối với ứng dụng ngay bây giờ, chúng ta sẽ thấy ứng dụng đang hoạt động chính xác:
- Khi ứng dụng cố gắng giải quyết redis, thay vì nhận được lỗi DNS, nó sẽ nhận được địa chỉ IP của vùng chứa Redis của chúng ta.

### Training wheels

This request was served by f927b966d8e5.  
f927b966d8e5 served 1 request so far.

The current ladder is:

- f927b966d8e5 → 1 request

Bánh xe tập đi OK



ĐẠI HỌC BÁCH KHOA HÀ NỘI  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Một vài lời về phạm vi

- Tên vùng chứa là duy nhất (chỉ có thể có một `--name redis`)
- Bí danh mạng không duy nhất
- Chúng ta có thể có cùng một bí danh

mạng trong các mạng khác nhau:

```
docker run --net dev --net-alias redis ... docker run --net prod
--net-alias redis ...
```

- Chúng ta thậm chí có thể có nhiều container có cùng bí danh trong cùng một mạng (trong

trường hợp đó, chúng ta nhận được nhiều mục nhập DNS, còn gọi là “DNS vòng tròn”)



## Tên là cục bộ cho mỗi mạng

Hãy thử ping container `es` của chúng ta từ một container khác khi container đó không nằm trên mạng `dev`.

```
$ docker run --rm alpine ping es ping: địa chỉ 'es'
không hợp lệ
```

Tên chỉ có thể được phân giải khi các container nằm trên cùng một mạng.

Các container chỉ có thể liên lạc với nhau khi chúng nằm trên cùng một mạng (bạn có thể thử ping bằng địa chỉ IP để xác minh).



## Bí danh mạng

Chúng tôi muốn có một mạng khác, prod, với container es riêng. Nhưng chỉ có thể có một container có tên là es!

Chúng tôi sẽ sử dụng bí danh mạng.

Một container có thể có nhiều bí danh mạng.

Bí danh mạng là bí danh cục bộ của một mạng nhất định (chỉ tồn tại trong mạng này).

Nhiều container có thể có cùng bí danh mạng (thậm chí trên cùng một mạng).

Kể từ Docker Engine 1.11, việc giải quyết bí danh mạng sẽ mang lại địa chỉ IP của tất cả các container chứa bí danh này.



## Tạo container trên mạng khác

Tạo mạng lưới sản xuất.

```
$ docker network tạo prod
5a41562fecf2d8f115bedc16865f7336232a04268bdf2bd816aecca01b68 d50c
```

Bây giờ chúng ta có thể tạo nhiều container với bí danh es trên mạng prod mới.

```
$ docker run -d --name prod-es-1 --net-alias es --net prod elasticsearch:2
```

```
38079d21caf0c5533a391700d9e9e920724e89200083df73211081c8a356 d771 $ docker run -d --name prod-
es -2
--net-alias es --net prod elasticsearch:2 1820087a9c600f43159688050dcc164c298183e1d2e62d5694fd46b10
bc3d
```



## Giải quyết các bí danh mạng

Trước tiên, chúng ta hãy thử phân giải DNS bằng cách sử dụng công cụ nslookup đi kèm với hình ảnh alpine.

```
$ docker run --net prod --rm alpine nslookup es
```

Tên: là

Địa chỉ 1: 172.23.0.3 prod-es-2.prod Địa chỉ 2: 172.23.0.2

prod-es-1.prod (Bạn có thể bỏ qua lỗi không thể giải quyết

'(null)'.)



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Kết nối với các container có bí danh

Mỗi phiên bản Elasticsearch đều có một tên (được tạo khi phiên bản đó được khởi động). Tên này có thể được nhìn thấy khi chúng ta đưa ra một yêu cầu HTTP đơn giản trên điểm cuối API Elasticsearch .

Hãy thử lệnh sau một vài lần:

```
$ docker run --rm --net dev centos curl -s es:9200 {
```

```
"tên" : "Tarot",
```

```
...
```

```
}
```

Sau đó hãy thử lại vài lần bằng cách thay thế --net dev bằng --net prod:

```
$ docker run --rm --net prod centos curl -s es:9200 {
```

```
"tên" : "Symbiote" ,
```

```
...
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



Thật tốt khi biết điều đó.

- Docker sẽ không tạo tên mạng và bí danh trên cầu nối mặc định mạng.
  - Do đó, nếu bạn muốn sử dụng các tính năng đó, bạn phải tạo một tùy chỉnh mạng đầu tiên.
  - Các bí danh mạng không phải là duy nhất trên một mạng nhất định.
  - nghĩa là nhiều container có thể có cùng một bí danh trên cùng một mạng.
  - Trong trường hợp đó, máy chủ DNS Docker sẽ trả về nhiều bản ghi. (tức là bạn sẽ nhận được DNS round robin ngay lập tức.)
  - Bật chế độ Swarm cho phép truy cập vào cụm và cân bằng tải với
- Bảo vệ môi trường (ZPV).
- Việc tạo mạng và bí danh mạng thường được tự động hóa bằng các công cụ



như Compose  
**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Một vài lời về DNS vòng tròn

Đừng chỉ dựa vào DNS vòng tròn để đạt được cân bằng tải.

Nhiều yếu tố có thể ảnh hưởng đến độ phân giải DNS và bạn có thể thấy:

- tất cả lưu lượng truy cập đến một phiên bản duy nhất;
- lưu lượng được chia tách (không đều) giữa một số trường hợp;
- hành vi khác nhau tùy thuộc vào ngôn ngữ ứng dụng của bạn;
- hành vi khác nhau tùy thuộc vào bản phân phối cơ sở của bạn;
- hành vi khác nhau tùy thuộc vào các yếu tố khác (sic).

Bạn có thể sử dụng DNS để khám phá các điểm cuối khả dụng, nhưng hãy nhớ rằng thỉnh thoảng bạn phải phân giải lại để khám phá các điểm cuối mới.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Mạng tùy chỉnh

Khi tạo mạng, có thể cung cấp thêm các tùy chọn. • `--internal` vô hiệu hóa lưu lượng truy cập đi (mạng sẽ không có cổng mặc định).

- `--gateway` chỉ ra địa chỉ nào được sử dụng cho cổng (khi lưu lượng truy cập đi được phép).
- `--subnet` (theo ký hiệu CIDR) chỉ ra mạng con được sử dụng.
- `--ip-range` (theo ký hiệu CIDR) chỉ ra mạng con được phân bổ từ đó.
- `--aux-address` cho phép chỉ định danh sách các địa chỉ được đặt trước (sẽ không được phân bổ cho các vùng chứa).



## Thiết lập địa chỉ IP của container

- Có thể thiết lập địa chỉ của container bằng `--ip`.
- Địa chỉ IP phải nằm trong mạng con được sử dụng cho container.

Một ví dụ đầy đủ sẽ trông như thế này.

```
$ docker network create --subnet 10.66.0.0/16 pubnet
42fb16ec412383db6289a3e39c3c0224f395d7f85bcb1859b279e7 a564d4e135 $ docker run --net
pubnet --ip
10.66.66.66 -d nginx b2887adeb5578a01fd9c55c435cad56bbbe802350711d2743691f9
5743680b09
```

Lưu ý: không mã hóa cứng địa chỉ IP của container trong mã của bạn!

Tôi nhắc lại: đừng mã hóa cứng địa chỉ IP của container trong mã của bạn!



## Trình điều khiển mạng

- Mạng được quản lý bởi trình điều khiển.
- Các trình điều khiển tích hợp bao gồm:
  - cầu (mặc định)
  - không có
  - chủ nhà
  - macvlan
  - lớp phủ (cho các cụm Swarm)
- Có thể cung cấp thêm nhiều trình điều khiển hơn bằng plugin (OVS, VLAN.)
- Một mạng có thể có một IPAM (bộ phân bổ IP) tùy chỉnh.



## Mạng lưới chồng chéo

- Các tính năng chúng ta thấy cho đến nay chỉ hoạt động khi tất cả các container đều nằm trên một máy chủ duy nhất.
- Nếu các container trải dài trên nhiều máy chủ, chúng ta cần một mạng phủ để kết nối chúng lại với nhau.
- Docker được tích hợp sẵn plugin mạng mặc định, overlay, triển khai mạng overlay tận dụng VXLAN, được kích hoạt bằng Chế độ Swarm.
- Các plugin khác (Weave, Calico.) cũng có thể cung cấp mạng lưới phủ.
- Khi bạn đã có mạng phủ, tất cả các tính năng chúng tôi đã sử dụng trong chương này đều hoạt động giống hệt nhau trên nhiều máy chủ.



## Mạng đa máy chủ (lớp phủ)

Nằm ngoài phạm vi của hội thảo giới thiệu này!

Hướng dẫn rất ngắn gọn:

- bật Chế độ Swarm (docker swarm init sau đó docker swarm join trên các nút khác)
- docker network tạo mynet --driver lớp phủ
- docker dịch vụ tạo --network mynet myimage

Nếu bạn muốn tìm hiểu thêm về chế độ Swarm, bạn có thể xem [video này](#) hoặc [những slide này](#).



## Mạng đa máy chủ (plugin)

Nằm ngoài phạm vi của hội thảo giới thiệu này!

Ý tưởng chung:

- cài đặt plugin (chúng thường được vận chuyển trong các thùng chứa)
- chạy plugin (nếu nằm trong container, plugin thường yêu cầu thêm tham số; đừng chỉ chạy docker một cách mù quáng!)
- một số plugin yêu cầu cấu hình hoặc kích hoạt (tạo một tệp đặc biệt thông báo cho Docker "sử dụng plugin có ổ cắm điều khiển ở vị trí sau")
- sau đó bạn có thể docker network create --driver pluginname



## Kết nối và ngắt kết nối động

- Cho đến nay, chúng tôi đã chỉ định mạng nào sẽ sử dụng khi bắt đầu thùng chứa.
- Docker Engine cũng cho phép kết nối và ngắt kết nối trong khi container đang chạy.
- Tính

năng này được thể hiện thông qua Docker API và thông qua hai  
Các lệnh Docker CLI:

- `docker network connect <mạng> <container>`
- `ngắt kết nối mạng docker <mạng> <container>`



## Kết nối động với mạng

- Chúng ta có một container có tên es được kết nối với một mạng có tên dev.
- Hãy bắt đầu một container alpine đơn giản trên mạng mặc định:

```
$ docker run -ti alpine sh / #
```

- Trong vùng chứa này, hãy thử ping vùng chứa es: / # `ping es` ping: bad  
address 'es'

Cách này không hiệu quả, nhưng chúng ta sẽ thay đổi bằng cách kết nối container.



## Tìm ID container và kết nối nó

- Tìm ra ID của container alpine; sau đây là hai phương pháp: •
  - xem `/etc/hostname` trong container, •
  - chạy `docker ps -lq` trên máy chủ.
- Chạy lệnh sau trên máy chủ: `$ docker network connect dev <container_id>`



## Kiểm tra những gì chúng tôi đã làm

- Thử lại ping es từ container. • Bây giờ nó sẽ hoạt động

chính xác: / # `ping es` PING es (172.20.0.3):

56 byte dữ liệu

64 byte từ 172.20.0.3: seq=0 ttl=64 time=0.376 ms 64 byte

từ 172.20.0.3: seq=1 ttl=64 time=0.130 ms ^C

- Ngắt nó bằng Ctrl-C.



## Nhìn vào thiết lập mạng trong container

Chúng ta có thể xem danh sách các giao diện mạng bằng `ifconfig`, `ip a` hoặc `ip l`:

```
/ # ip a 1:
lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc trạng thái noqueue KHÔNG XÁC ĐỊNH qlen 1000 liên kết/vòng lặp lại 00:00:00:00:00:00
 brd 00:00:00:00:00:00 inet 127.0.0.1/8 phạm vi máy chủ lo valid_lft mãi mãi preferred_lft
 mãi mãi

18: eth0@if19: <PHÁT SÓNG, NHIỀU PHÁT SÓNG, LÊN, HẠ THẤP LÊN, M-XUỐNG> mtu 1500 qdisc trạng thái không có hàng đợi LÊN
 liên kết/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff inet 172.17.0.2/16 brd
 172.17.255.255 phạm vi toàn cầu eth0 valid_lft mãi mãi preferred_lft mãi mãi

20: eth1@if21: <PHÁT SÓNG, NHIỀU PHÁT SÓNG, LÊN, HẠ THẤP LÊN, M-XUỐNG> mtu 1500 qdisc trạng thái không có hàng đợi LÊN
 liên kết/ether 02:42:ac:14:00:04 brd ff:ff:ff:ff:ff:ff inet 172.20.0.4/16 brd
 172.20.255.255 phạm vi toàn cầu eth1 valid_lft mãi mãi preferred_lft mãi mãi

/ #
```

Mỗi kết nối mạng được thực hiện bằng một giao diện mạng ảo.

Như chúng ta thấy, chúng ta có thể kết nối với nhiều mạng cùng một lúc.



## Ngắt kết nối khỏi mạng

- Hãy thử lệnh đối xứng để ngắt kết nối container: `$ docker network disconnect dev <container_id>`
- Từ bây giờ, nếu chúng ta thử ping es, nó sẽ không giải quyết được: 

```
/ # ping es ping:
bad address 'es'
```
- Cố gắng ping trực tiếp địa chỉ IP cũng không có tác dụng: 

```
/ # ping
172.20.0.3 ... (không có gì
xảy ra cho đến khi chúng ta ngắt nó bằng Ctrl-C)
```



## Các bí danh mạng được giới hạn theo từng mạng

- Mỗi mạng có một bộ bí danh mạng riêng.
- Chúng ta đã thấy điều này trước đó: es giải quyết các địa chỉ khác nhau trong dev và sản xuất
- Nếu chúng ta kết nối với nhiều mạng, trình phân giải sẽ tra cứu tên trong từng mạng (kể từ Docker Engine 18.03, đây là thứ tự kết nối) và dừng ngay khi tìm thấy tên.
- Do đó, nếu chúng ta kết nối với cả dev và prod, việc giải quyết es sẽ không cung cấp cho chúng ta địa chỉ của tất cả các dịch vụ es; mà chỉ cung cấp những dịch vụ trong dev hoặc prod.
- Tuy nhiên, chúng ta có thể tra cứu es.dev hoặc es.prod nếu cần.



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Tìm hiểu về mạng lưới và tên của chúng tôi

- Chúng tôi có thể thực hiện tra cứu DNS ngược trên địa chỉ IP của container.
- Nếu địa chỉ IP thuộc về một mạng (khác với cầu nối mặc định), kết quả sẽ là:

tên-hoặc-bí danh-đầu-tiên-hoặc-id-vùng-chứa.tên-mạng

- Ví dụ:

```
$ docker run -ti --net prod --net-alias xin chào Alpine
/ # apk thêm --không có bộ nhớ đệm khoan
...
OK: 5 MiB trong 13 gói
/ # nếu cấu hình
eth0 Liên kết encap:Ethernet HWaddr 02:42:AC:15:00:03
 địa chỉ inet : `172.21.0.3` Bcast: 172.21.255.255 Mặt nạ: 255.255.0.0
 ĐANG CHẠY PHÁT SÓNG ĐA PHÁT MTU:1500 Số liệu:1
...
/ # khoan -t ptr `3.0.21.172`.in-addr.arpa
...
;; PHẦN TRẢ LỜI:
3.0.21.172.in-addr.arpa. 600 TRONG PTR `hello.prod`.
```



**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



## Xây dựng với mạng lưới tùy chỉnh

- Chúng ta có thể xây dựng một Dockerfile với một mạng tùy chỉnh với docker xây dựng --mạng TÊN.
- Có thể sử dụng tính năng này để kiểm tra xem bản dựng có truy cập vào mạng hay không.  
(Nhưng hãy nhớ rằng hầu hết Dockerfile sẽ không thành công vì chúng cần cài đặt các gói và phụ thuộc từ xa!)
- Có thể sử dụng để truy cập kho lưu trữ gói nội bộ.  
(Nhưng hãy thử sử dụng bản dựng nhiều giai đoạn nếu có thể!)



- YAML không phải là ngôn ngữ đánh dấu (theo [yaml.org](https://yaml.org)) • Gần như bắt buộc khi

làm việc với các vùng chứa: • Tập Docker Compose • Bản kê khai

Kubernetes • Nhiều đường ống

CI (GitHub, GitLab.) • Nếu

bạn không biết nhiều về YAML, thì đây là dành

cho bạn!



## Đó là gì?

- Ngôn ngữ biểu diễn dữ liệu

- quốc gia: Pháp thủ đô: Paris mã vùng: fr dân số: 68042591

- quốc gia: Đức thủ đô: Berlin mã thành phố: de dân số: 84270625
    - quốc gia: Na

- Uy thủ đô: Oslo mã thành phố: no # Đây là một cái bẫy! dân số: 5425270



• Ngay cả khi không biết YAML, chúng ta vẫn có thể thêm một quốc gia vào tệp đó :)

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**  
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## Đang thử YAML

- Phương pháp 1: trong trình

- duyet <https://onlineyamltools.com/convert-yaml-to-json> <https://onlineyamltools.com/highlight-yaml>

- Phương pháp 2: trong shell

- yq.foo.yaml

- Phương pháp 3: trong Python

- ```
import yaml; yaml.safe_load(""" - quốc gia: Pháp
thủ đô: Paris """)
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Những thứ cơ bản

- Chuỗi, số, giá trị boolean, null • Chuỗi (=mảng, danh sách) • Ánh xạ (=đối tượng)
- Siêu tập của JSON

(nếu bạn biết JSON, bạn có thể chỉ cần viết JSON) • Bình luận bắt đầu bằng #

- Một tệp có thể có nhiều tài liệu (được phân tách bằng --- trên một dòng)



Trình tự

- Ví dụ: chuỗi các chuỗi ["Pháp", "Đức",
"Na Uy"] • Ví dụ: chuỗi tương tự, không có dấu ngoặc kép
[Pháp, Đức, Na Uy]
- Ví dụ: cùng một chuỗi, trong "kiểu tập hợp khối" (=nhiều dòng)
 - Pháp
 - Đức - Na Uy



Ảnh xạ

- Ví dụ: ánh xạ chuỗi thành số { "Pháp": 68042591, "Đức": 84270625, "Na Uy": 5425270 }
- Ví dụ: cùng một ánh xạ, không có dấu ngoặc kép { Pháp: 68042591, Đức: 84270625, Na Uy: 5425270 }
- Ví dụ: cùng một bản đồ, trong "phong cách thu thập khối"
Pháp: 68042591 Đức:
84270625 Na Uy: 5425270



Kết hợp các loại

- Trong một chuỗi (hoặc ánh xạ), chúng ta có thể có các loại khác nhau (bao gồm các chuỗi hoặc ánh xạ khác) • Ví dụ: câu

hỏi: [tên, nhiệm vụ, màu yêu thích] câu trả lời: ["Arthur, Vua của người Anh", Chén Thánh, màu tím, 42]

- Lưu ý rằng chúng ta cần trích dẫn "Arthur" vì dấu phẩy • Lưu ý rằng chúng ta không có cùng số lượng phần tử trong các câu hỏi và câu trả lời



Nhiều kết hợp hơn

- Ví dụ: - dịch

```
vụ: cổng nginx : [ 80, 443 ] -  
    dịch vụ: cổng bind : [ 53/tcp, 53/  
udp ] - dịch vụ: cổng ssh : 22
```

- Lưu ý rằng các cổng không phải lúc nào cũng có cùng loại (mã xử lý dữ liệu đó có thể sẽ phải thông minh!)



Boolean tự động

mã:

pháp: fr đức:

de na uy: không

-

```
{  
  "mã": { "Pháp":  
    "fr", "Đức": "de", "Na  
    Uy": sai  
  }  
}
```





Boolean tự động

- no có thể trở thành false (phụ thuộc vào trình phân tích cú pháp YAML được sử dụng)
- Thay vào đó, nên trích dẫn:
mã:
pháp: fr đức:
de na uy: "không"



Phao tự động

phiên bản:

```
libfoo: 1.10 foctl:  
1.0
```

-

```
{  
  "phiên bản":  
    { "libfoo": 1.1,  
      "foctl": 1  
    }  
}
```





Phao tự động

- Các số không theo sau biến mất • Những

số này cũng nên được trích dẫn: `phiên bản: libfoo:`

```
"1.10" foctl:
```

```
"1.0"
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



Thời gian tự động

```
portmap: -
```

```
80:80 -
```

```
22:22
```

```
-
```

```
{
```

```
  "portmap":
```

```
    [ "80:80",
```

```
      1342
```

```
    ]
```

```
}
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY



Thời gian tự động

- 22:22 trở thành 1342
- Tức là 22 phút và 22 giây = 1342 giây
- Một lần nữa, nó nên được trích dẫn



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Bộ tách tài liệu

- Một tệp YAML có thể có nhiều tài liệu được phân tách bằng ---:

Đây là một tài liệu chỉ gồm một chuỗi ký tự.



tên: Loại tài liệu thứ hai : Đây là một

ánh xạ (khóa giá trị)



- Văn bản thứ ba

- Đây là một chuỗi

- Một số người thích thêm --- vào đầu và/hoặc cuối (không bắt buộc nhưng có thể hữu ích khi ghép nhiều tệp lại với nhau) .footnote[Bỏ qua phần

này; phần này ở đây để giải quyết vấn đề này.]



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chuỗi nhiều dòng

Hãy thử khối sau trong trình phân tích cú pháp

YAML: thêm ngắt dòng: "trong chuỗi được đặt trong dấu ngoặc kép\n(như thế này)" giữ nguyên

ngắt dòng: | bằng cách sử dụng dấu

gạch ngang

(|) (điều này

rất tuyệt để nhúng các tập lệnh shell,

các tệp cấu hình...) không giữ

nguyên ngắt dòng: > bằng cách sử dụng dấu lớn

hơn (>)

(điều này rất tuyệt khi

nhúng các dòng rất dài)

Xem <https://yaml-multiline.info/> để biết các mẹo nâng cao về nhiều dòng!

(Ví dụ: loại bỏ hoặc giữ lại các ký tự \n thừa ở cuối khối.)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Tính năng nâng cao

Mở neo cho phép bạn "ghi nhớ" và sử dụng lại nội dung:

debian: &debian gói:

deb latest-stable:

bullseye

cũng-debian: *debian

ubuntu: <<:

*debian bản ổn định

mới nhất: jammy



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

YAML, tốt hay xấu?

- Tiến trình tự nhiên từ XML đến JSON đến YAML • Có những ngôn ngữ dữ liệu khác ngoài kia (ví dụ HCL, những thứ dành riêng cho miền được tạo bằng Ruby, CUE.) • Có sự thỏa hiệp, ví dụ: • thân thiện với người dùng hơn nhiều “ma thuật” hơn với các tác dụng phụ • mạnh mẽ hơn đường cong học tập dốc hơn
- Dù bạn có thích hay ghét thì cũng nên hiểu nó! • Công cụ thú vị nếu bạn đánh giá cao YAML: <https://carvel.dev/ytt/>



Dockerfile = tuyệt vời để xây dựng một hình ảnh container.

Nếu chúng ta có nhiều container thì sao?

Nếu một số trong số chúng yêu cầu các tham số chạy docker cụ thể thì sao?

Làm sao chúng ta có thể kết nối tất cả chúng lại với nhau?

. Compose giải quyết những trường hợp sử dụng này (và một số trường hợp khác).



Cuộc sống trước khi Compose

Trước khi có Compose, chúng tôi thường viết các tập lệnh tùy chỉnh để:

- xây dựng hình ảnh container,
- chạy container

bằng những hình ảnh này,

- kết nối các container với nhau,
- xây dựng lại, khởi động lại, cập nhật những hình ảnh và container này.



Cuộc sống với Compose

Compose cho phép thực hiện quy trình hướng dẫn đơn giản và mạnh mẽ:

1. Kiểm tra mã của chúng tôi.

2. Chạy lệnh `docker-compose up`.

3. Ứng dụng của chúng tôi đã hoạt động!





soạn thảo



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Cuộc sống sau khi Compose

(Hoặc: khi nào chúng ta cần thứ gì khác?) •

Compose không phải là trình điều phối

- Nó không được thiết kế để cần chạy các container trên nhiều nút (tuy nhiên, nó có thể hoạt động với Docker Swarm Mode)
- Compose không lý tưởng nếu chúng ta muốn chạy container trên Kubernetes • nó sử dụng các khái niệm khác nhau (dịch vụ Compose dịch vụ Kubernetes) • nó cần Docker Engine (mặc dù có thể sẽ có hỗ trợ containerd)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Lần đầu tiên tham gia Compose

1. Viết Dockerfile
2. Mô tả ngăn xếp container của chúng tôi trong tệp YAML có tên là docker-compose.yml
3. docker-compose up (hoặc docker-compose up -d để chạy trong lý lịch)
4. Compose kéo và xây dựng các hình ảnh cần thiết và bắt đầu thùng chứa
5. Compose hiển thị nhật ký kết hợp của tất cả các container (nếu chạy ở chế độ nền, hãy sử dụng docker-compose logs)
6. Nhấn Ctrl-C để dừng toàn bộ ngăn xếp (nếu chạy ở chế độ nền, hãy sử dụng docker-compose stop)



Lặp lại

Sau khi thực hiện các thay đổi đối với mã nguồn, chúng ta

có thể: 1. docker-compose build để xây dựng lại hình ảnh container

2. docker-compose up để khởi động lại ngăn xếp bằng các hình ảnh mới Chúng ta

cũng có thể kết hợp cả hai bằng lệnh docker-compose up --build Compose sẽ thông minh hơn và chỉ tạo lại các container đã thay đổi.

Khi làm việc với các ngôn ngữ được thông dịch: •

không xây dựng lại mỗi lần •

thay vào đó hãy tận dụng một phần khối lượng



Ra mắt Stack đầu tiên của chúng tôi với Compose

Bước đầu tiên: sao chép mã nguồn của ứng dụng mà chúng ta sẽ làm việc.

```
git clone
```

```
https://github.com/jpetazzo/trainingwheels cd trainingwheels
```

Bước thứ hai: khởi động ứng dụng.

`docker-compose up` Xem Compose xây

dựng và chạy ứng dụng.

Ngăn xếp Compose đó hiển thị máy chủ web trên cổng 8000; hãy thử kết nối với cổng đó.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Ra mắt Stack đầu tiên của chúng tôi với Compose

Chúng ta sẽ thấy một trang web như thế này:

Mỗi lần nạp lại, bộ đếm sẽ tăng lên.

Training wheels

This request was served by 5457fb09c174.

5457fb09c174 served 1 request so far.

The current ladder is:

- 5457fb09c174 → 1 request

ứng dụng soạn thảo



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Dừng ứng dụng

Khi chúng ta nhấn Ctrl-C, Compose sẽ cố gắng kết thúc tất cả các vùng chứa một cách bình thường.

Sau mười giây (hoặc nếu chúng ta nhấn ^C lần nữa), nó sẽ buộc chúng phải chết.



Tập docker-compose.yml

Sau đây là tập được sử dụng trong bản demo:

```
phiên bản: "3"
```

```
dịch vụ:
```

```
  WWW:
```

```
    xây dựng: www cổng:
```

```
      - ${PORT-8000}:5000 người dùng:
```

```
      không ai môi trường:
```

```
      GỖ LỖI: 1
```

```
      lệnh: python counter.py khối lượng: - ./www/src
```

```
  làm lại:
```

```
    hình ảnh: redis
```



Soạn thảo cấu trúc tập tin

Một tệp Compose có nhiều phần:

- phiên bản là bắt buộc. (Thông thường sử dụng "3".)
- dịch vụ là bắt buộc. Mỗi dịch vụ tương ứng với một container.
- mạng là tùy chọn và chỉ ra các mạng mà container sẽ được kết nối. (Theo mặc định, container sẽ được kết nối trên mạng riêng, theo từng tệp soạn thảo.)
- khối lượng là tùy chọn và có thể xác định khối lượng được sử dụng và/hoặc chia sẻ bởi các vùng chứa.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Soạn thảo các phiên bản tập tin

- Phiên bản 1 là phiên bản cũ và không nên sử dụng.
(Nếu bạn thấy tệp Compose không có phiên bản và dịch vụ thì đó là tệp v1 cũ.)
- Phiên bản 2 đã bổ sung hỗ trợ cho mạng và khối lượng.
- Phiên bản 3 đã bổ sung hỗ trợ cho các tùy chọn triển khai (mở rộng quy mô, cập nhật liên tục, v.v.).
- Thông thường sử dụng phiên bản: "3".

Tài [liệu Docker](#) có thông tin tuyệt vời về định dạng tệp Compose nếu bạn cần biết thêm về các phiên bản.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Các container trong docker-compose.yml

Mỗi dịch vụ trong tập YAML phải chứa bản dựng hoặc hình ảnh.

- build biểu thị đường dẫn chứa Dockerfile.
- hình ảnh biểu thị tên hình ảnh (cục bộ hoặc trên sổ đăng ký).
- Nếu cả hai đều được chỉ định, một hình ảnh sẽ được xây dựng từ thư mục xây dựng và được đặt tên là image.

Các thông số khác là tùy chọn.

Họ mã hóa các tham số mà bạn thường thêm vào docker chạy.

Đôi khi chúng có một số cải tiến nhỏ.



Tham số container

- lệnh chỉ ra lệnh cần chạy (giống như CMD trong Dockerfile).
- ports dịch thành một (hoặc nhiều) tùy chọn -p để ánh xạ ports. Bạn có thể chỉ định các ports cục bộ (tức là x:y để hiển thị cổng công khai x).
- volumes dịch thành một (hoặc nhiều) tùy chọn -v. Bạn có thể sử dụng đường dẫn tương đối ở đây.

Để biết danh sách đầy đủ, hãy kiểm tra: <https://docs.docker.com/compose/compose-file/>



Biến môi trường

- Chúng ta có thể sử dụng các biến môi trường trong các tệp Compose (như `$THIS` hoặc `${THAT}`)
- Chúng ta có thể cung cấp các giá trị mặc định, ví dụ `$PORT-8000`
- Compose cũng sẽ tự động tải tệp môi trường `.env` (phải chứa `VAR=giá trị`, mỗi giá trị một dòng)
- Đây là một cách tuyệt vời để tùy chỉnh các tham số xây dựng và chạy (phiên bản hình ảnh cơ sở để sử dụng, bí mật xây dựng và chạy, số cổng.)



Cấu hình ngăn xếp Compose

- Thực hiện theo [nguyên tắc cấu hình ứng dụng 12 yếu tố](#) (cấu hình ứng dụng thông qua các biến môi trường)
- Cung cấp (trong kho lưu trữ) một tệp môi trường mặc định phù hợp cho quá trình phát triển (không có giá trị bí mật hoặc nhạy cảm)
- Sao chép tệp môi trường mặc định vào `.env` và chỉnh sửa nó (hoặc: cung cấp một tập lệnh để tạo `.env` từ một mẫu)



Chạy nhiều bản sao của một ngăn xếp

- Sao chép ngăn xếp vào hai thư mục khác nhau, ví dụ như front và frontcopy
- Soạn tiền tố hình ảnh và vùng chứa với tên thư mục:
front_www, front_www_1, front_db_1
frontcopy_www, frontcopy_www_1, frontcopy_db_1
- Hoặc, sử dụng docker-compose -p frontcopy
(để đặt --project-name của một ngăn xếp, mặc định là tên thư mục)
- Mỗi bản sao được tách biệt khỏi các bản sao khác (chạy trên một mạng khác nhau)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Kiểm tra trạng thái ngăn xếp

Chúng ta có ps, docker ps và tương tự như vậy, docker-compose ps:

```
$ docker-compose ps
```

| Tên | Yêu cầu | Tình trạng |
|---|------------------|----------------------|
| Cổng | | |
| ----- | | |
| trainingwheels_redis_1 /entrypoint.sh red | Lên 6379/tcp | trainingwheels_www_1 |
| 0.0.0.0:8000- | | |
| >5000/tcp | bộ đếm python.py | Hưởng lên |

Hiển thị trạng thái của tất cả các container trong ngăn xếp của chúng ta.

Không hiển thị các thùng chứa khác.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Dọn dẹp (1)

Nếu bạn đã khởi động ứng dụng của mình ở chế độ nền bằng Compose và muốn dừng nó một cách dễ dàng, bạn có thể sử dụng lệnh kill:

```
$ docker-compose giết
```

Tương tự như vậy, docker-compose rm sẽ cho phép bạn xóa các container (sau khi xác nhận):

```
$ docker-compose rm
```

```
Sẽ xóa trainingwheels_redis_1, trainingwheels_www_1
```

```
Bạn có chắc chắn không? [yN] y
```

```
Đang xóa trainingwheels_redis_1...
```

```
Đang xóa trainingwheels_www_1...
```



Dọn dẹp (2)

Ngoài ra, lệnh docker-compose down sẽ dừng và xóa các container.

Nó cũng sẽ xóa các tài nguyên khác, như mạng được tạo cho ứng dụng.

```
$ docker-compose xuống
```

```
Đang dừng trainingwheels_www_1 ... xong
```

```
Đang dừng trainingwheels_redis_1 ... xong
```

```
Xóa trainingwheels_www_1 ... xong
```

```
Xóa trainingwheels_redis_1 ... xong
```

Sử dụng docker-compose down -v để xóa mọi thứ bao gồm cả ổ đĩa.



Xử lý đặc biệt khối lượng

- Khi một hình ảnh được cập nhật, Compose sẽ tự động tạo một hình ảnh mới thùng chứa
- Dữ liệu trong container cũ bị mất.
- .Trừ khi container đang sử dụng một ổ đĩa •

Compose sau đó sẽ gắn lại ổ đĩa đó vào container mới (và dữ liệu sau đó được lưu giữ trong suốt quá trình nâng cấp cơ sở dữ liệu)

- Tất cả các hình ảnh cơ sở dữ liệu tốt đều sử dụng khối lượng
(ví dụ tất cả hình ảnh chính thức)



Gotchas với khối lượng

- Thật không may, các ổ đĩa Docker không có nhãn hoặc siêu dữ liệu • Compose theo dõi các ổ đĩa nhờ vào vùng chứa được liên kết của chúng • Nếu vùng chứa bị xóa, ổ đĩa sẽ bị mồ côi • Ví dụ: docker-compose down && docker-compose up
 - ổ đĩa cũ vẫn tồn tại, tách khỏi vùng chứa của nó • một ổ đĩa mới được tạo • docker-compose down -v/--volumes xóa các ổ đĩa
(nhưng không phải docker-compose down && docker-compose down -v!)



Quản lý khối lượng một cách rõ ràng

Tùy chọn 1: các tập được đặt tên

dịch vụ:

ứng

dùng: khối

lượng: - dữ liệu:/some/path khối

lượng: dữ liệu:

- Ổ đĩa sẽ được đặt tên là <project>_data • Nó sẽ không bị mờ với docker-compose down • Nó sẽ được xóa chính xác với docker-compose down -v



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Quản lý khối lượng một cách rõ ràng

Tùy chọn 2: đường dẫn tương đối

dịch vụ:

ứng

dùng: khối

lượng: - ./data:/some/path

- Giúp dễ dàng đặt chung ứng dụng và dữ liệu của ứng dụng (để di chuyển, sao lưu, thống kê mức sử dụng đĩa.) • Sẽ không bị xóa bởi docker-compose down -v



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Quản lý các ngăn xếp phức tạp

- Compose cung cấp nhiều tính năng để quản lý các ngăn xếp phức tạp (có nhiều vùng chứa)
- `-f/--file/$COMPOSE_FILE` có thể là danh sách các tệp Compose (phân cách bằng `:` và được hợp nhất với nhau)
- Dịch vụ có thể được gán cho một hoặc nhiều cấu hình • `--profile/$COMPOSE_PROFILE` có thể là danh sách các cấu hình được phân tách bằng dấu phẩy (xem [mục Sử dụng cấu hình dịch vụ](#) trong tài liệu Compose)
- Các biến này có thể được thiết lập trong `.env`



Phụ thuộc

- Một dịch vụ có thể có phần phụ thuộc (liệt kê một hoặc nhiều dịch vụ khác) • Phần này được sử dụng khi đưa ra các dịch vụ riêng lẻ (ví dụ `docker-compose up blah` hoặc `docker-compose run foo`)
- ⚠ Nó không khiến cho một dịch vụ phải “chờ” một dịch vụ khác hoạt động!



Một chút lịch sử và thông tin thú vị

- Compose ban đầu có tên là “Fig” •

Compose là một trong số ít thành phần của Docker được viết bằng Python (hầu hết mọi thứ khác đều bằng Go) • Vào

năm 2020, Docker đã giới thiệu “Compose CLI”: • lệnh

`docker compose` để triển khai các ngăn xếp Compose lên một số đám mây • dần dần đạt được tính năng tương đương với `docker-compose` • cũng cung cấp nhiều cải tiến (ví dụ: tận dụng BuildKit theo mặc định)



Hãy cùng viết một tệp Compose cho ứng dụng Wordsmith!

Mã ở đây: <https://github.com/jpetazzo/wordsmith> # Giảm kích thước hình ảnh



- Trong ví dụ trước, hình ảnh cuối cùng của chúng tôi bao gồm:
 - chương trình hello của chúng ta
 - mã nguồn của nó
 - trình biên dịch
- Chỉ có chương trình đầu tiên là bắt buộc.
- Chúng ta sẽ xem cách lấy hình ảnh mà không cần các thành phần thừa.



Chúng ta không thể xóa các tập tin thừa bằng RUN được sao?

Điều gì xảy ra nếu chúng ta thực hiện một trong các lệnh sau?

- CHẠY `rm -rf` ...
- CHẠY `apt-get remove` ...
- CHẠY làm sạch ...
-

Thao tác này sẽ thêm một lớp để loại bỏ một loạt các tập tin.

Nhưng các lớp trước đó (đã thêm các tập) vẫn tồn tại.



Xóa các tập tin có thêm một lớp

Khi tải xuống hình ảnh, tất cả các lớp phải được tải xuống.

| Hướng dẫn Dockerfile | Kích thước lớp | Kích thước hình ảnh |
|---------------------------------|--|--|
| TỪ ubuntu | Kích thước của hình ảnh cơ sở | Kích thước của hình ảnh cơ sở |
| ... | . | Tổng của lớp này + tất cả các lớp trước đó |
| CHẠY apt-get cài đặt một số gói | Kích thước của các tập tin được thêm vào (ví dụ: vài MB) | Tổng của lớp này + tất cả các lớp trước đó |
| ... | . | Tổng của lớp này + tất cả các lớp trước đó |
| CHẠY apt-get loại bỏ một số gói | Gần như bằng không (chỉ có siêu dữ liệu) | Giống như cái trước |

Do đó, lệnh RUN rm không làm giảm kích thước của ảnh hoặc giải phóng dung lượng đĩa.

Xóa các tập tin không cần thiết

Có nhiều kỹ thuật khác nhau để có được hình ảnh nhỏ hơn: • thu gọn các lớp, • thêm các tệp nhị phân được xây dựng bên ngoài Dockerfile, • nén hình ảnh cuối cùng, • xây dựng nhiều giai đoạn.

Chúng ta hãy cùng xem qua chúng một cách nhanh chóng.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Các lớp súp đổ

Bạn sẽ thường xuyên thấy Dockerfiles như thế này:

```
TỪ ubuntu CHẠY apt-get  
update && apt-get install xxx && ... && apt-get remove xxx && ...
```

Hoặc biến thể (dễ đọc hơn):

```
TỪ ubuntu CHẠY apt-get  
update \ && apt-get install xxx \ && ...  
  \ && apt-get remove xxx \  
  
&& ...
```

Lệnh RUN này cung cấp cho chúng ta một lớp duy nhất.

Các tệp được thêm vào rồi xóa trong cùng một lớp sẽ không làm tăng kích thước lớp.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Các lớp thu gọn: ưu và nhược điểm

Ưu điểm:

- hoạt động trên tất cả các phiên bản của Docker
- không yêu cầu thêm công cụ

Nhược điểm:

- không dễ đọc lắm •

một số tệp không cần thiết vẫn có thể còn sót lại nếu việc dọn dẹp không được thực hiện kỹ lưỡng • lớp đó tốn kém (xây dựng chậm)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Xây dựng các tệp nhị phân bên ngoài Dockerfile

Điều này tạo ra Dockerfile trông như thế này:

```
TỪ ubuntu SAO CHÉP
```

```
xxx /usr/local/bin Tất nhiên, điều này ngụ ý
```

rằng tệp xxx tồn tại trong ngữ cảnh xây dựng.

Tệp đó phải tồn tại trước khi bạn có thể chạy lệnh docker build.

Ví dụ, nó có thể: •

tồn tại trong kho lưu trữ mã, •

được tạo bởi một công cụ khác (script, Makefile.), •

được tạo bởi một hình ảnh chứa khác và được trích xuất từ hình ảnh.

Ví dụ, hãy xem [hình ảnh chính thức của busybox](#) hoặc [hình ảnh busybox cũ hơn này](#).



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Xây dựng nhị phân bên ngoài: ưu và nhược điểm

Ưu điểm:

- hình ảnh cuối cùng có thể rất nhỏ

Nhược điểm:

- yêu cầu một công cụ xây dựng bổ

sung • chúng tôi quay lại địa ngục phụ thuộc và "hoạt động trên máy của tôi"

Nhược điểm, nếu nhị phân được thêm vào kho lưu trữ mã:

- phá vỡ khả năng di động trên các nền tảng khác nhau •

làm tăng đáng kể kích thước kho lưu trữ nếu nhị phân được cập nhật thường xuyên



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Làm bẹp hình ảnh cuối cùng

Ý tưởng là chuyển đổi hình ảnh cuối cùng thành hình ảnh một lớp.

Điều này có thể được thực hiện theo (ít nhất) hai

cách. • Kích hoạt các tính năng thử nghiệm và nén hình ảnh cuối cùng:

```
docker image build --squash ...
```

- Xuất/nhập hình ảnh cuối cùng.

```
docker build -t temp-image .
```

```
docker run --entrypoint true --name temp-container temp-image docker  
export
```

```
temp-container | docker import - final-image docker rm temp-container  
docker rmi temp-image
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Làm méo hình ảnh: ưu và nhược điểm

Ưu điểm:

- hình ảnh một lớp nhỏ hơn và tải xuống nhanh hơn • các tệp đã xóa không còn chiếm dung lượng lưu trữ và tài nguyên mạng

Nhược điểm:

- chúng ta vẫn cần phải chủ động xóa các tệp tin không cần thiết • hoạt động squash có thể mất nhiều thời gian (trên các hình ảnh lớn) • hoạt động squash không được hưởng lợi từ bộ nhớ đệm (ngay cả khi chúng ta chỉ thay đổi một tệp tin nhỏ, toàn bộ hình ảnh cần phải được squash lại)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Xây dựng nhiều giai đoạn

Xây dựng nhiều giai đoạn cho phép chúng ta có nhiều giai đoạn.

Mỗi giai đoạn là một hình ảnh riêng biệt và có thể sao chép các tệp tin từ các giai đoạn trước.

Chúng ta sẽ xem cách chúng hoạt động chi tiết hơn.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Xây dựng nhiều giai đoạn



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

- Tại bất kỳ thời điểm nào trong Dockerfile, chúng ta có thể thêm một dòng FROM mới.
- Dòng này bắt đầu một giai đoạn xây dựng mới của chúng ta.
- Mỗi giai đoạn có thể truy cập các tập tin của các giai đoạn trước đó bằng lệnh COPY --from=....
- Khi bản dựng được gắn thẻ (với docker build -t ...), giai đoạn cuối cùng sẽ được gắn thẻ.
- Các giai đoạn trước đó không bị loại bỏ: chúng sẽ được sử dụng để lưu trữ đệm và có thể được tham khảo.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Xây dựng nhiều giai đoạn trong thực tế

- Mỗi giai đoạn được đánh số, bắt đầu từ 0 •

Chúng ta có thể sao chép một tệp từ giai đoạn trước bằng cách chỉ ra số của nó, ví dụ:

```
COPY --from=0 /file/from/first/stage /location/in/  
current/stage
```

- Chúng ta cũng có thể đặt tên cho các giai đoạn và tham chiếu đến các tên này:

```
TỪ golang AS builder CHẠY ...
```

```
TỪ alpine COPY  
--from=builder /go/bin/mylittlebinary /usr/local/bin/
```



Xây dựng nhiều giai đoạn cho chương trình C của chúng tôi

Chúng ta sẽ thay đổi Dockerfile thành: • đặt biệt

danh cho giai đoạn đầu tiên: trình biên dịch • thêm giai đoạn thứ hai

sử dụng cùng một ảnh cơ sở ubuntu • thêm nhị phân hello vào giai đoạn thứ hai • đảm

bảo rằng CMD đang ở giai đoạn thứ hai Dockerfile kết quả sẽ nằm ở

trang chiếu tiếp theo.



Dockerfile xây dựng nhiều giai đoạn

Sau đây là Dockerfile cuối cùng:

```
TỪ ubuntu AS trình biên dịch CHẠY apt-get
update CHẠY apt-get install -y
build-essential SAO CHÉP hello.c / CHẠY make hello TỪ ubuntu SAO CHÉP --
from=compiler /hello /
hello CMD /hello
```

Hãy xây dựng nó và kiểm tra xem nó có hoạt động chính xác

```
không: docker build -t hellomultistage . docker run
hellomultistage
```



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

So sánh kích thước hình ảnh xây dựng một giai đoạn/nhiều giai đoạn

Liệt kê các hình ảnh của chúng ta với hình ảnh docker và kiểm tra kích thước của:

hình ảnh cơ sở ubuntu, • hình ảnh

hello một giai đoạn, • hình ảnh

hellomultistage nhiều giai đoạn.

Chúng ta có thể tạo ra những hình ảnh thậm chí còn nhỏ hơn nếu sử dụng những hình ảnh cơ sở nhỏ hơn.

Tuy nhiên, nếu chúng ta sử dụng các hình ảnh cơ sở chung (ví dụ nếu chúng ta chuẩn hóa trên Ubuntu), các hình ảnh chung này sẽ chỉ được kéo một lần cho mỗi nút, do đó chúng về cơ bản là "miễn phí".



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Xây dựng mục tiêu

- Chúng ta cũng có thể gán thẻ một giai đoạn trung gian bằng lệnh sau:
`docker build --target GIAI ĐOẠN --tag TÊN`
- Điều này sẽ tạo ra một hình ảnh (có tên là NAME) tương ứng với giai đoạn STAGE • Điều này có thể được sử dụng để dễ dàng truy cập vào một giai đoạn trung gian để kiểm tra (thay vì phân tích cú pháp đầu ra của docker build để tìm ra ID hình ảnh)
- Điều này cũng có thể được sử dụng để mô tả nhiều hình ảnh từ một Tập Docker (thay vì sử dụng nhiều Dockerfile, có thể không đồng bộ)

-



Xử lý bộ nhớ đệm tải xuống

- Trong một số trường hợp, hình ảnh của chúng tôi chứa các tệp được tải xuống tạm thời hoặc bộ nhớ đệm (ví dụ: các gói được tải xuống bởi pip, Maven, v.v.) • Đôi khi có thể vô hiệu hóa những thứ này (ví dụ `pip install --no-cache-dir ...`)
- Bộ nhớ đệm cũng có thể được dọn sạch ngay sau khi cài đặt (ví dụ: `pip install ... && rm -rf ~/.cache/pip`)



Tải xuống bộ nhớ đệm và bản dựng nhiều giai đoạn

- Tải xuống + cài đặt các gói trong giai đoạn xây dựng
- Sao chép các gói đã cài đặt vào giai đoạn chạy • Ví

dụ: trong trường hợp cụ thể của Python, hãy sử dụng môi trường ảo (cài đặt trong môi trường ảo; sau đó sao chép thư mục môi trường ảo)



Tải xuống bộ nhớ đệm và BuildKit

- BuildKit có tính năng lưu trữ đệm cho các giai đoạn chạy
- Nó có thể xử lý bộ đệm tải xuống một cách tinh tế
- Ví dụ:

```
RUN --mount=type=cache,target=/pipcache pip install -- cache-dir /pipcache ...
```

- Bộ nhớ đệm sẽ không có trong hình ảnh cuối cùng, nhưng nó sẽ tồn tại trong suốt quá trình xây dựng



Chúng tôi đã xây dựng được những hình ảnh đầu tiên.

Bây giờ chúng ta có thể xuất bản nó lên Docker Hub!

Bạn không cần phải thực hiện các bài tập trong phần này vì chúng yêu cầu phải có tài khoản trên Docker Hub và chúng tôi không muốn ép buộc bất kỳ ai phải tạo một.

Tuy nhiên, xin lưu ý rằng việc tạo tài khoản trên Docker Hub là miễn phí (và không yêu cầu thẻ tín dụng) và việc lưu trữ hình ảnh công khai cũng miễn phí.



Đăng nhập vào tài khoản Docker Hub của chúng tôi

- Điều này có thể được thực hiện từ Docker CLI:

đăng nhập docker

.warning[Khi chạy Docker cho Mac/Windows hoặc Docker trên máy trạm Linux, nó có thể (và sẽ khi có thể) tích hợp với vòng chìa khóa của hệ thống để lưu trữ thông tin xác thực của bạn một cách an toàn. Tuy nhiên, trên hầu hết các máy chủ Linux, nó sẽ lưu trữ thông tin xác thực của bạn trong ~/.docker/config.]



Thẻ hình ảnh và địa chỉ đăng ký

- Thẻ hình ảnh Docker giống như thẻ và nhánh Git. •

Chúng giống như dấu trang trỏ đến một ID hình ảnh cụ thể.

- Việc gắn thẻ một hình ảnh không đổi tên hình ảnh: nó thêm một

thẻ khác. • Khi đẩy hình ảnh vào sổ đăng ký, địa chỉ sổ đăng ký sẽ nằm trong th

Ví dụ: `registry.example.net:5000/image`

- Còn hình ảnh Docker Hub thì sao?

–

- `jpetazzo/clock` thực chất là
`index.docker.io/jpetazzo/clock`

- `ubuntu` thực chất là `library/ubuntu`, tức
là `index.docker.io/library/ubuntu`



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Đánh dấu một hình ảnh để đẩy nó lên Hub

- Hãy gắn thẻ hình ảnh `figlet` của chúng ta (hoặc bất kỳ hình ảnh nào khác
mà chúng ta thích): `docker tag figlet jpetazzo/figlet`

- Và đẩy nó vào Hub: `docker
push jpetazzo/figlet`

- Vậy là xong!

–

- Bây giờ bất kỳ ai cũng có thể chạy `jpetazzo/figlet` ở bất kỳ đâu.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Sự tốt đẹp của việc xây dựng tự động

- Bạn có thể liên kết kho lưu trữ Docker Hub với kho lưu trữ GitHub hoặc BitBucket
- Mỗi lần đẩy lên GitHub hoặc BitBucket sẽ kích hoạt bản dựng trên Docker Hub
- Nếu bản dựng thành công, hình ảnh mới sẽ có trên Docker Hub
- Bạn có thể ánh xạ thẻ và nhánh giữa hình ảnh nguồn và hình ảnh vùng chứa
- Nếu bạn làm việc với kho lưu trữ công khai, điều này miễn phí



Thiết lập bản dựng tự động

- Chúng ta cần một kho lưu trữ Dockerized!
- Hãy truy cập <https://github.com/jpetazzo/trainingwheels> và fork nó.
- Truy cập Docker Hub (<https://hub.docker.com/>) và đăng nhập. Chọn "Repositories" trong màu xanh lam menu điều hướng.
- Chọn "Create" ở thanh trên cùng bên phải và chọn "Create Repository+".
- Kết nối tài khoản Docker Hub của bạn với tài khoản GitHub.
- Nhấp vào nút "Tạo".
- Sau đó vào thư mục "Builds".
- Nhấp vào biểu tượng Github và chọn người dùng và kho lưu trữ mà chúng ta vừa fork.
- Trong khối "Build rules" gần cuối trang, đặt /www vào cột "Build Context" (hoặc bất kỳ thư mục nào mà Dockerfile nằm trong).
- Nhấp vào "Lưu và xây dựng" để xây dựng kho lưu trữ ngay lập tức (không cần chờ git push).
- Các bản dựng tiếp theo sẽ diễn ra tự động, nhờ vào GitHub hooks.



Xây dựng trên bay

- Một số dịch vụ có thể xây dựng hình ảnh ngay lập tức từ kho lưu trữ • Ví dụ: _____

`ctr.run .lab` • Sử dụng `ctr.run` để tự động xây dựng hình ảnh chứa và chạy nó:
`docker chạy ctr.run/github.com/undefinedlabs/hello-world`

]

Có thể phải mất một khoảng dừng dài trước khi lớp đầu tiên được kéo, vì API đăng sau lệnh `docker pull` không cho phép truyền phát nhật ký xây dựng và không có phản hồi trong quá trình xây dựng.

Bạn có thể xem nhật ký xây dựng bằng cách thiết lập tài khoản trên ctr.run.



Chúng ta sẽ xem cách:

- Giảm số lượng lớp. • Tận dụng bộ đệm dựng để dựng nhanh hơn. • Nhúng thử nghiệm đơn vị vào quy trình dựng.



Giảm số lượng lớp

- Mỗi dòng trong Dockerfile tạo ra một lớp mới.
- Xây dựng Dockerfile của bạn để tận dụng hệ thống lưu trữ đệm của Docker.
- Kết hợp các lệnh bằng cách sử dụng `&&` để tiếp tục lệnh và `\` để ngắt dòng các dòng.

Lưu ý: Dockerfile thường được xây dựng theo từng dòng:

```
CHẠY apt-get cài đặt thisthing
```

```
CHẠY apt-get install vàthatthing vàthatotherone
```

```
CHẠY apt-get cài đặt somemorestuff
```

Và sau đó chỉnh sửa lại một cách đơn giản trước khi xuất xưởng:

```
CHẠY apt-get cài đặt cái này cái kia và cái kia cái kia cái kia nữa
```



Tránh cài đặt lại các phụ thuộc ở mỗi lần xây dựng

- Vấn đề Dockerfile cổ điển:

“Mỗi lần tôi thay đổi một dòng mã, tất cả các phần phụ thuộc của tôi đều được cài đặt lại!”

- Giải pháp: SAO CHÉP danh sách các phụ thuộc (package.json, requirements.txt, v.v.) riêng lẻ để tránh phải cài đặt lại các phụ thuộc không thay đổi mỗi lần.



Ví dụ Dockerfile "xấu"

Các phụ thuộc được cài đặt lại mỗi lần, vì hệ thống xây dựng không biết liệu requirements.txt đã được cập nhật hay chưa.

```
TỪ python WORKDIR /  
src COPY .
```

```
CHẠY pip install -qr requirements.txt EXPOSE 5000 CMD  
["python",  
"app.py"]
```



Đã sửa Dockerfile

Việc thêm các phụ thuộc dưới dạng một bước riêng biệt có nghĩa là Docker có thể lưu trữ đệm hiệu quả hơn và chỉ cài đặt chúng khi requirements.txt thay đổi.

```
TỪ python WORKDIR /  
src COPY requirements.txt .
```

```
CHẠY pip install -qr requirements.txt SAO CHÉP.
```

```
PHÁT HIỆN 5000 CMD  
["python", "app.py"]
```



Hãy cẩn thận với `chown`, `chmod`, `mv`

- Các lớp không thể lưu trữ hiệu quả các thay đổi về quyền hoặc quyền sở hữu.
- Các lớp cũng không thể biểu diễn hiệu quả thời điểm một tệp được di chuyển.
- Do đó, các hoạt động như `chown`, `chmod`, `mv` có thể tốn kém.
- Ví dụ, trong đoạn mã Dockerfile bên dưới, mỗi dòng `RUN` tạo một lớp với toàn bộ bản sao của `some-file`.

SAO CHÉP một số tệp tin.

```
CHẠY chown www-data:www-data some-file CHẠY chmod 644
some-file CHẠY mv some-file /var/
WWW
```

- Làm sao chúng ta có thể tránh được điều đó?



Đặt các tệp tin vào đúng vị trí

- Thay vì sử dụng `mv`, hãy đặt trực tiếp các tệp vào đúng vị trí.

Khi giải nén các tệp lưu trữ (`tar`, `zip`), hãy hợp nhất các thao tác trong một lớp duy nhất.

Ví dụ:

```
...
CHẠY wget http://.../foo.tar.gz \ && tar -zxvf
foo.tar.gz \ && mv foo/fooctl /usr/
local/bin \ && rm -rf foo foo.tar.gz
...
```



Sử dụng COPY --chown

- Lệnh COPY của Dockerfile có thể sử dụng tham số --chown.

Ví dụ:

...

`COPY --chown=1000 một_số_tệp.`

`SAO CHÉP --chown=1000:1000 một_số_tập_tin.`

`SAO CHÉP --chown=www-data:www-data một_số_tệp.`

- Cờ --chown có thể chỉ định người dùng hoặc cặp người dùng:nhóm.
- Người dùng và nhóm có thể được chỉ định dưới dạng tên hoặc số.
- Khi sử dụng tên, tên phải tồn tại trong /etc/passwd hoặc /etc/group.

(Trong thùng chứa, không phải trên máy chủ!)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thiết lập quyền chính xác cục bộ

- Thay vì sử dụng chmod, hãy thiết lập quyền tệp cục bộ phù hợp.
- Khi sao chép tệp bằng COPY, quyền sẽ được giữ nguyên.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Nhúng các bài kiểm tra đơn vị vào quá trình xây dựng

```
TỪ <hình ảnh cơ sở > CHẠY <cài  
đặt phụ thuộc> SAO CHÉP < mã> CHẠY < mã xây dựng >  
CHẠY <cài đặt phụ  
thuộc thử nghiệm > SAO CHÉP <bộ  
dữ liệu thử nghiệm và đồ đạc> CHẠY <kiểm tra đơn vị> TỪ <hình  
ảnh cơ sở > CHẠY <cài đặt phụ thuộc> SAO CHÉP < mã> CHẠY <mã xây  
dựng> CMD, EXPOSE ...
```

- Việc xây dựng sẽ thất bại ngay khi một lệnh không thành công
- Nếu RUN <kiểm tra đơn vị> không thành công, bản dựng sẽ không tạo ra hình ảnh • Nếu thành công,
nó sẽ tạo ra một hình ảnh sạch (không có thư viện kiểm tra và dữ liệu)



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Ví dụ Dockerfile



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Có một số mẹo, thủ thuật và kỹ thuật mà chúng ta có thể sử dụng trong Dockerfiles.

Nhưng đôi khi, chúng ta phải sử dụng các phương pháp khác nhau (thậm chí là đối lập) tùy

thuộc vào: • mức độ phức tạp của

dự án, • ngôn ngữ lập trình hoặc khuôn khổ mà chúng ta đang sử

dụng, • giai đoạn của dự án (MVP ban đầu so với sản xuất siêu ổn định),

• liệu chúng ta đang xây dựng hình ảnh cuối cùng hay cơ sở cho các hình ảnh tiếp theo

• vân vân.

Chúng tôi sẽ trình bày một số ví dụ sử dụng các kỹ thuật rất khác nhau.



Khi nào cần tối ưu hóa hình ảnh

Khi tạo hình ảnh chính thức, bạn nên giảm càng nhiều càng tốt: • số lượng lớp, • kích

thước của hình ảnh cuối

cùng.

Điều này thường phải đánh đổi bằng thời gian xây dựng và sự tiện lợi cho người duy trì hình ảnh; nhưng khi một hình ảnh được tải xuống hàng triệu lần, thì việc tiết kiệm ngay cả vài giây thời gian kéo cũng có thể đáng giá.

```
CHẠY apt-get update && apt-get install -y libpng12-dev libjpeg-dev && rm -rf /var/lib/apt/lists/* \ && docker-php-ext-  
configure gd --with-png-dir=/usr --  
with-jpeg-dir=/usr \ && docker-php-ext-install gd
```

...

```
CHẠY curl -o wordpress.tar.gz -SL https://wordpress.org/wordpress- ${WORDPRESS_UPSTREAM_VERSION}.tar.gz  
\ && echo "$WORDPRESS_SHA1 *wordpress.tar.gz" | shasum -c  
- \ && tar -xzf wordpress.tar.gz -C /usr/src/ \ && rm wordpress.tar.gz \ && chown -R www-  
data:www-data /usr/src/wordpress
```

(Nguồn: [Hình ảnh chính thức của Wordpress](#))



Khi nào không nên tối ưu hóa hình ảnh

Đôi khi, tốt hơn là ưu tiên sự tiện lợi của người bảo trì.

Đặc biệt, nếu: •

hình ảnh thay đổi nhiều, •

hình ảnh có rất ít người dùng (ví dụ chỉ có 1 người, người bảo trì!),

• hình ảnh được xây dựng và chạy trên cùng một máy, •

hình ảnh được xây dựng và chạy trên các máy có liên kết rất nhanh .

Trong những trường hợp này, chỉ cần giữ mọi thứ đơn giản!

(Slide tiếp theo: Dockerfile có thể được sử dụng để xem trước trang web Jekyll / github.)



TỪ debian:sid

```
CHẠY apt-get update -q
CHẠY apt-get install -yq build-essential make
CHẠY apt-get install -yq zlib1g-dev
CHẠY apt-get install -yq ruby ruby-dev
CHẠY apt-get install -yq python-pygments
CHẠY apt-get install -yq nodejs
CHẠY apt-get install -yq cmake
CHẠY cài đặt đá quý --no -rdoc --no-ri github-pages
```

```
SAO CHÉP . /blog
WORKDIR /blog
```

TẬP /blog/_site

PHÁT HIỆN 4000 CMD

```
["jekyll", "serve", "--host", "0.0.0.0", "--incremental"]
```



Hệ thống phiên bản đa chiều

Hình ảnh có thể có thể để chỉ ra phiên bản của hình ảnh.

Nhưng đôi khi, có nhiều thành phần quan trọng và chúng ta cần chỉ ra phiên bản cho tất cả các thành phần đó.

Điều này có thể được thực hiện bằng các biến môi trường:

```
ENV PIP=9.0.3 \  
    ZC_BUILDOUT=2.11.2 \  
    SETUPTOOLS=38.7.0 \ PLONE_MAJOR=5.1  
    \ PLONE_VERSION=5.1.0 \  
    PLONE_MD5=76dc6cfc1c749d763c32fff3a9870d8d  
(Nguồn: Hình ảnh chính thức của Plone)
```



Điểm vào và wrapper

Việc xác định điểm vào tùy chỉnh là rất phổ biến.

Điểm vào đó thường sẽ là một tập lệnh, thực hiện bất kỳ sự kết hợp nào sau đây:

- kiểm tra trước khi bay (nếu không có phụ thuộc bắt buộc, hiển thị thông báo lỗi sớm thay vì thông báo lỗi khó hiểu trong tệp nhật ký sâu),
- tạo hoặc xác thực tệp cấu hình,
- xóa đặc quyền (ví dụ với su hoặc gosu, đôi khi kết hợp với chown),
- và nhiều hơn nữa.



Một tập lệnh điểm vào điển hình

```
#!/bin/sh đặt
-e
```

```
# đối số đầu tiên là '-f' hoặc '--some-option' # hoặc đối số
đầu tiên là 'something.conf' nếu [ "${1#-}" != "$1" ] ||
[ "${1%.conf}" != "$1" ]; sau đó đặt -- redis-server "$@"
```

có

```
# cho phép container được bắt đầu với '--user' nếu [ "$1" = 'redis-server' -a "$(id
-u)" = '0' ]; sau đó
chown -R redis .exec su-
exec redis "$0" "$@"
```

có

thực hiện "\$@"

(Nguồn: [Hình ảnh chính thức của Redis](#))



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Phân tích thông tin

Để thuận tiện cho việc bảo trì (và tránh lỗi của con người), hãy tránh lặp lại thông tin như:

- số phiên bản,
- URL tải sản từ xa (ví dụ: tarball nguồn) .

Thay vào đó, hãy sử dụng biến môi trường.

```
PHIÊN BẢN NÚT ENV 10.2.1
...
CHẠY ...
    && curl -fsSLO --compressed "https://nodejs.org/dist/v$NODE_VERSION/node-
v$NODE_VERSION.tar.xz" \ && curl
    -fsSLO --đã nén "https://nodejs.org/dist/
v$NODE_VERSION/SHASUMS256.txt.asc" \
    && gpg --batch --decrypt --output SHASUMS256.txt SHASUMS256.txt.asc \ "node -v$NODE_VERSION.tar.xz\ $"
    SHASUMS256.txt | sha256sum -c - \ && grep && tar -xf "node-v$NODE_VERSION.tar.xz " \ && cd "node-
v$NODE_VERSION" \
...

```

(Nguồn: [Hình ảnh chính thức của Nodejs](#))



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Ghi đề

Về lý thuyết, hình ảnh phát triển và hình ảnh sản xuất phải giống nhau.

Trong thực tế, chúng ta thường cần kích hoạt các hành vi cụ thể trong quá trình phát triển (ví dụ: câu lệnh gỡ lỗi).

Một cách để dung hòa cả hai nhu cầu này là sử dụng Compose để kích hoạt các hành vi này.

Hãy cùng xem ứng dụng demo [trainingwheels](#) làm ví dụ.



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Hình ảnh sản xuất

Dockerfile này xây dựng một hình ảnh tận dụng gunicorn: **FROM**

```
python RUN pip  
install flask RUN pip install  
gunicorn RUN pip install redis COPY . /  
src WORKDIR /src CMD gunicorn --  
bind 0.0.0.0:5000  
--workers 10  
counter:app EXPOSE 5000
```

(Nguồn: [trainingwheels Dockerfile](#))



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Phát triển tập tin Compose

Tập Compose này sử dụng cùng một hình ảnh, nhưng có một vài ghi đề cho quá trình phát triển: • máy chủ

phát triển Flask được sử dụng (ghi đề CMD), • biến môi trường DEBUG được

đặt,

• khối lượng được sử dụng để cung cấp quy trình phát triển cục bộ nhanh hơn.

dịch vụ:

www:

xây dựng: `www` cổng:

- 8000:5000

người dùng: không

ai môi trường:

GỖ LỖI: 1

lệnh: `python counter.py` khối lượng: - `./www:/src`

(Nguồn: [tập tin Compose trainingwheels](#))



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Làm sao để biết phương pháp nào tốt hơn?

• Mục tiêu chính của container là giúp cuộc sống của chúng ta dễ dàng hơn. • Trong chương này, chúng tôi đã chỉ ra nhiều cách để viết Dockerfile. • Các Dockerfile này đôi khi sử dụng các kỹ thuật đối lập nhau. • Tuy nhiên, chúng là những kỹ thuật "phù hợp" cho một tình huống cụ thể. • Bắt đầu đơn giản và phát triển khi cần thiết là điều bình thường (thậm chí được khuyến khích). • Hãy thoải mái xem lại chương này sau (sau khi viết một vài Dockerfile) để lấy cảm hứng!



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Hãy cập nhật Dockerfiles để tận dụng các bản dựng nhiều giai đoạn!

Mã ở đây: <https://github.com/jpetazzo/wordsmith> Sử dụng thẻ

khác cho những hình ảnh này để chúng ta có thể so sánh kích thước của chúng.

Sự khác biệt về kích thước giữa bản dựng một giai đoạn và nhiều giai đoạn là gì?

